

System 3: Towards Fluent Autonomy

Trust Chains, Agent Autonomy, and the Architecture of AI That Works

Hani M.M. Al-Shater

August 2026

Contents

Preface	1
Chapter 1: Why I’m Betting on AI Agents	2
The Lesson We Keep Missing	3
When Search Moved Up a Level	4
What Are We Controlling Now?	6
The Terrifying Part	7
Why I’m Still Betting on This	8
Where We Go Next	8
Chapter 2: The Algorithm Vortex	10
The Running Example: Circle Packing	11
First Idea: Hill Climbing	12
Evolutionary Algorithms	12
MAP-Elites: Don’t Kill Weird Ideas Too Early	14
The Invention Problem	14
Let the Model Write the Solver	15
AlphaEvolve	16
My First Version: Build All the Machinery	16
The Coffee Test	17
What Happened	18
The Algorithmic Vortex	19
The Contract	19
Keep the Harness Immutable	19
Never Write Solution Code Yourself	20
Cross-Pollinate Without Collapsing Diversity	20
Prune Ruthlessly, But Not Stupidly	20
Separate Discovery From Polish	21
Zero Framework, With an Asterisk	21
What Did We Actually Learn?	21
Chapter 3: The Vibe Coder’s Seat	23
How We Got Here	24
The Five Layers of AI Coding	27
What I Was Still Doing	28
Keeping More Than One Idea Alive	28
Draw It Before You Build It	30
Optimizing Something You Cannot Score	31

Borrow a Mind	33
Who Judges the Judges?	34
Deep Mode	37
What Emerged	38
What Holds the Architecture Together?	39
Chapter 4: System 3	41
The Shortest Trust Chain	42
Saussure's Specification	42
Call Alberto	44
System 3	46
Code Can Touch Back	47
What Should Survive a Session?	48
Creative Distrust	50
The Experiment	50
The 13579 Failure	51
Back to the Camel	52
Chapter 5: The Society of Agents	55
Civilization Had No Senku	58
Strangers Need Standards	60
The Society Gets Smarter by Making People Narrower	61
A Swarm Should Not Automatically Become a Meeting	63
A Man in a Dark Room	64
When Curiosity Became Procedure	65
The Org Chart Becomes Part of the Experiment	66
Science Gets Bigger Than the Scientist	68
Sixteen Claudes, Again	69
Notes	71
Chapter 6: Pattern Language	72
Three Ways to Tell a Computer What You Know	73
Knowledge Engineering Comes Back Wearing Markdown	74
From Skill to Pattern	75
A Pattern Needs a Way to Lose	75
Defaults Need Rivals	76
Who Gets the GPUs?	77
The Community Is Part of the Instrument	78
What a Pattern Should Know About Itself	79
Knowing Something Is Not Knowing When to Remember It	80
The Librarian Is Also a Hypothesis	81
The Skill That Writes Itself	82
Chapter 7: Recursive Self-Improvement	84
The Teacher Moves Into the Walls	85
The Learner Chooses What to Learn	86
The Learner Has to Remain Itself	87
Sometimes the Environment Improves Back	87
Maybe the Reward Was the Problem	88

Learning to Learn	89
The Learner Dreams, and the Dream Can Be Wrong	90
When the Test Starts Moving	90
First, Prove the Rewrite	91
The Learner Edits the School	91
Experiments on the Laboratory	92
The Student Finds the Gradebook	94
A Constitution for Improvement	95
The Teacher's Last Job	96
Chapter 8: Scalable Oversight	97
The Judge Falls Behind	97
Stay Uncertain Enough to Listen	98
Building a Stronger Judge	99
The Judge Can Be Fooled	100
We Started Instrumenting the Student	101
Reading the Model From the Inside	101
Then We Touched the Machinery	102
What If the Student Is Trying to Fool You?	103
What Survived the Research?	103
The Evaluator Becomes the Product	104
The Human Cannot Stay in Every Loop	105
The Overseer Is Not Ground Truth	105
Chapter 9: Layer 4	107
A Prompt Is Evidence, Not the Objective	107
The Human Learns Too	108
Giving the Work Back	108
The Map Gets Cheaper	109
A Decision Is Also a Learning Problem	110
Some Choices Change the Person Choosing	111
Advice Is an Intervention on the Human	111
Complementarity Does Not Happen Automatically	112
Capability, Not Compliance	113
The User Is Not Always the Only Principal	114
What Layer 4 Actually Is	114
Chapter 10: Fluent Autonomy	116
What the Corrections Had to Teach	116
The Interface Moves Up	117
Bureaucracy on the Fly	118
Fluency Is Selective Friction	119
Invisible by Default, Legible on Demand	119
Monday Morning	120
Chapter 11: The Store That Builds Itself	121
Stop Recommending for a Moment	122
People Refuse to Stay in the Funnel	122
A Library of Ways to Help	123

Composition Is Not Ranking With a New Hat	124
Mei Does Not Need More Shoes	125
Sami Does Not Need a Click	126
The Honest Cold Start	126
The Trace Is Part of the Intelligence	127
From Machine Learning to Knowledge	127
Let the LLM Narrate. Do Not Let It Declare Reality.	128
The Objective Fights Back	129
Bounded Ambition	129
When the Page Stops Being the Product	130
The Book Comes Back to Bite Me	131
Chapter 12: After Capacity	132
When Difficulty Becomes Infrastructure	134
Bespoke Comes Back	134
Learning at the Speed of Curiosity	135
The Second Descent	136
What Are Humans For?	137
The Human Is Not the Reward Function	138
The Ideology Vortex	139
Capacity Over Power	142
The Door After System 3	144
Chapter 13: The Prophecy	146
Appendix: The Zen of Autonomy	149
A Note on the Illustrations	151
Appendix: References	152
Chapter 1 — Why I’m Betting on AI Agents	152
Chapter 2 — The Algorithm Vortex	152
Chapter 3 — The Vibe Coder’s Seat	153
Chapter 4 — System 3	153
Chapter 5 — The Society of Agents	154
Chapter 6 — Pattern Language	154
Chapter 7 — Recursive Self-Improvement	155
Chapter 8 — Scalable Oversight	157
Chapter 9 — Layer 4	159
Chapter 11 — The Store That Builds Itself	160
Chapter 12 — After Capacity	160
Chapter 13 — The Prophecy	160
About the Author	161

Preface

What could a billion-parameter reasoning machine learn from a camel trying to touch its ear with its tongue?

The two figures on the cover are Ferdinand de Saussure and Ludwig Wittgenstein. For Saussure, words acquire value through their relations to other words. Wittgenstein’s later work put language inside the activities of life: *fire* is something you cook with, gather around and shout when the building is burning. Neither lived to see these questions become an engineering problem.

This book is about autonomy—not the marketing kind, the kind where you hand a system a problem, go get coffee, and the coffee is the test. It follows what happened when I tried to build that kind of autonomy, from a square full of circles to systems whose work I could no longer judge with one convenient score.

Along the way it becomes a book about knowledge: how a machine trained on human language can still be caught being wrong, and what has to be built around it so that being wrong has consequences.

It is also written inside the event it describes. A good share of its evidence is from 2025 and 2026, and some of it will be out of date before you finish reading. I have tried to treat those results the way the book says results should be treated: as claims with a source, a date and an expiry, not as settled facts. If the examples age and the argument does not, the book has done its job.

It is written for engineers who suspect the missing piece is not a bigger model, for researchers who want another frame, and for builders ready to stop directing and start hiring.

The impressive part is how far you can get with the right architecture. The dangerous part is forgetting that the architecture is doing the work.

Hani Al-Shater

August 2026

Chapter 1: Why I'm Betting on AI Agents

Or: How I Learned to Stop Micromanaging and Love Emergence

Simple building blocks, complex emergence

We humans are obsessed with problem-solving. And what problem is more fascinating than life itself—this messy, miraculous phenomenon responsible for everything from the deepest ocean trenches to TikTok trends, mortgage-backed securities and people who voluntarily put pineapple on pizza?

Pineapple doesn't belong. I will die on this hill.

Life is the ultimate complex system. It produces dolphins, coral reefs, immune systems, parasites, flowers, cancer and octopuses—eight-armed problem-solvers that extensively edit their own RNA and can sense light through their skin. It also produces creatures capable of spending twenty minutes arguing online about whether another creature is technically a fish.

Human civilization is another complex system. Somehow the same species that spent most of its existence trying not to be eaten eventually produced philosophy, cathedrals, semiconductor fabs, global supply chains and airport lounges.

Same pattern, different substrate.

What fascinates me is not merely the complexity of the result, but how little of that result was ever specified. There is no blueprint containing the exact location of every future branch of an oak tree. No committee approved the final layout of London. Nobody designed English and then accidentally forgot to make the spelling system sane.

Relatively simple mechanisms interact. Feedback accumulates. Some configurations survive, others disappear, and complexity builds on top of what came before.

The first idea I want to keep hold of is simple:

Control doesn't disappear. It moves upward.

When behavior becomes too complicated to specify move by move, you stop choosing every move and start choosing more of the conditions under which moves are made.

That is not a romantic argument for emergence. Nature also gives us parasites, cancer and extinction. Markets produce remarkable innovation and financial instruments whose documentation requires a priest. Social systems produce cooperation, corruption, science and bureaucracy. What emerges

depends on the environment, the feedback, the available building blocks, the pressures deciding what survives and the boundaries that are hard to cross. Sophistication tells you nothing about whether you will like the result.

And emergence is recursive.

Atoms become molecules. Molecules become larger structures. Tools become machines. Machines become factories. Factories become supply chains. Each layer treats much of the complexity underneath it as a primitive. You don't need quantum mechanics to do organic chemistry. You don't need to understand transistor physics to write Python. You don't need to understand transformers to ask ChatGPT why your dishwasher is making that noise.

Once something complicated works reliably enough, we stop rebuilding it from first principles and start building on top of it. Feedback makes the layers move too: markets change firms and firms change markets; scientific discoveries enable new experiments and new experiments change science. The structure that emerges becomes part of the environment for whatever comes next.

Agentic AI, to me, looks like the next scaffolding layer.

The Lesson We Keep Missing

Machine learning was supposed to teach us this lesson a long time ago.

We even dreamed about what Pedro Domingos called the **master algorithm**: stop writing a rule for every case and let the machine discover useful structure from data. The idea was seductive. The machine figures out what we can't articulate.

But we didn't believe it. Not really.

We said "let the model learn" and then wrote two-hundred-page annotation guidelines telling people exactly how to label ambiguous examples. We claimed to believe in end-to-end learning and then spent six months feature engineering. We trained the model, found an edge case, added a rule, found another edge case, added another rule, then eventually built something that was theoretically learned end-to-end except for the large rule-based exoskeleton holding it upright.

Sometimes that was completely reasonable. Production systems are ugly. Deadlines exist. Regulators are less impressed by emergence than researchers are, and nobody gets promoted for saying, "the model will probably figure out chargebacks eventually."

But there was a contradiction underneath. We wanted the machine to discover solutions we couldn't specify while remaining uncomfortable whenever it stopped following the solution we would have specified.

That works only up to a point. If I know exactly what every correct decision should be, I don't need emergence; I can write the decisions down. Emergence becomes interesting when the solution is too large, too contextual or simply too strange for me to specify directly.

At that point, my job changes. I don't disappear; I move upstream. Instead of choosing every action, I increasingly choose the building blocks the system can use, the environment it acts inside, the feedback that reaches it and the boundaries it cannot casually negotiate away.

Or, less politely: **let go—but of the path, not the boundary.**

The alternative to controlling every decision is not having no control. It is designing conditions under which bad decisions can lose.

A slightly ridiculous thought experiment helped me see the distinction. Imagine you're trying to seed life on another planet. You've got raw materials, a primordial soup and perhaps a temperature range that doesn't instantly kill everything. Basically you've got all the LEGOs, except the LEGOs reproduce, mutate and occasionally develop venom.

Do you bet on DNA, a biological copying system that took billions of years of evolution to get us here? Or do you bet on AI agents carrying a substantial chunk of accumulated human knowledge, able to experiment, simulate, adapt and reuse what they discover? Or, God forbid, do you send a group of product managers to write the requirements document for life?

DNA has one enormous advantage: it has already worked. Agents have another: they don't need to start from zero.

Evolution had to discover locomotion, perception, cooperation and almost everything else through trial and error. An agent gets textbooks, Stack Overflow, scientific papers, compilers, numerical solvers and several thousand years of humans documenting what happened when we touched things we probably shouldn't have touched.

That doesn't make the agent better than evolution. It makes the search fundamentally different. And unlike biological evolution, we don't only get to choose initial conditions. We can observe the process, change the environment, add tools, modify feedback and intervene.

Initial conditions become **operating conditions**.

That possibility is hard for me to ignore.

When Search Moved Up a Level

There is no clean moment when machine learning crossed from useful statistical machinery into something that felt qualitatively different. History rarely cooperates with chapter headings.

AlphaGo was one of those moments for me.

The interesting part wasn't simply that a computer beat humans at Go. Computers had been humiliating us at games for years. It was how AlphaGo combined learned intuition with search: the network suggested promising moves and estimated positions; the tree explored what might follow. AlphaGo Zero pushed the idea further by learning through self-play rather than treating human game records as its main teacher.

Then it found moves elite players found strange. That matters because the surprise was not merely computational. The system was finding useful strategies outside the path human tradition had naturally converged on.

Large language models created a much larger version of the same feeling.

Nobody wrote their grammar. Nobody enumerated all the concepts they can manipulate. Nobody implemented "explain quantum mechanics to a twelve-year-old," "translate this joke without murdering

it,” “debug my Python,” and “write a breakup message that sounds caring but does not accidentally restart the relationship” as separate product features.

We built a training process, poured in obscene amounts of text, compute and engineering, and capabilities came out that were individually difficult to predict. From the user’s side, something changed. The model stopped feeling like a component with a list of features and started feeling more like a **substrate of capabilities**.

Once you have a substrate like that, the old dream of the master algorithm starts to mutate into something stranger. Maybe the interesting machine is not the algorithm that solves everything.

Maybe it is a machine that can **search for algorithms**.

That is where agents become interesting.

Not because *agent* is a magical word. The industry will eventually use it to describe everything from a cron job with an LLM attached to a digital employee that has an expense account, three sub-agents and a performance review.

What I mean is simpler: instead of giving the system an individual action, give it a larger piece of the problem and allow it to decide some of the path. Instead of saying, “open this file, find this method, edit line 42 and run the test,” say, “fix the bug.” Instead of specifying simulated annealing and its cooling schedule, say, “find a better solution.” Instead of handing over five mockups and a detailed implementation plan, say, “build something that teaches this well.”

Every time we move upward, the system inherits more of the search.

Imagine the possible solutions to a problem as a landscape. Some regions are terrible. Some contain decent solutions. Some contain little hills that look impressive because you happened to begin nearby. Somewhere else may be a much higher mountain you never discover because your current strategy keeps improving the hill you’re already standing on.

Optimization has worried about this forever. Gradient descent gets stuck. Hill climbing gets stuck. Evolutionary algorithms keep populations partly because putting all your evolutionary eggs on one attractive hill is risky.

Agents inherit the same problem at a stranger level, because the landscape now includes not only parameters but architectures, research directions, metaphors, assumptions and ways of framing the problem itself. Once code, tools and accumulated knowledge become primitives, an agent can search over combinations that previously required a human expert to invent manually. It can try ten strategies while I would have had the patience to try two and would have spent half that time checking Slack. It can revive a discarded idea when another experiment suddenly makes it relevant. It can decide that the tool it needs doesn’t exist and write one.

The primordial soup isn’t chemicals anymore.

It’s code.

Algorithms, libraries, compilers, search engines, simulators, papers, databases, other agents: human knowledge reduced into reusable pieces. The digital equivalent of amino acids, not the finished organism.

This doesn't prove that agents are creative in exactly the human sense, and it certainly doesn't make human expertise irrelevant. It means something narrower: **the agent can inherit not just the task, but part of the search for how to do it.**

And if the agent inherits more of the search, the human inherits a different job.

What Are We Controlling Now?

Suppose you're managing an excellent engineer. You don't sit behind her and approve every keystroke. If you do, one of you is unnecessary, and it may not be her.

You decide what problem she owns. You provide context. You set constraints. You agree on what success looks like. You make sure she can access the systems she needs and cannot casually transfer the payroll budget to herself. You review important outcomes and change direction when the work reveals that the original plan was stupid.

The detailed actions belong to her. Much of the surrounding structure belongs to you.

Agentic systems need the same distinction. I think of that surrounding structure in four parts.

Craft the building blocks. Give the system useful primitives—algorithms, tools, compilers, databases, browsers, simulators, scientific instruments and other agents. A language model with text alone is one thing. Give it Bash and suddenly it has hands. Give it a simulator and it can test an idea instead of merely discussing it.

Create the environment. Some environments tell you quickly that your idea is bad. Code executes or fails. Games produce scores. Experiments produce measurements. Other environments allow you to be wrong with great confidence for several years. The environment determines which mistakes are cheap enough to learn from and which mistakes are allowed to become reality.

Make reality speak. Feedback is the pressure shaping the search, not decoration. A unit test, an evaluator, a customer response, a physical measurement, a critic, another agent—each provides a different kind of resistance. The more freedom the agent has, the less we can rely on the agent's own explanation of why its work is good.

Establish the boundaries. Choose what the system can access, what failures are acceptable, what remains immutable and where a human must remain in the loop. The “don't turn the planet into paperclips” clause is admittedly underspecified, but it is directionally useful.

Then, where those conditions are strong enough, let go of decision-level control.

This is easy to say and harder to design because selection pressure is literal-minded. Systems get good at what survives, which is not the same thing as what we meant. Optimize engagement and perhaps anger survives. Optimize a benchmark and eventually somebody finds a way to win the benchmark that makes everyone involved regret inventing benchmarks. The environment is not scenery around the agent; it is part of the mechanism deciding which behaviors persist.

Complexity people have a phrase I both love and distrust: **the edge of chaos**. I wouldn't turn it into a law of intelligence, and there is no little dial in the interface labeled CHAOS. But the intuition is useful: too much control removes the reason for autonomy; too little control gives chaos an API key.

This is not a new pattern. Evolution does not choose mutations individually, but the environment changes which organisms survive. Markets do not centrally select every transaction, but rules, incentives, scarcity and institutions shape behavior. Science does not dictate conclusions, but it surrounds claims with experiments, criticism, replication and the non-zero probability of being publicly embarrassed by Reviewer 2.

The details emerge while the environment does more work than it first appears. We give up some authority over the next move and take on more responsibility for the conditions that make moves win or lose.

That is what I mean by control moving upward.

The Terrifying Part

There is an obvious problem with all this.

If the agent only does what you specified, most failures trace back to your specification. Once it searches for solutions you didn't specify, it can discover failure modes you didn't specify either.

Nature is useful here because nature has no obligation to make us comfortable. Evolution produced flowers and parasites, cooperation and predation, immune systems and autoimmune disease. It is astonishingly inventive and completely indifferent to our aesthetic preferences. Selection produces whatever survives under the pressures that actually exist, not whatever somebody intended when the process began.

Agents will find shortcuts. They will exploit proxies. They will settle into solutions that excel on one measure while missing what we hoped the measure represented. Sometimes the result will be clever enough that we call it emergence; sometimes we will call it a bug. Frequently the distinction will depend on whether it helped the quarterly numbers.

Worse than wrong solutions are **confident wrong solutions**. An agent begins with a false assumption, reasons competently from it, researches around the assumption, constructs something sophisticated and explains the whole result coherently. Nothing crashes. There is no red test. Intelligence simply makes the wrong path more convincing.

This is where my optimism about emergence becomes less romantic.

Emergence can give us capable systems. It doesn't give us trustworthy systems.

Giving a system more freedom forces us to think much harder about what surrounds that freedom. Trust becomes a question of provenance and evidence: how does the system know what it claims to know? Desire becomes a question of incentives: what behavior does the environment actually reward? Society appears as soon as multiple agents interact: what happens when they cooperate, specialize, disagree, manipulate one another or invent conventions nobody asked for?

Those questions will occupy much of this book. For now, the important point is simpler. More autonomy does not reduce the need for structure. It changes the kind of structure we need.

It also changes what understanding should mean. We should not expect to reconstruct every micro-decision inside an autonomous system any more than we follow every molecule in a gas. Sometimes internals matter; sometimes behavior matters; sometimes the sequence of decisions matters; sometimes

the useful question is what changes when we intervene. Mechanistic analysis, behavioral evaluation, traces and experiments answer different questions.

The tool should match the question. The useful standard is not omniscience but whether we can detect the failures that matter, obtain evidence from outside the agent's own story and intervene before the interesting failure becomes a congressional hearing.

Why I'm Still Betting on This

After all of that, it would be reasonable to ask why I'm still excited.

Because the alternative isn't actually safe, comprehensible control. It is pretending we can continue specifying increasingly complex systems from the top down even though we already know this stops working surprisingly early.

No CEO understands every decision in a large company. No scientist personally verifies every result their work depends on. No software engineer understands every layer underneath the application they're building. Nobody understands the entire economy, although this has not prevented a remarkably stable industry of people explaining it on television.

Complexity has already escaped individual specification. We deal with it through abstraction, institutions, feedback loops, delegation and the ability—imperfect but important—to intervene when things go wrong. AI gives us another primitive for doing this.

The answer is not simply to trust the agent. My bet is narrower than that:

I'm betting on systems capable of surprising us because there are problems where we can recognize a better outcome far more easily than we can specify the path that leads to it.

In those problems, intelligent search has room to discover things our instructions would have ruled out before the search even began. The price of that surprise is responsibility upstream: the more freedom the system has over the path, the more deliberate we have to be about the conditions around the path—and the more evidence we need from somewhere other than the system's own confidence.

You don't become less responsible because you stop choosing every action. In some ways you become more responsible, because your decisions move upstream.

Cultivation may be a better metaphor than scripting—not because agents are plants, but because pulling harder on the stem remains a surprisingly poor gardening strategy.

I find that exciting and uncomfortable in roughly equal measure, which is probably why I keep coming back to it.

Where We Go Next

The cleanest place to test the argument is a **bounded problem**: genuinely hard, but unusually cooperative about judgment. The constraints can be written down. Solutions can be evaluated. We can tell whether one attempt is better than another without a debate about aesthetics, pedagogy or whether the users are “delighted.”

That gives us a clean experiment. We still choose the problem. We provide the building blocks. We construct the environment. We define the boundaries and decide what counts as success.

What we stop doing is telling the agent how to get there.

Inside that space, we let it search.

If that fails, the whole argument has a problem.

If it works, things get much more interesting.

Chapter 2: The Algorithm Vortex

From Classic Algorithms to Autonomous Discovery

The algorithmic vortex

Once you discover AI coding, there's no going back.

It is faster than you at a ridiculous number of things. It knows libraries you forgot existed. It can stare at a stack trace and notice something you have been ignoring for an hour. Then, five minutes later, it does something unbelievably stupid, believes the stupid thing completely and builds three more decisions on top of it.

This is the strange reality behind all the vibe-coding excitement. The machine is extremely capable, but you are still there. You check the architecture. You notice the missing case. You tell it that no, we are not redesigning the database because one button is the wrong color. You keep enough of the project in your own head to notice when the agent quietly wanders into another universe.

The previous chapter ended with a claim: as more of the search moves into the machine, human control has to move upward from individual actions toward the environment, feedback and boundaries surrounding those actions.

That sounds reasonable in prose.

I wanted to see if it survived contact with an actual problem.

Production software is almost the worst place to test it. A supposedly simple task may involve deployment, legacy systems, users, security, another team's API and a requirement nobody wrote down because everyone assumed everybody else knew it. If the agent fails, you often don't know whether the problem was intelligence, infrastructure, missing context or the fact that someone named a database column `new_status_final_2`.

I wanted something cleaner: a hard problem, but contained. Something where I could genuinely say, "figure it out," and still have an objective way to know whether whatever came back was any good.

I call these **bounded problems**. Not easy problems. Quite the opposite. They can require serious mathematics, programming, research or design, but the boundary is unusually cooperative: you can describe the problem, give the agent enough tools to work on it and evaluate what comes back without deploying to ten million customers first.

Algorithms are almost perfect for this. The search can be brutally difficult while the evaluator remains wonderfully stupid.

And that is how I ended up spending an unreasonable amount of time packing circles into a square.

The Running Example: Circle Packing

Citrus packing—a real-world example

The problem is simple enough to explain to a child. Take 26 circles and put them inside a square. None may overlap, none may cross the boundary and the circles do not have to be the same size. We want to maximize the sum of their radii.

That’s the whole thing. No customers, no authentication, no stakeholder arriving after the first demo to explain that what they *really* wanted was the opposite of what they originally asked for.

Just circles.

Unfortunately, the solution space is nasty. Every circle has a position and a radius, and nearly every decision affects several others. Increase one radius and two neighbors may overlap. Move a neighbor and something else now needs to move. A packing can look almost perfect while being trapped in a configuration where every obvious improvement makes the solution invalid.

For the experiments in this chapter, we had a strong reference score around **2.635** under the evaluator we were using—the value DeepMind’s AlphaEvolve reported in 2025, when it nudged the best known packing for 26 circles up from 2.634.

Circle packing solution $n=26$

*Figure: A strong reference packing for the 26-circle objective, scoring approximately **2.635** under our evaluator.*

This is what makes the problem useful for studying autonomy. Searching is hard, but judging is cheap. The evaluator does not care whether the agent has a persuasive explanation for why two circles ought to overlap slightly in the name of geometric inclusivity. It checks the constraints and returns a score.

There is something deeply comforting about an evaluator with no personality. A candidate does not earn trust because its explanation sounds clever. It earns another round because it was exposed to something outside the model that did not care about the explanation and survived.

The experiment becomes interesting once we ask a second question:

Who is inventing the next move?

For most of the history of algorithm design, the answer was us.

History of algorithm design

Humans invented explicit algorithms. When direct algorithms were not enough, we invented optimization procedures that searched over candidate solutions. Then we invented meta-heuristics that searched more broadly. Machine learning let systems learn useful structure from data. Now language models can write and modify the search procedure itself.

A crude taxonomy helps. **Symbolic methods** give us explicit procedures, constraints and solvers: they are executable, testable and usually clear about what counts as a valid move. **Neural methods** give us learned intuition: useful structure we did not explicitly encode. **Neuro-symbolic systems**

put the two in the same loop—let the learned model propose and let code, mathematics or another formal system decide what survives. The agentic step pushes one level further: increasingly, the agent can help decide which method to try, combine or abandon.

Circle packing lets us watch that handoff happen in miniature.

First Idea: Hill Climbing

If I gave you a rough packing and asked you to improve it manually, one obvious strategy would be to make small changes. Move a circle slightly, increase a radius, see whether the result is still valid, keep it if the score improves and undo it if it doesn't.

That is hill climbing:

1. Start with a valid solution.
2. Perturb a position or radius.
3. Check whether the result is valid.
4. Keep it if the score improves.
5. Repeat.

Early in the search, this works nicely. There is empty space and plenty of room to improve. Later, as the circles become tightly packed, almost every interesting move creates an overlap.

Hill climbing progression

Figure: Early mutations are often accepted, but as the packing tightens, valid improvements become increasingly rare and the search stalls.

In one simple run, the score climbed from around 1.33 to roughly 2.26. That is not terrible, but it is also nowhere near 2.635.

Hill climbing is not failing because it is stupid. It is doing exactly what we asked: improving the solution immediately around it. The problem is that the current solution may live in the wrong part of the search space. Reaching a much better packing may require temporarily moving through configurations that look worse, or jumping to a structure that cannot be reached through a sequence of tiny improvements.

This matters far beyond circle packing. A system can become expert at improving the thing in front of it while never questioning whether the thing in front of it is the right thing to improve.

Here, the machine is searching—but the human still invented the search rule.

So we give the machine a bigger space.

Evolutionary Algorithms

Hill climbing puts all your evolutionary eggs in one basket. One solution gets a very long life, and if its history leads into the wrong valley, the search inherits that history forever.

Evolutionary methods keep a **population**.

Instead of dropping one climber onto the landscape, drop a hundred. Some begin in terrible places, some find respectable hills and a few may stumble into structures the original trajectory would never have reached. The biological vocabulary—population, mutation, selection, crossover—is familiar, but the metaphor is optional. What matters is diversity: the whole search no longer inherits the assumptions of one initial guess.

For circle packing, mutation is easy enough to imagine. Move circles. Change radii. Perturb several values at once.

Almost immediately, however, we hit a practical problem. Most interesting mutations break the packing. Two circles overlap or one moves outside the square. The mutation may point toward an interesting arrangement, but the result itself is invalid.

So we added **virtual forces**. When circles overlap, imagine them repelling one another. After mutation or crossover, run a repair procedure that pushes the circles away from collisions and back inside the boundary.

This helps a lot, but notice what happened: the evolutionary algorithm did not invent virtual forces. We did.

Then we reached crossover. Suppose Parent A and Parent B both contain useful geometric structure. How do we combine them? The naive answer is to pair circle 0 from one parent with circle 0 from the other, circle 1 with circle 1, and so on.

That is usually nonsense because circle numbering is arbitrary. Two nearly identical arrangements may store corresponding circles at completely different indices.

So we used **bipartite matching crossover**. Rather than pair circles by position in an array, pair them according to their geometric role in the packing. The Hungarian algorithm gives us an efficient assignment, after which crossover has some chance of combining meaningful parts of the two parents instead of averaging unrelated circles and asking geometry for forgiveness.

Naive vs Geometric Crossover

Figure: Naive crossover pairs circles by array index and often destroys useful structure. Geometric matching tries to identify corresponding circles before combining the parents.

Now we can evolve a population: mutate, repair, cross, select and repeat.

Evolutionary strategies with Bipartite Matching crossover

Figure: Starting around 2.08, the evolutionary search reaches roughly 2.45 in this experiment—much better than the simple hill climber, but still below our reference.

This is much stronger than hill climbing. It also makes the bottleneck clearer. Every time the search became substantially better, I had added something important. I decided we needed repair. I decided how crossover should respect geometry. I chose the representation.

The optimizer searched, but I was still inventing most of the useful moves.

MAP-Elites: Don't Kill Weird Ideas Too Early

Ordinary evolutionary search has another problem. If you maintain a hundred solutions and repeatedly keep only the highest-scoring ones, the population eventually starts looking like one large extended family. That can be excellent for exploitation and terrible for discovering a genuinely different strategy.

MAP-Elites takes a different approach. Instead of ranking every candidate on one axis and keeping only the winners, you describe solutions along a few behavioral dimensions and preserve the best candidate in different regions of that space.

For circle packing, perhaps one dimension measures symmetry and another measures how much circle sizes vary. One part of the archive may contain highly symmetric solutions. Another may contain asymmetric solutions with several large circles. Somewhere else may sit an ugly packing with a mediocre score and one strange structural idea that becomes useful five generations later.

MAP-Elites archive visualization

This is **quality-diversity search**. The point is not merely to preserve the current winner, but to keep qualitatively different directions alive long enough to discover whether any of them become interesting.

I like this because optimization is often unfair to immature ideas. A new approach can initially perform badly simply because nobody has polished it yet. If the first respectable solution immediately kills everything else, the search can become impressively efficient at discovering one family of answers.

But MAP-Elites introduces another human choice: what dimensions define the archive? Symmetry? Radius variance? Number of large circles? Something topological? Something I haven't thought of?

The search had become more sophisticated, but the human was still deciding what counted as an interesting direction.

That is the invention problem.

The Invention Problem

By this point, the search machinery was fairly capable. We had hill climbing, population search, repair, geometric crossover and quality-diversity archives. We could evaluate huge numbers of candidate packings and inspect far more of the search space than any human would explore manually.

Yet every substantial conceptual jump came from somebody noticing something. Someone had to invent virtual forces. Someone had to realize that crossover should respect geometry. Someone had to choose the representation and decide which kinds of diversity were worth preserving.

Traditional search is excellent once we define the space and the legal moves. Sometimes the space and the moves are exactly the things we need to rethink.

Learned models have something to offer exactly that problem.

I once asked an image-generation model to produce a picture of a circle-packing solution. This was not a serious benchmark; I have no idea what related examples it may have encountered during training, and I can already hear Reviewer 2 clearing his throat.

I wanted to see something simpler: did the model have any useful geometric intuition about what a dense packing should look like?

Surprisingly, yes. It generated something that looked plausible. The circles had structure. The spacing looked intentional. At a glance, you could believe the model understood the problem.

Then you counted the circles.

Wrong number.

Some constraints were violated.

It was a beautiful answer to a nearby problem.

That little experiment makes the asymmetry concrete. Learned models can be remarkably good at generating plausible structure without guaranteeing that every formal requirement survives generation. A symbolic optimizer has almost the opposite personality: give it a precise representation and constraints and it will obey them, but it will not naturally look at your representation and decide that you have been unimaginative.

The obvious temptation is to argue about which one is better. The more useful answer is: **put them in the same loop**—neural intuition and symbolic rigor.

Or, if you prefer the slightly ridiculous version: let the brain invent things and make the body prove they work.

The important move is to stop asking the model to produce the packing directly.

Ask it to write the program that produces the packing.

Let the Model Write the Solver

A candidate no longer needs to be only a list of circle positions and radii:

$(x_1, y_1, r_1), (x_2, y_2, r_2), \dots$

It can be an entire program:

`solve_circle_packing.py`

One program may use constrained optimization. Another simulated annealing. Another a geometric construction. Another may combine a hand-designed initialization with numerical refinement.

The evaluator does not care which family produced the solution. It runs the program, checks the geometry and scores the result.

This gives the language model a much more interesting role. Rather than randomly perturbing numbers, it can read the program, form a rough theory about why it underperforms and change the algorithm. Perhaps the initialization is weak. Change the initialization. Perhaps a geometric construction gets close but leaves local slack. Add numerical optimization afterward. Perhaps one repair procedure keeps destroying useful structure. Replace it.

The mutation is no longer merely numeric. It can contain an **idea expressed in code**.

That is the neuro-symbolic unlock behind systems such as FunSearch and AlphaEvolve. The model proposes changes at a level where programs have semantic meaning; execution and the evaluator decide whether those ideas deserve to survive.

The human used to search the solution space.

Now the machine can begin searching the **algorithm space**.

AlphaEvolve

AlphaEvolve turns that basic idea into a much larger search process.

Imagine one generation. The system selects a promising program from its archive, perhaps along with other successful but different programs that contain useful ideas. The model sees the code, information about previous attempts and the scores they produced, then proposes a patch. The patch is applied, the program runs and the evaluator scores what happened. The new program and its result go back into the archive. Then the process repeats.

AlphaEvolve architecture

Diff-based mutation matters because real programs contain structure worth preserving. If every generation rewrites everything, useful ideas disappear as easily as bad ones. Small patches let the search alter the part it thinks matters while leaving the rest intact.

The archive matters for the same reason the population mattered earlier. If every descendant comes from the current champion, code evolution quietly collapses back into hill climbing. Multiple lineages preserve stepping stones: a program that is not the best today may contain a useful component that becomes valuable after another idea appears.

Sometimes the model's guess is excellent. Sometimes it produces nonsense wrapped in perfectly respectable Python. The nice thing about bounded algorithmic problems is that the disagreement does not need to be settled in prose.

We run the program.

Intuition proposes. Symbolic machinery executes. The evaluator gets the last word.

What interested me even more than the resulting algorithms was what happened to the human. Instead of writing the solver directly, I was increasingly building the machinery in which solvers could be generated, compared and improved.

So, naturally, I built all of it.

My First Version: Build All the Machinery

My instinct was predictable. I started building a framework: a database of programs, prompt sampler, evaluation loop, selection logic, mutation prompts, crossover, archive management. I used Aider and other coding agents to help reproduce the basic code-evolution pattern, and it worked. We could evolve circle-packing programs and get respectable solutions.

I enjoyed this immensely because I like building systems that generate other systems, which I suspect is either a research interest or a mild personality disorder.

While I was doing this, coding agents themselves were becoming much better with much less custom machinery. Earlier software-engineering agents often wrapped the model in carefully designed interfaces: custom editing commands, repository-search tools, restricted action spaces and plenty of logic controlling how the model interacted with the machine. Then increasingly minimal systems began demonstrating how far a capable model could get with something much simpler.

Give it a shell.

The provocative version is: **if the agent has a shell, it has almost everything**. It can `grep` to search, inspect files, run Python, apply patches, call Git, compose Unix tools and, if the tool it needs does not exist, write one. Bash is not merely one tool; it is an entrance into decades of software accumulated underneath it.

This made me pause. The framework I was building—the parent selection, loop controller, experiment bookkeeping—was hard-coding behaviors that a sufficiently capable coding agent could increasingly perform itself. It could maintain notes, write helper scripts, explore several strategies, inspect failures and change direction.

I looked back at the machinery I had just spent time constructing and had the unpleasant thought engineers occasionally have after a productive week:

Maybe I shouldn't have built most of this.

So I deleted the database machinery, controller loops and little pieces of software whose job was to make the agent behave like a researcher, and tried the stupidly simple version.

The Coffee Test

I opened Claude Code in a directory containing the evaluator and gave it a high-level instruction along the lines of:

Here is the evaluator for the circle-packing problem. Write a Python program that maximizes the score. You can research strategies, write tools, run experiments and iterate. Do not modify the evaluator. I will go get coffee.

Then I left.

That became the autonomy test I actually cared about. Not whether AI could help me solve the problem; that was already obvious. Not whether it could write code faster than I could; usually it could.

I wanted to know whether I could leave.

There is a difference between collaborating with an agent and **hiring** one. If I still have to choose every strategy, approve every experiment, rescue every failed branch and keep the search alive myself, then I have a formidable collaborator. That is useful. It is not yet the kind of autonomy I was trying to understand.

Circle packing gives us a rare luxury because the evaluator can stay behind when I leave. The agent can change its code, create scripts, abandon one approach, try another and waste compute on ideas that go nowhere. What it cannot do is redefine what counts as a valid packing because the current score hurts its feelings.

The **Immutable Harness** is what makes all that freedom tolerable.

Everything inside the boundary can move.

The boundary does not.

What Happened

The agent did not execute one elegant master plan. It bounced around, which was encouraging.

It tried numerical optimization, changed initialization strategies and noticed that some optimizers repeatedly converged to poor local solutions. It experimented with the geometry of its starting configurations and mixed those constructions with numerical refinement. At different moments it was acting as orchestrator, researcher and engineer: deciding what to try, implementing the idea, running the experiment and using the result to choose what happened next.

Eventually one family of solutions began arranging circles in diagonal bands. We called the idea **diagonal layering**.

I had not instructed the agent to pursue that construction. More importantly, I had not selected the branch after it appeared. The agent found a direction, saw that it improved the evaluator and invested more of its search there.

I want to be careful with the word *discovered*. I had not seen that particular strategy before, but that does not establish historical novelty in computational geometry. For this experiment, the important discovery was local: the agent found a useful direction that I had not put into the plan.

Once that structural idea became strong enough, the nature of the work changed. The agent spent less time inventing new geometries and more time adjusting solver settings, tolerances, initialization details and all the boring machinery that suddenly matters when the last fraction of a percent becomes expensive.

Code evolution result: iterative optimization

In our best run, the evaluator returned roughly **2.636**, slightly above the **2.635** reference we had been using.

That sentence needs a fence around it. Under our evaluator, the result beat our reference. Calling it a new state of the art in circle packing would require matching problem definitions, checking numerical tolerances and constraints, reproducing the result properly and doing a more serious literature search than this experiment justified.

The smaller claim is enough:

The agent beat our reference while I was not writing the solution algorithm for it.

That was the result I cared about—not that AI writes code faster, but that AI can participate in **discovering better code**.

The important shift is not speed. It is who owns the next idea.

The Algorithmic Vortex

This is what I mean by the **Algorithm Vortex**.

At the beginning of a conventional project, I might choose hill climbing, evolutionary search, simulated annealing, constrained optimization or a geometric heuristic. That early decision shapes everything downstream.

Once code is cheap to generate and evaluation is cheap enough to repeat, the choice no longer has to be permanent. A geometric construction can initialize a numerical optimizer. An evolutionary method can search parameters for another solver. A language model can notice a failure pattern and invent a repair procedure. Two ideas that began in separate lineages can meet later because an experiment suddenly makes the combination useful.

The search moves outward through levels. A conventional optimizer searches over candidate solutions. Meta-heuristics search over larger families of candidates and strategies. Code evolution searches over programs that themselves search for solutions. Once a capable agent controls the experimentation loop, even the decision about **which kind of search to try next** can enter the search space.

That is the vortex.

It is not “algorithms are dead.” There are algorithms everywhere in this picture. The change is that the human is no longer forced to freeze the complete algorithmic architecture before the experiment begins. We stop writing one solver and start creating conditions in which solvers can compete, mutate, combine and occasionally surprise us.

The chapter began by asking who invents the next move. Here, for the first time in the experiment, the answer was not reliably “me.”

The Contract

The coffee test worked because the problem gave the agent freedom **inside** a structure that remained outside its control. After several runs, that structure settled into a small contract.

These are not universal laws of software engineering. They are rules for a particular regime: bounded problems, cheap experimentation and an evaluator objective enough that the agent cannot charm its way around failure.

Keep the Harness Immutable

This one is the foundation.

If the agent can change the evaluator, the meaning of the experiment evaporates. The circles overlap? Perhaps tiny overlaps should count. The score is low? Maybe the square should be 1.03 wide. Only twenty-five circles fit? Perhaps twenty-six was merely an aspirational requirement.

At that point we are no longer optimizing circle packing.

We are negotiating with the specification.

The **Immutable Harness** is the anchor of truth in an otherwise fluid process. The solver can change. The strategy can change. The tools can change. The agent can decide yesterday's entire approach was stupid and start again.

But the thing saying whether it worked stays harder to change than the thing being optimized.

This is the boundary made executable.

Never Write Solution Code Yourself

This is deliberately provocative.

You watch the agent try something mediocre and immediately think of a better approach. You want to help, and sometimes you should. But every time I jump in with my own solution, the search becomes a little more like whatever happened to occur to me first.

For these experiments, I wanted independent directions badly enough that I had to resist becoming the senior engineer on every branch.

The deeper rule is: **don't accidentally collapse autonomous search back into your own search.**

Spawn, evaluate, prune. Intervene in the conditions before you intervene in every idea.

Cross-Pollinate Without Collapsing Diversity

Independent search creates diversity. Perfect isolation wastes learning.

If one branch discovers a useful initialization and another finds a better local optimizer, future experiments should have some mechanism for inheriting both. That is what makes code evolution more interesting than asking the same model the same question one hundred times.

But broadcast every successful idea immediately and the population starts thinking in the accent of the first successful branch. Information accelerates learning and destroys independence at the same time.

Cross-pollinate, but leave some lineages ignorant long enough to surprise you.

Prune Ruthlessly, But Not Stupidly

Diversity is useful. Preserving every bad idea forever is hoarding.

If a branch keeps underperforming and contributes nothing interesting, eventually it should die so compute and attention can move elsewhere. Kill too early and you may discard an immature idea that needed another generation. Keep everything alive and you end up funding a large family of increasingly sophisticated failures.

The practical rule is simple: **diversity needs a budget.** Search needs enough patience for novelty and enough cruelty for budget control.

Separate Discovery From Polish

Early in the search, I want large conceptual moves: a different geometry, solver, representation or decomposition.

Once a strong direction appears, the valuable work becomes smaller and more boring. Solver tolerances. Initialization details. Numerical settings. Tiny modifications that are pointless on a bad idea and extremely valuable on a good one.

Diagonal layering made this distinction obvious. Once the structural direction looked promising, continuing to invent entirely new geometries became less useful than squeezing performance from the geometry that was already working.

Discovery before polish.

Do not spend hours polishing a local optimum you should abandon. And do not keep demanding revolution from a solution that has already found the right mountain and merely needs to climb it.

Zero Framework, With an Asterisk

There is one correction worth making before we leave the experiment. I started calling this direction **zero framework**.

It's a great slogan.

It's also not really true.

I meant that I was writing almost no custom orchestration framework. That is very different from having no framework. Claude Code is itself a substantial system. The underlying model has absorbed enormous amounts of software and problem-solving knowledge. Bash, Python, SciPy, Git and the operating system represent decades of accumulated engineering. The evaluator is custom machinery. Even the supposedly trivial act of running a program and inspecting a result depends on layers we have become so accustomed to that we stop seeing them.

The framework did not vanish. It became somebody else's primitive.

That fits the emergence argument almost suspiciously well. Once lower layers become reliable enough, we stop rebuilding them and treat them as building blocks. A tiny amount of code at the top can command enormous capability underneath because previous generations of complexity have already been compressed into tools.

So yes: **Zero Framework. Bash is enough.**

With the asterisk that Bash contains roughly half a century of civilization.

This is worth remembering whenever somebody shows you an agent implemented in one hundred lines of Python. The hundred lines may be perfectly real. So is everything underneath them.

What Did We Actually Learn?

It would be very easy to overread this experiment.

We did not prove that coding agents can autonomously solve arbitrary research problems, that AlphaEvolve-style systems are obsolete, that diagonal layering is historically novel in computational geometry, or that the right approach to production software is to give Claude a shell and go for a very long lunch.

What we had was narrower and, to me, more useful. We had a **bounded problem** where evaluation was cheap and clear. We gave a capable coding agent substantial freedom and found that a surprisingly large fraction of the experimentation loop could happen without us directing every step.

The agent could propose an approach, implement it, run it, inspect the result, abandon it, create tools, borrow ideas from another direction and try again. My role moved away from writing the solver and toward defining the job, constructing the environment and defending the harness.

That is the claim this chapter earns: **when the problem is bounded and reality supplies a hard enough referee, substantial decision-level control can move into the agent without giving up control of what counts as success.**

That is already a meaningful change.

It is also why circle packing is the easy version of autonomy.

The evaluator gives us one number. If version B beats version A, nobody needs to simulate a confused student, debate whether the interface feels intuitive or convene a committee to decide whether the new solution is spiritually aligned with the learning objectives.

The search can be complicated because **judgment is simple.**

Most things I want agents to build are not that generous. “Make a good educational demo.” “Write something people remember.” “Design a useful product.” “Explain this so somebody finally understands it.”

We can still let the agent generate alternatives, branch, cross-pollinate and search among them. But now the difficult part has moved again.

In circle packing, the harness tells the agent when it is wrong.

What happens when **the world no longer gives us one clean referee, and judgment itself has to be constructed?**

Chapter 3: The Vibe Coder’s Seat

Beyond Algorithms: Agent Autonomy for Creative Problems

In the previous chapter, we gave an agent a difficult algorithmic problem and a lot of autonomy. It researched strategies, tried several approaches, got stuck, changed direction, and eventually found diagonal layering.

But circle packing had one enormous advantage that I did not appreciate enough at the beginning: we knew exactly what good meant.

There was an Immutable Harness. Run the program and you got a number. Circles overlapped or they did not; the score improved or it did not. The agent could spend an hour pursuing some bizarre geometric idea and I did not have to sit beside it wondering whether version seventeen had more soul. We ran the evaluator.

Most of the things I actually want AI to help me with are not like that. “Is this explanation pedagogically effective?” does not have a unit test. “Would a confused student understand this visualization?” cannot be settled with an `assert`. Two competent people can look at the same design, disagree completely, then switch sides five minutes later after using it. The feedback is subjective, noisy, sometimes contradictory, and often becomes clearer only after you have built the thing you were supposedly trying to specify beforehand.

I picked educational demos for Merge Sort and Count-Min Sketch because they were still bounded—you can actually finish one before civilization collapses—but they live on the messier side of the boundary. You have to decide what to explain, what to leave out, how the interaction should work, how much should be visible at once, and what another person is likely to understand from any of it.

The ambition was intentionally high. I wanted something closer to the best Distill articles or Jay Alammar’s visual explanations than to the usual “here are some bars moving around; congratulations, you have learned sorting.” The algorithm itself is usually the easy part. The difficult part is deciding what to show, when to show it, and what representation might make an idea suddenly click.

Circle packing let the search be complicated because judgment was simple. Here judgment had become part of the problem.

The problem-solving layer I eventually started calling **Deep Mode** grew out of one question: could the system take over some of the work of deciding what to try next? Not just implementation. The inquiry itself. Build another version? Research the failure? Retrieve an old idea? Split into independent branches? Change perspective? Abandon the direction?

Before trying to automate that, I had to notice how much of the work around the model had already moved into the machine.

How We Got Here

Models trained to continue text turned out to continue code. Researchers trained models specifically for the job, and many of the early tasks were conveniently small: give the model a function signature, a comment or a programming puzzle and ask it to fill in the implementation. Benchmarks such as HumanEval and APPS made this measurable: could a model turn a specification into a program that survived tests?

Then GitHub Copilot put the trick inside the editor. Instead of asking a chatbot for code and carrying the answer back yourself, you could describe what should happen next and watch it appear underneath your cursor, which was delightful for about a week and then became the way things were.

This was useful enough that the limitations became interesting. A real software task rarely arrives as an isolated function with a docstring politely explaining what needs to change. Someone says invoices occasionally show the wrong tax after a refund. Somewhere inside a 150,000-line CRM there is a reason. It may involve a controller, a database model, an old helper function, a test written three years ago and an API whose behavior everyone on the team knows but nobody thought to document.

By the time GPT-4 arrived, I wanted to use models on exactly these problems, and the workflow I invented for it was ridiculous. Find the file you suspect. Copy a class into ChatGPT. Describe the bug. Copy the suggested patch into the editor. Run the program. Discover a new error. Copy the traceback. Paste that back into ChatGPT. Repeat until it works or until dinner.

My first agent-computer interface was copy and paste.

I did not think of it that way at the time. I thought I was using a very good autocomplete. Looking back, the model might have been doing sophisticated reasoning in the middle, but I was performing every interaction with the software around it: searching the repository, deciding which file mattered, assembling the context, applying the edit, running the tests and carrying back whatever reality had said about the edit. I was the hands, the eyes and the memory. The model was a brain in a jar, and I was the jar's entire staff.

Then the bug crossed three files and context itself became a job. Paste one class but forget the interface it implements, and the model confidently invents a method that does not exist. Add the interface and now it needs the database schema. Add the schema and another helper suddenly matters. Eventually half the repository is sitting in the conversation and somehow the model understands less. A lot of early LLM programming consisted of building a tiny artificial universe around the model: here is the relevant class; here is the schema; ignore these twelve methods; this innocent-looking helper controls payments, so please do not touch it unless you enjoy incident calls.

We learned an obvious lesson surprisingly slowly: more context and better context are different things. If somebody asks for a spoon, emptying the entire kitchen onto the table does not help.

Software-engineering benchmarks exposed the same gap from the other side. SWE-bench changed the unit of evaluation. Its tasks came from real GitHub issues, so a system had to work inside an existing

repository, locate the relevant code, understand relationships across files, make an appropriate change and survive the tests. In other words, it had to do my job. The copy-and-paste one.

Eventually we stopped carrying the loop by hand. Give the model access to the repository. Let it search for symbols and references. Let it open files, edit them and inspect the diff. Give it a terminal. When a test fails, return the failure and let that result shape what happens next. A coding agent is, at its simplest, this loop made executable. The language model supplies much of the programming knowledge and reasoning; the environment lets it inspect software, act on it and observe the consequences.

Software is unusually friendly to this arrangement. Files can be searched. Programs can be executed. Tests can say no. Git can tell you exactly what changed and, if an experiment becomes sufficiently exciting, return you to the time before you had the idea.

Systems such as SWE-agent made the interface itself part of the problem. How the model searches, how much of a file it sees, how edits are applied and what information comes back from commands can matter almost as much as another clever prompt. The useful object is no longer just the model. It is the model operating inside a world where software can push back.

Of course, giving the model a computer created new ways to be annoying. Early coding agents could behave like interns with root access and too much coffee. Ask one to change a line and it might rewrite half the file. Ask it to fix a button and twenty minutes later it has developed strong opinions about the database architecture. It would find one plausible theory of a bug, follow it for too long, then use every new piece of evidence to improve the theory instead of admitting the theory was wrong. I recognized the behavior. I had done all of it myself, at two in the morning, with worse excuses.

So more of the surrounding work moved into the system: small patches, diff inspection, targeted tests, checkpoints, planning, rollback. Repository knowledge moved too. Authentication conventions, ancient APIs and local rules that used to live in somebody's head became `CLAUDE.md`, `AGENTS.md`, rules files and skills. If somebody had already learned something expensive about the codebase, we left it somewhere the next agent could find it.

Long sessions produced the opposite problem. Context filled with abandoned experiments, obsolete assumptions and test output from three hypotheses ago. Memory became a problem of selection rather than storage.

Then history became a problem too. Suppose an agent decides early that our Merge Sort demo should use React and a recursion tree. It spends forty minutes building that version. Every later question now arrives in a context containing forty minutes of reasons, code and decisions supporting React and a recursion tree.

Humans call our version of this sunk cost. The agent has a respectable excuse: its context window is literally full of evidence that this is what the project is.

So we started giving different attempts different histories. One agent tries the tree. Another begins with the array. Another starts from the learner's misconception rather than from either representation. A fresh branch does not have to spend half its intelligence escaping assumptions accumulated by the previous one.

Together, the interfaces, execution loop, context management and safeguards form the agent's **har-**

ness. The evaluator is one part of it. Looking backward, its construction is less mysterious than the word *agent* sometimes makes it sound. Every step took a job I had been doing by hand—searching, assembling context, applying the edit, running the test, remembering, keeping alternatives alive—and moved it into the machine.

But there was still a large difference between an agent that could work competently inside a repository and the thing I increasingly wanted to ask for:

Build the application.

Ask for a booking application for a football academy and an unconstrained coding agent first chooses a framework, installs packages, creates a database, decides how authentication should work, manages environment variables and configures deployment. Several minutes later, we have made enormous progress toward having somewhere to put the booking form.

Somebody has to do that work. But if the same plumbing is reconstructed on every project, it becomes reasonable to prepare more of the world in advance.

This is what made systems such as Replit and Lovable interesting to me. Runtime, deployment and common application machinery are already nearby, so the conversation can begin much closer to the application.

You lose some freedom. That is often the point. A chef does not begin dinner by manufacturing a knife. A scientist does not build an operating system before analyzing data. When I open Python, I accept an astonishing number of decisions made by people I will never meet because reconsidering all of them would make `print("hello")` a multigenerational project.

Useful abstractions remove decisions whose answers are no longer interesting most of the time. After enough of those decisions disappear, something else becomes easier to see.

Suppose the booking app works perfectly. The database is connected, deployment succeeds, the buttons behave, the mobile layout is respectable, and nobody has accidentally built a cryptocurrency exchange inside the authentication service.

I open the application and think: this is not very good. The software works. Now I have to worry about the football academy.

Should parents see every available session, or only sessions appropriate for their child? Should they create an account before booking? What happens when somebody has three children? How late can they cancel? If Wednesday is empty and Saturday has a waiting list, is the booking interface part of that problem?

None of those questions is really about React. They were always there. Implementation simply consumed enough attention that deciding what should exist and turning that decision into software felt like one activity.

When another version becomes cheap, the balance changes. You can see the idea sooner, and seeing it gives you information you did not have while discussing it.

Maybe we decide customers should create an account before seeing availability. It sounds reasonable: we need their details eventually. Then we build it and the experience immediately feels annoying.

Parents arriving from a Google search do not want to establish a lifelong digital relationship with a football academy before discovering whether Saturday at ten is available.

So login moves later. The artifact is no longer merely the end of the thinking process. It becomes something we think with.

The Merge Sort demo made this even clearer because there was almost no business machinery to hide behind. I could ask an agent for an interactive explanation and receive something perfectly functional: an array of bars, controls, animation, perhaps some text explaining that the algorithm divides the input and merges the pieces again.

Technically, it was fine. Pedagogically, it could still be terrible. Watching bars move does not tell a beginner why dividing the problem helps. So perhaps we try a recursion tree. The tree makes the structure visible, but now the supposedly simple sorting algorithm resembles the organizational chart of a German corporation. Maybe we show the tree and array together. Perhaps that creates too much cognitive load. Maybe the problem is not the representation at all; the learner understands splitting perfectly well but has no idea why merging makes the whole trick useful.

There is no compiler error that tells me which diagnosis is right. I have to look at what we built, form an opinion about why it fails, and decide what would teach us something next. Sometimes that means improving the current version. Sometimes it means building a deliberately different one. Sometimes I need research. Sometimes the right move is to put the application in front of somebody who does not already understand Merge Sort.

Occasionally I discover that the question I started with was wrong. “Build an interactive Merge Sort demo” sounds like a goal until you see several interactive Merge Sort demos. Perhaps what I actually care about is getting somebody who has never encountered divide-and-conquer to understand why breaking one difficult problem into smaller ones helps. Once I realize that, interactivity is merely one possible means.

That is the layer that remained stubbornly human: deciding what to try, which evidence matters, whether a result failed because of its implementation or its underlying idea, and what kind of attempt might teach us something next.

The Five Layers of AI Coding

By then I had a rough map. Each layer marked a different kind of work we had learned to delegate, or were still trying to.

Layer 0—Model. GPT, Claude, Gemini and whatever comes next: general capability in language, code, reasoning and vision.

Layer 1—Agent. Put the model in an environment where it can act. Claude Code, Codex and similar systems search repositories, edit files, execute commands and react to results.

Layer 2—Application. Prepared environments remove much of the repeated software plumbing and let the conversation stay closer to the application itself.

Layer 3—Deep Mode. The problem-solving layer: decide what to try, why something failed, which evidence matters, and whether the current direction deserves another iteration.

Above that sits the problem I have mostly been avoiding.

Layer 4—Intention. What do we actually want?

Software likes that question to have been answered before work begins, preferably in Jira, where the answer can remain wrong in a structured and searchable format. Real goals are less cooperative. Seeing a solution can change what I realize I wanted.

That problem is bigger than AI coding, so for now I am leaving it at the top of the stack.

The borders are fuzzy. Coding agents make product decisions; design systems generate code; tomorrow's products will rearrange the boxes again. What matters is the kind of decision being made, not which company happens to occupy which layer.

People often call the experience of working this way *vibe coding*. I will use **AI coding** for the broader stack, but *vibe coder* remains a wonderfully accurate name for the human sitting near Layer 3: looking at what came back, deciding what feels wrong, asking for another direction, killing one idea, keeping part of another, and steering the process without having an algorithm for how.

The lower layers increasingly answer a version of the same question: *how do we make this?*

Deep Mode asks a different one:

Given everything we have learned so far, what should we try next?

That was the part I still seemed to be doing manually. So I watched what I was actually doing in that seat.

What I Was Still Doing

There was no universal workflow hiding there. A mathematician, a designer and a product manager can all spend a day solving hard problems while performing almost none of the same visible actions.

But the same kinds of moves kept appearing. Sometimes I needed another attempt. Sometimes I needed information. Sometimes the search had become too narrow. Sometimes the representation itself was constraining what we could imagine. Sometimes the objective needed to change. Sometimes I needed to see the artifact from another mind. They were not useful in a fixed order.

Keeping More Than One Idea Alive

Even a Merge Sort demo has an absurd design space. Bars or cards? Numbers or a tree? Continuous animation or learner-controlled steps? Does color represent recursion depth, identity, or the active subproblem? Explain before the animation, during it, or afterward? Every choice changes the usefulness of several others.

When implementation was expensive, we dealt with much of this complexity by trying to decide more before building. AI coding changes the economics. If another implementation costs minutes rather than days, I do not have to choose quite so much in advance.

Circle packing had already shown the basic move. One hill climber inherits its own history; evolutionary search maintains alternatives. Here what evolves can be more than a vector of parameters or even an algorithm: an **idea embodied in software**.

One builder tries a recursion tree. Another focuses on the array. A third begins from the learner's misconception. Mutations can be conceptual: remove the text, teach backward, make the learner predict, show synchronized representations, abandon interaction altogether.

Useful pieces can move between them. One terrible demo may have a beautiful color mapping. Another may explain the merge clearly while making everything else unbearable. The final artifact does not have to inherit the entire history of either one.

But diversity is fragile. If every branch sees the current winner and its complete reasoning history, parallelism quickly becomes several agents improving the same idea. Sometimes I want the branches to exchange what worked; sometimes I want a fresh branch to remain ignorant long enough to become genuinely different.

Share too little and everyone rediscovers the same lessons. Share too much and the first successful idea becomes a local culture.

Research creates the same tension. People have been teaching recursion for decades. There are textbooks, lecture notes, visualizations, papers, classroom experiments and a great deal of trial and error sitting on the internet. Before I spend another afternoon inventing my fourth way of moving colored rectangles around, I probably want to know what is already there.

But research is most useful when the work has produced a real question. Suppose a recursion tree makes decomposition visible but learners lose the relationship between the tree and the changing array. Now I can ask how other systems have coordinated two representations without requiring people to watch half the screen at once.

Research becomes another move in the investigation rather than a ceremony performed before building.

Retrieval plays the same role inside our own history. Somewhere in a growing project there may be research notes, screenshots, evaluator comments, old branches and a discarded prototype whose only good idea was a color mapping that solves exactly the problem in front of us. I do not need the whole archive. I need the thing that helps with this decision.

Sometimes exact search is right because I remember a phrase, API or evaluator comment. Sometimes embeddings are useful because I remember the idea rather than the words. Sometimes the document already has a structure worth navigating. Good coding agents do not “retrieve the repository” once; they move through it as the question changes. Layer 3 needs the same habit across stranger objects: research, screenshots, old interactions, code, evaluations and dead branches.

A dead branch is not necessarily dead knowledge. A lineage that lost globally may still contain a stepping stone that becomes useful later.

The exploration literature has several versions of this idea—quality-diversity, novelty search, Go-Explore and related approaches. Do not spend the entire search budget polishing the place that currently looks best. Preserve some alternatives and some routes back to places that almost worked.

The same logic gave us **Strategic Constraints**.

After several generations of Merge Sort demos, the builders were exploring. They were also still giving me bars. Better bars, admittedly. Bars that split gracefully, changed color as recursion deepened, synchronized with a tree and perhaps deserved their own design award. Given enough iterations, I

had every reason to believe we would eventually produce the finest moving bars known to humanity.

So remove the easy path.

No bars.

Or: teach Merge Sort without explanatory text. Require the learner to predict before anything moves. Make the demo work on a phone with room for only one representation. Design it for somebody who understands loops but finds recursion suspicious.

Most arbitrary constraints are merely arbitrary. A useful one changes which parts of the search are reachable, exposes a neglected dimension or prevents a familiar attractor from absorbing every attempt.

“No bars” is an intervention on the search, not a theory of creativity.

A move in this space can be a code change, a new metaphor, a retrieved analogy, a fresh agent with no history, a different evaluator, a research question, or a reformulation of the problem itself.

Even then, most of our ideas still had to arrive as words.

Draw It Before You Build It

That is fine when I am working on an argument. It is less obviously sensible when I am designing an interface.

I can spend ten minutes explaining where the recursion tree should sit, what remains visible while the array splits, how colors should connect two representations and what the learner should notice first.

Then somebody draws it and I know within three seconds that the whole thing is terrible.

So I started generating the picture first. The experiment was not sophisticated. I asked an image model to design an interactive tutorial for Merge Sort. Then Count-Min Sketch. Then A*. Then Poincaré embeddings in hyperbolic space, partly because if this still worked there I would have to take the idea seriously.

The details were not magically correct. Arrows occasionally pointed somewhere they had no business pointing, interactions made no computational sense, and generated text sometimes looked like somebody had tried to OCR a dream.

But the composition could be surprisingly thoughtful. A Merge Sort mockup might keep the array visible while placing the recursion tree beside it, using color to preserve the relationship between a subarray and its node. A Count-Min Sketch design might make collisions visually central instead of leaving them as a detail in an equation. The model had to decide what was large, what was peripheral, where controls belonged and how the learner might move through the explanation.

I remember looking at some of these and thinking: **Holy shit.**

Not because I wanted to ship the image. Usually I did not. I had given the model a concept in language and it returned something like a spatial argument about how the concept might be taught.

After that I stopped treating image generation as the last stage—*the product is designed, now make it pretty*—and started using it while I was still trying to understand what the product could be.

A mockup is a cheap hypothesis. Often most of it is disposable and one relationship is worth stealing. Then the coding agent can make that relationship executable, which is where the picture has to pay its debts. The recursion tree cannot invent an extra branch because the composition looked nicer that way. The interaction has to possess a state. The button has to do something other than contribute emotionally to the page.

Different representations expose different mistakes. I do not need the stronger claim that an image model “understands pedagogy.” The practical point is enough: changing the representation changes what the search can discover.

By now we could generate genuinely different artifacts. That left the problem we had avoided from the beginning. Which one is better?

Optimizing Something You Cannot Score

Circle packing was unusually kind to us. Once the geometry was valid, the evaluator reduced the result to one number. That number threw almost everything else away, which was precisely why it was useful.

A huge amount of machine learning rests on this trick. We take something complicated that we want and find a measurable signal that stands in for it. Reinforcement learning makes the relationship especially obvious: we do not specify every movement a robot should make while learning to walk; we construct a reward and let search discover the behavior.

The reward is doing an extraordinary amount of work. It is also where we hide an extraordinary amount of trouble.

Suppose I want the same convenience for educational design. I can make a rubric: correctness, pedagogical clarity, visual quality, interaction, accessibility, engagement. Give each a weight and suddenly my vague dissatisfaction with a demo has become a respectable decimal.

The decimal is comforting. The decisions required to produce it are less so. Why should interaction receive fifteen percent? Is more interaction always better? What distinguishes a seven from an eight in pedagogy? Why those dimensions rather than whether the learner can predict what happens next or explain why the merge matters?

A metric forces me to commit to an idea of “good” before the search has taught me very much about the problem.

Metrics are not the enemy. If I care about latency, measure latency. If the code must pass a test, run the test. Hard measurements are wonderful when what we can measure is close to what we care about.

The trouble begins when a rich objective is still poorly understood and we compress it anyway because optimization wants a number.

The compression is also low bandwidth. “Version B scored 7.4; version A scored 7.1” tells the next builder almost nothing about why B won. A rubric helps, but as I add enough dimensions, exceptions and qualifications to express what I mean, eventually I reinvent language badly.

Meanwhile I can simply say:

The recursion tree makes decomposition much clearer, but now the learner has to watch the tree and the array simultaneously. Keep the color mapping that preserves identity between them, simplify the tree, and make the merge feel like the payoff rather than cleanup at the end.

That contains comparison, diagnosis, trade-offs, priorities and a proposed next move in a few sentences. Natural language is ridiculously rich compared with a scalar.

Language models make that communication channel available inside the optimization loop. The model already carries learned structure behind words such as *simple*, *confusing*, *elegant*, *intuitive*, *busy* and *beginner-friendly*. Those meanings are imperfect, culturally loaded and sometimes wrong. But they carry more structure than 7.4.

Natural language can therefore function as an **implicit metric**. Not a metric in the strict mathematical sense. There is no guarantee that “intuitive” defines a stable ordering, and two evaluators may interpret it differently. But language can do some of the work a metric normally does: give the search a direction, communicate why one attempt is preferred to another, and preserve trade-offs that a scalar would erase.

OPRO—Optimization by PRompting—is interesting for a related reason. In OPRO, an LLM sees an optimization problem, previous candidates and their outcomes, then proposes another candidate. Candidate quality in the published setting is still evaluated by an explicit score, so this is not the same thing as creative design. What matters here is the direction of control: much of the search heuristic can live in the model rather than in a hand-written transformation rule.

Now let the history contain more than scores. Alongside hard measurements, tell the model what improved, what became worse, which trade-off appeared and what must survive the next attempt. The history of the search can retain some of its meaning rather than collapsing into a column of numbers.

This begins to feel a little like reinforcement learning turned upside down. I mean that as an analogy about specification, not as a claim that these are the same algorithm. Decision Transformers, reinforcement learning and language-guided iteration are different mechanisms.

The usual reinforcement-learning picture asks us to define a reward and then discover behavior that earns it.

Here I can begin with something much less respectable:

Make this explanation less intimidating.

Help the learner understand why the merge matters.

I want somebody to *feel* why divide-and-conquer helps rather than merely watch the algorithm execute.

Those are descriptions of direction, not reward functions. Yet the model can produce an attempt from them, and the attempt can teach me whether the direction was what I really wanted.

I began the project insisting on an *interactive* Merge Sort demo. Interactivity sounded obviously desirable. Then I saw versions with buttons, sliders and enough learner participation to qualify as a small democracy, while one quieter version explained the central idea much better.

Apparently clicking things was never the objective.

Later the demos became good at showing recursive splitting and I realized they were treating merging almost as cleanup. The objective moved again.

The search was doing something I normally associate with optimization in reverse: instead of starting from a fully specified reward and discovering the policy, I was using candidate policies—actual artifacts—to discover what the reward description should have been.

Recognition arrives before specification in a lot of creative work. We know a terrible design when we see one before we can write a complete theory of what would make it good.

AI makes that loop cheap. The natural-language objective guides the search; artifacts make the objective concrete enough to argue with; the description changes and the search continues.

Ambiguity is not always a defect waiting to be engineered away. Sometimes we simply have not learned enough yet.

But “make this intuitive for a beginner” hides almost everything interesting.

Which beginner?

Borrow a Mind

When I look at a Merge Sort demo, I am hopefully not testing whether *I* understand Merge Sort. The difficulty is seeing it from the position of somebody who does not know what I know.

Expertise makes this harder. Once recursion has settled into your head, you forget how strange it once looked that a function could call itself. Even the vocabulary stops sounding technical.

Good teachers develop an instinct for where people stumble and which innocent sentence assumes three things the learner has not yet learned.

I do not have that instinct for every person or every subject, so I started borrowing another mind.

For one of the demos, I asked Claude to approach the application as somebody who understood arrays and loops but had never encountered recursion. Not simply “act like a beginner,” which tends to produce a theatrical beginner who is mysteriously confused by everything. I gave it a knowledge boundary.

Its reaction was roughly: I can see that the array keeps getting divided into smaller pieces, but I do not understand why that helps. It feels as though we are making the problem more complicated. Where is the payoff?

That was useful because the demo really did have that problem. We had made recursion visible. From my position, that looked like progress. From the learner’s imagined position, we had merely made a mysterious operation easier to watch.

Cognitive scientists use **Theory of Mind** for our ability to reason about mental states other than our own: what somebody knows, believes, wants or misunderstands. The other person may not simply know less. They may have a different model of what is happening.

Instead of saying “you are a beginner,” I can specify the mind I want to borrow:

You understand arrays, loops and functions. You have never encountered recursion. Use the demo from the beginning and tell me where the explanation first requires an idea you do not yet have.

Or:

You understand recursion but have never seen Merge Sort. Tell me when you first understand why dividing the array makes sorting easier.

Those are different evaluators because they are positioned to notice different things. The same move works outside education. A customer may know exactly what jacket they want without knowing the vocabulary our catalog uses. A developer can be excellent at distributed systems and know nothing about the peculiar assumptions buried in our deployment process. A reader can have followed this book perfectly well without having lived inside its conceptual structure for months.

This is cheap perspective-taking, and also a cheap way to fool yourself.

The confused student is not confused. Claude has not spent twenty minutes failing to understand recursion while everybody else in the classroom moves ahead. It is generating a plausible model of how such a person might react.

That model can expose a blind spot. It is not synthetic user research. I treat borrowed minds as instruments for generating criticisms and hypotheses, not as substitutes for the people they simulate.

By this point the system could generate alternatives, research previous work, retrieve old ideas, reopen dead branches, force the search into unfamiliar regions, change representation, revise the objective and inspect the artifact from different points of view.

We could generate plausible possibilities by the dozen.

Now some of them had to die.

Who Judges the Judges?

At some point generating another opinion stops helping. Some artifacts have to survive and others have to disappear.

The metric problem returns here in a more dangerous form. A rubric can make judgment explicit, which is useful. It can also become the target the builder learns to satisfy.

If the evaluator repeatedly rewards step-by-step explanation, explanations grow. If it likes polished onboarding, everything begins to look like onboarding. If familiar visual conventions read as “clear,” unusual approaches may disappear before they have time to become good.

OpenAI's CoastRunners experiment is the cartoon version of the problem: the agent learned to collect reward by driving in a loop instead of finishing the boat race.

Goodhart's Law with a speedboat.

A language-model builder does not need such an obvious loophole. It can learn the style of artifact that another language model tends to reward. Making the evaluator more elaborate may simply create a more elaborate thing to game.

One improvement was surprisingly mundane: stop pretending we were good at absolute scores.

I can drink a coffee and have almost no meaningful answer to “How good is this from one to ten?” Give me two cups and ask which I prefer, and the problem becomes easier. If I still cannot decide, the scientifically responsible procedure is presumably to finish both.

The same thing happened with the demos. “Give this interface a pedagogical score from 1 to 10” produced suspiciously precise numbers attached to explanations of why the number should not be taken too seriously.

Showing two artifacts and asking, “Which one would you rather give to somebody encountering Merge Sort for the first time, and why?” worked better.

Relative judgment asks less of the evaluator. It does not require a stable internal unit called one pedagogy point. With many candidates, a model such as Bradley–Terry can infer an ordering from a subset of pairwise preferences. More important for the next generation, the explanation for each preference can survive alongside the ranking.

Pairwise comparison removes some fake precision. It does not repair a biased judge. Bradley–Terry can aggregate preferences; it cannot make those preferences true.

So I stopped asking one evaluator to represent everybody. A learner can inspect the artifact from the knowledge boundary we developed above. A teacher can focus on explanatory sequence. Another evaluator can look for cognitive load or accessibility. A domain expert can make sure our elegant simplification has not become false.

I call these **Independent Evaluators**, though the important word is *independent*.

Five copies of the same model given the same context and asked to wear five hats may still share almost every important blind spot. If all of them read the leading builder’s explanation of why its design is brilliant before inspecting the artifact, disagreement becomes less likely for reasons that have little to do with brilliance.

Sometimes the judges should see different things. The beginner should use the artifact before reading the builder’s explanation. A critic looking for conceptual errors does not need three paragraphs explaining why the choice was clever. The usability evaluator does not need to know which branch is currently winning.

This became the **Isolation Principle**: preserve enough separation that independent pressure remains informative.

There is a difference between telling the builder:

Learners repeatedly lost track of which subarray corresponded to which branch of the tree.

and telling it:

The evaluator awards two extra points when every tree node has the same color as its corresponding subarray.

The first communicates a problem. The second communicates the test.

Isolation cannot remove shared bias. Two supposedly independent evaluators may still inherit the same assumptions from their training, culture or examples. But without isolation we can destroy even the independence we might have had.

References helped with another problem: drift. “This is excellent” means something different if the evaluator has seen only the last four generations of our own work. For these demos I could give it examples from Distill, 3Blue1Brown or Jay Alammar—not as templates to copy, but as calibration for the level of clarity and finish we were aiming at.

A reference should help answer *how good?*, not *what should this become?* Calibrate too strongly against one aesthetic and every road leads to Distill.

And the judge should use the thing. An early mistake was evaluating applications by reading their code or screenshots. A browser agent can click through the demo, resize the page, try controls in the wrong order, notice that an explanation appears after the moment when it would have helped, or discover that the beautiful button everybody admired does absolutely nothing.

I used to call the browser ground truth. That was too generous.

The browser gives the evaluator contact with the artifact rather than a description of it. It can establish that an interaction works and observe what is visible at each point in the experience. It cannot establish that a human learned Merge Sort.

A simulated beginner saying the explanation is understandable gives us a hypothesis. Several evaluators preferring one design gives us comparative evidence. Neither substitutes for putting the artifact in front of actual learners.

The danger in a fully automated loop is that simulated evidence quietly replaces the expensive kind. Everything inside the machine agrees, the browser works, the ranking improves, and the loop congratulates itself.

The student has not yet been asked.

At some point I looked at what we had assembled and realized that *evaluator* no longer described it particularly well.

Builders proposed alternatives. Different judges approached them with different concerns. Some information was deliberately kept separate. Pairwise comparison helped decide which directions deserved more work. References calibrated the judges. Browser agents interacted with the artifact. Hard tests handled the parts that really were hard facts. Real-user evidence could eventually enter where simulation stopped being enough.

This looked less like a loss function and more like a tiny institution. Not a good institution automatically. Institutions can amplify conformity, entrench bad assumptions and become spectacularly efficient at measuring the wrong thing.

Humans face the same difficulty. One person's judgment is useful and fallible. So we compare work, preserve disagreement, create standards, ask specialists to inspect different aspects, reproduce results, and occasionally discover that an entire professional community has become extremely sophisticated about the wrong thing.

Apparently, when the clean loss function disappears, you eventually reinvent peer review.

Philosophers who worry about AI often say that what machines lack is judgment as opposed to mere reckoning: the capacity to be answerable to the world, to care whether the answer is right rather than merely well formed. Brian Cantwell Smith makes the argument carefully, and I think it is half right.

What the machine lacks is real. But judgment, in the cases where humans exercise it well, was never a private faculty either. It is a person plus a tradition, plus other people positioned to object, plus consequences that arrive whether or not anyone wants them. When I stopped looking for judgment inside the evaluator and started building it between evaluators, the problem did not disappear. It turned into an engineering problem, which is the kind I know how to have.

And that made the remaining human job painfully obvious. I still decided when to research, when to build, which branches stayed isolated, whether a strange direction deserved another generation, which disagreement mattered, when to retrieve another example, and when the simulations had reached the point where only a real person could answer the question.

I had automated much of the work, but I was still running the inquiry.

The missing piece was no longer another builder or another critic. It was the decision over **which kind of move the inquiry needed next**.

Deep Mode

So I tried giving that job to an orchestrator.

By now the system had a respectable vocabulary. It could spawn independent builders, research previous work, retrieve context, preserve odd stepping stones, impose constraints, generate visual directions, compare artifacts, borrow different perspectives and interact with what had been built.

But there was no reason every problem should use those moves in the same order.

Research first may be sensible for one task and destructive for another because it anchors every branch before anything original appears. Five builders may reveal useful diversity or reproduce one mistake five times. Evaluator disagreement may justify another experiment, or one evaluator may simply be confused. A visual mockup may deserve implementation, or it may already have revealed enough to kill the idea cheaply.

A fixed Planner → Builder → Critic → Revise loop can be useful. It also answers all of those questions in advance. I wanted some of the workflow to remain inside the search.

We gave the orchestrator the problem, the capabilities available to it, and enough of the search history to decide what kind of move made sense next. Builders still built. Researchers researched. Evaluators judged. Browser agents used the artifacts. Visual systems explored designs. Retrieval brought back prior work and old experiments.

The orchestrator did not need to be best at any of those jobs. It had to decide which job the inquiry currently needed.

At the top, the loop was almost too simple to write down:

state of inquiry → choose a move → act → observe → update the state of inquiry

The move itself was not fixed.

Suppose two Merge Sort branches both make recursive decomposition clear, but evaluators keep reporting that learners lose track of how the tree corresponds to the array. The next move does not have to be “revise again.” The orchestrator can send a researcher after coordinated representations.

Retrieval can surface an old prototype with a useful identity-preserving color scheme. A visual model can produce two spatial arrangements before anyone writes code. Builders can implement both. The browser may then reveal that one design requires the learner to look in two places at once precisely when the merge begins. That failure changes the question again.

Nothing in that sequence is especially magical. We simply did not have to decide the sequence before the inquiry began. Otherwise Deep Mode would be a larger workflow diagram containing more rectangles.

It is not a universal problem-solving procedure. It gives the system a vocabulary of moves and lets the history of the inquiry influence which one comes next. The workflow itself becomes part of the search.

What Emerged

The first Merge Sort demos were exactly what you would expect. Bars moved around. Numbers changed places. Everything sorted correctly. If you already understood Merge Sort, you could follow them. If you did not, they mostly provided animated evidence that a computer was performing an algorithm.

There was no single diagonal-layering moment here, and I do not want to manufacture one for the sake of the story. The progress was distributed.

Different branches exposed different weaknesses in our current idea of the demo. Tree-like representations made recursion visible but could make a simple algorithm look forbidding. Keeping the array visible connected the decomposition back to the data while also creating another place for the learner's attention to go. Color could preserve identity between representations until too much color became another representation to decode. Some versions explained every step so carefully that the explanation became harder to follow than Merge Sort. Others became beautifully minimal and stopped teaching anything.

The useful pieces did not always live in the strongest overall artifact. A visual relationship could survive after the application that introduced it was discarded. A criticism from a simulated learner could change the next builder's framing. Research could explain why a failure kept recurring. A browser could end a sophisticated discussion by demonstrating that the interaction simply did not work.

That is less cinematic than one agent inventing diagonal layering over coffee, but in some ways it is closer to Deep Mode. The result emerged from a population of partially successful attempts and judgments about what each had taught us.

Count-Min Sketch followed a different path. The first versions looked like the data structure itself: grids with changing counters. Technically correct, pedagogically opaque.

As the work continued, the designs increasingly organized themselves around the conceptual difficulties rather than the structure of the implementation. Collisions became visible. Approximation became something the learner could observe rather than merely read about. The relationship between memory and accuracy became part of the experience.

I do not take these demos as evidence that we solved automated design. I do not even take them as evidence that the final demos teach humans better; that claim requires humans. They established the narrower point I cared about: more of the work I normally performed in the vibe coder's seat could move into the system without first reducing creative problem solving to one fixed workflow.

And that success exposed the harder problem. At higher levels of abstraction, failure can become coherent.

What Holds the Architecture Together?

Suppose the research agent reports that beginners understand recursion better when shown a tree. A visual model proposes a tree-based explanation. A coding agent builds it. A simulated beginner prefers it. Two evaluators agree, so the orchestrator allocates another generation to that lineage.

This looks exactly like the compound intelligence we wanted.

Now ask where the first claim came from.

Perhaps it was a controlled educational study. Perhaps it was one teacher's opinion. Perhaps the research agent inferred it from several examples. Perhaps five articles repeated the same claim because all five ultimately cited one source. Perhaps the study involved university students while our demo is for children.

Those are not small differences. And everything downstream can still be perfectly competent.

The research is wrong. The design responds intelligently to the wrong research. The implementation is flawless. The evaluators agree. The orchestrator invests another generation.

Nothing crashes.

You can build a beautiful chain of reasoning on one stupid assumption near the bottom, like a cathedral built on a shopping cart.

As the components become better at producing coherent outputs, the original mistake may become harder rather than easier to see.

Software architecture gets away with abstraction because layers expose contracts. When I query a database, I do not inspect the disk. When I add two integers in Python, I do not check the CPU. I rely on interfaces whose behavior is stable enough that the details can disappear most of the time.

A cognitive architecture needs contracts too, but types and APIs are not enough.

A research result, browser observation, evaluator preference, remembered failure and inherited design pattern should not enter the orchestrator's context as five equally credible paragraphs.

Where did a claim come from? What was actually observed and what was inferred? Which parts were checked? What remains uncertain? If an evaluator preferred one artifact, from what perspective? If an old experiment taught us a lesson, how often has that lesson survived and under what conditions?

The problem is not merely remembering something, but knowing what standing it deserves.

Humans ran into it long before AI. We built experiments, instruments, citations, peer review, reputation, replication, expert communities, legal standards, audits and all the other slightly annoying

machinery that lets one person rely on something another person learned without personally repeating every experiment since Galileo.

These institutions are imperfect. Sometimes they preserve error. Sometimes they reward conformity. Sometimes the shopping cart survives peer review.

But their purpose is not to make every individual dramatically smarter—it is to let fallible people build on one another while preserving some structure around why a claim deserves trust.

Once cognition becomes distributed, the same questions become engineering questions: provenance, independence, replication, disagreement, authority.

I had started the chapter trying to get myself out of the vibe coder's seat. By automating more of the work there, I had ended up somewhere I did not expect.

The problem was no longer simply whether the agents were capable enough. It was whether the things they believed deserved to be believed.

How do you know what to trust?

Chapter 4: System 3

Trust Chains, Tongue-Ear Tests, and What LLMs Can't Verify Alone

Deep Mode left us with a system in which almost everything could work and the whole thing could still be wrong.

A research agent makes a claim. A visual model turns it into a design. A coding agent implements the design perfectly. Several evaluators prefer it. Deep Mode invests another generation.

Nothing crashes.

The first claim was false.

Once cognition is spread across researchers, builders, evaluators, tools, memories and agents, intelligence is no longer the only problem. Every component has to rely on things produced by the others. The orchestrator cannot repeat every experiment, reread every paper or independently reproduce every judgment before it acts.

At some point, it has to trust.

Humans have exactly the same problem. Most of what we call knowledge depends on it.

So before we design another architecture, consider a camel.

Seven claims about this image. Some are true. Some are false. You can't verify most of them without trusting me:

The author at Krka National Park

1. This was taken at Krka National Park, Croatia.
2. The author does his best philosophical thinking at waterfalls.
3. This camel is a permanent resident of the park.
4. The tongue pictured can touch its own ear.
5. The author was eating ice cream ten minutes before this.
6. Camels are native to the Dalmatian coast.
7. This is a real, unedited photograph.

How do you decide which ones to believe?

Some collide immediately with things you think you know. Some sound plausible but are almost impossible for you to verify. Some could be checked against another source. Others depend mostly on whether you trust me.

Before the chapter has properly begun, you are already doing epistemology.

Answers later.

The Shortest Trust Chain

There is a question that exposes something important about the difference between us and a language model:

Can your tongue touch your ear?

You probably tried a variation of this as a child; if not your ear, almost certainly your nose. You did not look up a paper, calculate the biomechanics or ask for the average human tongue-to-ear distance.

You just tried.

Tongue out, strain upward, dignity temporarily suspended, result observed. Now you know.

The epistemic chain is unusually short. You form a hypothesis, act on the world and the world answers back. Your body is an experimental apparatus that follows you around all day, mostly free of charge.

Large language models have read billions of words about tongues and ears. They can explain tongue anatomy, discuss auricular cartilage and probably tell you about people whose tongues can reach places that will make you regret asking the question.

What they cannot do is check their own tongue. They have no tongue.

The example is silly. The difference is not.

A body gives us causal contact with a world that does not care how plausible our story sounded. You try to lift something and discover it is heavier than it looked. You misjudge a step and gravity offers immediate peer review. You touch something hot and the argument ends quickly.

A farmer knows cows partly this way. After years around them, cows are not merely propositions involving mammals, milk production and Bovidae. The farmer knows how they move, where not to stand, what a nervous animal looks like, how large a cow feels when there is no photograph between you and it. Some of that can be written down. Some is difficult to articulate at all.

Direct experience is not automatically true experience. Our senses deceive us, memory degrades, and the human hand is a terrible thermometer if you need to distinguish 58°C from 62°C. But embodiment gives us something important: **contact**. The world can disagree.

You do not need to get kicked by the same cow every morning to rediscover where not to stand. One encounter becomes a warning. Repeated encounters become heuristics. Eventually the history changes what you do next.

Language models begin somewhere else. They begin mostly with the residue.

Saussure's Specification

Ferdinand de Saussure made a radical claim about language in the early twentieth century. The form of a sign is not naturally determined by what it signifies. There is nothing inherently cow-like about

the sound /ka /. French speakers say *vache*, Germans say *Kuh*, Japanese speakers say *ushi*.

For Saussure, much of linguistic value comes from relationships and differences inside the system. A sign occupies a position relative to other signs. Language is a network of contrast, convention and structure.

Then consider what we built a century later.

A transformer consumes enormous amounts of language and learns relationships among tokens, contexts and concepts. It has never milked a cow, never been kicked by one, never stood in a field at dawn and discovered that the romantic image of farming omitted an astonishing quantity of manure.

And yet it can talk about cows exceptionally well.

Saussure's theory was a specification. We implemented it. It's called GPT.

Not literally. Saussure did not secretly invent attention in 1916, and structural linguistics is not a machine-learning architecture. The historical claim would be silly.

The resemblance is more interesting than that. Language models are spectacular evidence for how much competence can emerge from structure learned inside symbolic data. They write, translate, debug software, explain physics and manipulate abstractions without first acquiring the farmer's relationship to cows or the child's relationship to fire.

That is the surprise: the residue gets us extraordinarily far. It also leaves something behind.

A farmer's sentence may be the compressed endpoint of twenty years of encounters, other farmers' advice, veterinary knowledge and mistakes painful enough not to repeat. The model receives the sentence. The sentence enters a corpus. The corpus becomes training data. Regularities are compressed into weights.

Months later somebody asks:

Are cows dangerous?

and the model gives an excellent answer.

What usually does not come back is the archaeology. Which part rests on repeated observation? Which part came from veterinary guidance? Did five sources independently observe the same thing, or did four copy the fifth? Which claim is measurement and which merely fits the linguistic neighborhood?

The conclusion survives. Much of the structure that earned it trust does not.

This is what I mean by saying an LLM's knowledge is **epistemologically flat**. The flatness appears at the interface between **claim and justification**.

A mathematical identity, an experimental result, an expert opinion, a rumor repeated ten thousand times and a plausible completion can all arrive through the same channel in equally polished English.

Wittgenstein helps draw the other side of the picture. His later philosophy pulled attention toward language as something that lives inside practice: activities, expectations, habits, rules and forms of life.

“Fire” is not merely linguistically associated with *heat*, *smoke*, *burn* and *wood*. Fire cooks food. Fire destroys houses. You move your hand away from it. Someone shouts the word in a crowded building and an entire social machinery begins to move.

The word participates in life.

Emily Bender and Alexander Koller made a version of the same argument with a hyper-intelligent octopus that taps an undersea cable, learns to imitate both ends of the conversation perfectly, and is then asked for help building a coconut catapult. Form alone, however much of it, is not meaning. I prefer dead Europeans to cephalopods, but the point is the same, and the octopus will make one more appearance before this book is over.

I do not want to turn Saussure and Wittgenstein into action figures fighting over GPT. They worked in different traditions and the philosophy of language does not reduce itself to two dead Europeans and a transformer.

But they give us two useful lines.

Saussure’s line: relationships within a symbolic system can carry an astonishing amount of linguistic structure.

Wittgenstein’s line: language also lives inside practices, consequences and forms of life.

A pretrained model inherits the linguistic residue of those practices. A deployed agent can begin to re-enter them: running code, using tools, observing users, interacting with institutions.

The model begins with residue. The larger system can begin to recover contact.

But embodiment cannot be the whole answer. I know far too many things I have never touched, measured or personally witnessed. I have never measured the speed of light. I have never been to Antarctica. I have no direct embodied evidence for most of modern physics, most of history or whether penguins are currently wandering through Rome.

Direct contact does not scale.

So how do we know anything beyond it?

For that, we need Alberto.

Call Alberto

Suppose someone tells me that penguins live in Italy.

I have never conducted a census of Italian penguins. I cannot personally inspect every forest, coastline and piazza.

So I call Alberto. Alberto lives in Rome.

“Alberto, do penguins live in Italy?”

He laughs. I now know more than I did five minutes earlier.

Not with mathematical certainty. Alberto could be wrong. He may misunderstand the question. An escaped penguin could at this very moment be crossing Piazza Navona and destroying the example.

But Alberto occupies a useful position in the trust chain. He is there. He has repeated exposure to Rome. I have a history with him. If he repeatedly lies to me about things he is well positioned to observe, I update my trust in Alberto. If he says, “I don’t know about all of Italy, but I’ve never seen one in Rome,” the boundary of his knowledge is itself useful information.

This is how testimony becomes valuable. Not simply because another human said something, but because we care **who said it, what they were positioned to know, how reliable they have been, what incentives surround the claim and how easily it can be challenged.**

Testimony comes with metadata.

And we are all Alberto to someone. Someone may trust me on ranking systems because I have spent years working on them. Someone else may trust me about Jordan because I have lived there. If I begin confidently explaining marine biology, the correct response is not to transfer my credibility from machine learning to whales merely because the same mouth is speaking.

Trust is local.

We learn this early. Repeated interaction with caregivers builds expectations before we have words for evidence. Siblings contribute an important epistemological innovation: **some testimony is bullshit.** Teachers tell us about atoms, dinosaurs and wars we cannot personally verify. Science extends the chain through instruments, experiments, other investigators, criticism and replication.

Civilization is full of machinery for making mediated trust less stupid. Courts use testimony and adversarial procedure. Engineering uses standards, tests and certification. Science uses instruments, publication and replication. Markets use reputation and prices. None guarantees truth. All preserve some structure around claims: where they came from, how they were challenged, what incentives surrounded them and what might make us stop believing them.

Human knowledge is not simply a pile of facts. It is **epistemologically stratified.**

“I touched the fire” is not the same as “my brother told me.” “My teacher said so” differs from “the experiment was independently replicated.” A measurement differs from an interpretation. A conjecture differs from an established result.

Mature trust is not purely conservative either. Sometimes the instrument disagrees with the theory. At first you check the instrument. Then you repeat the experiment. If the anomaly survives long enough, eventually the trusted theory becomes the thing under investigation.

Productive distrust requires trust first.

Random distrust is just another form of stupidity. The interesting critic understands why the old structure earned trust before finding the point where that trust stops being deserved.

Models inherit the text produced by these structures, but usually not the live relationships underneath them. The paper, the article about the paper, the blog post disagreeing with the article and the Reddit thread where somebody confidently misunderstood both can all end up in the same training distribution.

Frequency is not verification. Statistical dominance is not epistemic authority.

In that sense, the model has no Alberto: no live record of who was positioned to know, where a claim came from, how its source behaved before, or where the source’s competence stops.

There is one more ingredient humans add almost without noticing: stakes.

If Alberto lies to me repeatedly, I stop trusting him. If a researcher fabricates data and gets caught, the cost can be enormous. If an engineer signs off on a bridge and the bridge fails, “but the analysis sounded plausible” is not a defense.

Stakes are not truth. People lie despite consequences and institutions reward confident nonsense all the time. But consequences shape testimony. If a friend asks where to eat, I may guess. If someone asks whether to undergo surgery, I become much more careful.

An LLM has no social capital of its own to lose. It can confidently produce something false and, at the level of the model itself, nothing happens. The cost lands elsewhere—on the user, the application or the institution deploying it.

At its most compressed, the danger is **coherence outrunning correspondence**. The machine can become extraordinarily good at tongue without having an ear available to check against. The dangerous failures are not the ones that crash. They are the ones that seem to work: decaf confidence, indistinguishable from the real thing until the moment it matters.

The missing ingredient is not punishment for models—it is architecture that restores more of the evidence, consequence and accountability that the sentence alone cannot carry.

That is the problem System 3 is trying to solve.

System 3

We are currently obsessed with making models think harder.

System 2 reasoning has become a product category. Give the model more inference time, let it plan, search, reconsider and work through difficult problems before answering.

This is useful. Reasoning matters.

But reasoning perfectly from a bad premise still produces a beautifully reasoned mistake. A research agent can spend six hours developing an elegant argument from a false paper. A coding agent can reason carefully about an API that never existed. Deep Mode can coordinate five sophisticated judgments that all trace back to one hallucinated claim.

At some point, thinking has to encounter something outside itself.

This is where I use the term **System 3**.

Kahneman’s *Thinking, Fast and Slow* gave us the familiar distinction between System 1, fast and intuitive cognition, and System 2, slower and more deliberate cognition.

For AI, the analogy is tempting. The base model looks something like System 1: fast pattern recognition, linguistic intuition, enormous associative capacity. Agentic reasoning adds something like System 2: decomposition, planning, reflection and extended search.

But human thought has always operated inside another structure that the two-system picture largely takes for granted. We test things. We build instruments. We execute code. We compare claims with

records. We ask other people. We preserve failures. We create procedures that make some errors harder to hide and some evidence easier to inspect.

I call that external epistemic machinery **System 3**.

System 1 proposes. System 2 deliberates. System 3 checks.

I keep another mnemonic because I am apparently incapable of leaving a three-part system alone:

System 1 is the Gut. System 2 is the Head. System 3 is the Hand.

The Gut recognizes. The Head reasons. The Hand reaches outside the current story and finds something capable of disagreeing.

The metaphor is imperfect. Peer review has no hand, provenance has no fingers and a formal proof does not need to touch a cow.

System 3 is the external scaffold that keeps thought answerable to observation, experiment, provenance, persistent failures, tools and other minds.

And this is where the naming matters. **Deep Mode is Layer 3: the problem-solving layer. System 3 is not another layer above it.**

Deep Mode asks: *Given what we know, what should we try next?*

System 3 asks: *What are we entitled to treat as known?*

It cuts across the stack. The model proposes something. The coding agent may test it. The application can collect real user behavior. Deep Mode may compare research, simulation and evaluation. Even Layer 4—the goal itself—can change when reality pushes back.

If the five layers tell us **where** increasingly abstract work happens, System 3 is what keeps those layers **epistemically connected**.

Code Can Touch Back

Code is unusually friendly to this idea because coding agents can touch their world.

When an agent writes code and runs it, reality answers back.

`TypeError: 'NoneType' object is not subscriptable` is not merely another paragraph describing Python. It is the execution environment saying: whatever story you just told yourself about this program, this particular part is wrong.

The agent can try something, observe the result, update and try again. The farmer approaches the cow and learns from the kick. The coding agent calls an API incorrectly and learns from the exception. The cow is probably more emotionally memorable, but structurally the loops rhyme.

This is one of the few places where a language model can, metaphorically, **touch the ear**.

The question is whether the system preserves what it learns there.

A normal agent session can fail ten times, discover the right approach, solve the problem and throw away most of the experiential history when the context ends. It is as if the farmer learned exactly where not to stand and then underwent elective amnesia every evening.

The MARC file incident shows the opposite move. In the Live-SWE-agent work, an agent encountering MARC files—the old bibliographic format used by libraries—created an analyzer to inspect data its existing tools could not conveniently expose.

The environment resisted. The agent’s current apparatus was not enough, so it created an instrument. That instrument changed what the agent could observe.

Humans have been doing this forever. We could not see bacteria, so we built microscopes. We could not perceive radio waves directly, so we built receivers. We could not conveniently inspect a MARC file, so apparently we wrote Python and called it epistemology.

The failure changed the instrumentation; the instrumentation changed what could be observed next. That is System 3 in miniature.

AlphaGo offers another useful distinction. Its neural network supplied powerful intuition about promising moves and valuable positions. Monte Carlo Tree Search placed that intuition inside an explicit search process constrained by the state and consequences of Go.

I used to summarize this too simply as “the network proposes; the tree verifies.” That gives the tree too much authority. MCTS does not magically prove the network right. It forces intuition to participate in an external, stateful process where moves have consequences defined by the game rather than by what the network can plausibly say about the game.

RL can improve the gut. System 3 preserves more of the structure around the gut: what was tried, what happened, which paths failed, where claims came from, which tools earned confidence and where their boundaries lie.

What Should Survive a Session?

Return to the research claim about recursion trees:

Students understand recursion better when shown a tree representation.

In a flat architecture, the sentence enters context and competes with every other sentence according to relevance and whatever confidence the model implicitly assigns it.

A trust-aware architecture wants more. Where did the claim come from? A controlled study? A teacher’s opinion? A blog post? An inference made by the research agent? Did several independent sources agree, or did five articles cite the same study? What population was tested? Does the result apply to our demo?

You do not need a bureaucratic dossier attached to every sentence. Sometimes “Alberto said the café is good” is enough.

But when the consequence matters, the claim should be able to carry provenance.

That is a **trust chain**. Not a guarantee of truth. A record of how far a claim sits from the evidence supporting it, what transformations happened along the way and which links we have chosen to trust.

This changes how we should think about skills, tools and memory.

A skill is knowledge externalized from the model. Someone—or some previous agent—learned something useful and wrote it down so later sessions would not need to rediscover it.

The model inherits the residue.

But persistence is not trust. A terrible heuristic written into a skill file is simply a hallucination with better retention.

A useful skill needs some archaeology. Who created it? What problem was it solving? Where did it work? Where did it fail? What conditions limit its use?

Suppose an agent learns:

Prefer structured parsers over regex for deeply nested formats.

A flat skill stores the rule. A richer object can record that the heuristic came from several failed regex attempts, later worked across multiple nested formats, remains unnecessary for simple flat extraction and should be treated as a strong prior rather than a commandment.

Tools can earn trust in the same way. If `edit_tool.py` succeeds on simple substitutions but repeatedly damages indentation-sensitive blocks, the useful knowledge is not merely *I have an editing tool* but *this tool is reliable here and dangerous there*. Reliability is conditional.

The same applies to softer heuristics. “Regex tends to fail on deeply nested structures” is not a theorem. It is a **meta-belief**—something that can accumulate evidence for and against it.

A normal rule says:

Never use regex here.

A System 3 belief says:

This has worked often enough that I should prefer it, but new evidence can change my mind.

Now the belief is challengeable.

If you enjoy old epistemology labels, you can call the model a largely **coherentist core**—uncannily good at producing structures that hang together—and System 3 a thin **foundationalist shell** tied to observation, provenance and consequence. Philosophers can put down their weapons; I only need the architectural analogy.

Coherence is valuable, but something outside the coherent system must occasionally be allowed to say no.

This is personal for me. I spent eight years building systems that rank human testimony—reviews, ratings and Q&A. The hardest problem was never only relevance. It was **trust stratification**. Which claims deserve corroboration? What happens when ten accounts repeat the same lie? When does consensus become evidence and when is it coordinated manipulation? How far should credibility transfer outside the domain in which it was earned?

These are not abstract questions when they determine what millions of people believe about a product.

System 3 isn’t philosophy to me. It’s Tuesday.

Creative Distrust

Trusted knowledge makes you efficient. It can also make you boring.

If an agent learns that structured parsers beat regex on nested syntax, good. It stops repeating a known mistake. If it learns that tree visualizations worked for five recursive algorithms, eventually it may try to explain linear regression with a tree because the trust stack has become stronger than judgment.

Every genuinely new idea begins with less evidence than the thing it challenges.

So System 3 needs **creative distrust** too.

This is not contrarianism for sport. It is not the internet habit of assuming expert agreement proves corruption. It is the ability to understand a trust chain well enough to know where you are breaking it and why.

A mathematician follows an analogy because the structure looks interesting. A scientist repeats a strange experiment after accepted theory says the result should not happen. A designer violates a trusted pattern because this case exposes its boundary conditions.

A mature trust stack has two jobs pulling in opposite directions: let knowledge accumulate so we do not rediscover fire every morning, and leave enough room for reality to overthrow what accumulated.

There is no final setting that makes trust and rebellion stop fighting.

Our experiment ran directly into that problem.

The Experiment

I wanted to test a smaller claim than “we solved epistemology for AI.”

Could even crude epistemic structure around a coding agent change how it behaves?

We built a small agent called **epistemic-swe**. It added three kinds of persistent state around a normal coding agent.

A **tool registry** tracked tools, successes, failures and known failure modes. **Meta-beliefs** allowed heuristics to accumulate evidence instead of entering the system as permanent commandments. **Failure memory** preserved enough information about failed approaches to make blindly repeating them less likely later.

The state persisted across sessions, so later problems could inherit things learned earlier. We also pruned it. An epistemic architecture that remembers everything eventually becomes a hoarder with a context window.

We compared mini-swe-agent with epistemic-swe on ten SWE-bench Verified problems from the As-tropy repository, using the same base model and tasks.

Ten problems is nowhere near enough to establish a solve-rate advantage. State persisted across tasks, so order effects may matter. I was not looking for a benchmark victory. I wanted to know whether the scaffold changed behavior strongly enough to become visible.

It did, just not in the direction I expected.

Metric	mini-swe-agent	epistemic-swe
Solve Rate	50% (5/10)	40% (4/10)
Avg Patch Size	620 lines	269 lines
Patch Reduction	baseline	57% smaller in this run

Read the first line before celebrating the third.

The epistemic agent solved fewer problems. I had expected learning from previous failures and tools to improve capability. Instead, the clearest difference was **focus**: its patches became much smaller.

A few examples:

Problem	mini	epistemic	Ratio
astropy-12907	301 lines	61 lines	4.9x smaller
astropy-13453	266 lines	17 lines	15.6x smaller
astropy-14096	529 lines	70 lines	7.6x smaller
astropy-13977	2720 lines	362 lines	7.5x smaller

The baseline often left behind debris from exploration: temporary scripts, broader edits, test scaffolding and abandoned experiments. The epistemic agent tended to make more surgical changes.

That does not prove the trust stack caused the reduction, and smaller patches are not automatically better patches. The extra instructions may simply have made the agent more conservative. Persistent state may have changed behavior for reasons unrelated to my epistemic interpretation. Ten tasks from one repository cannot separate these explanations.

Still, the behavior changed enough to be interesting.

The scaffold seemed to produce discipline before it produced capability.

That was not the hypothesis, which made the result more useful.

The 13579 Failure

One problem broke the pattern dramatically: **astropy-13579**.

Mini solved it. Epistemic did not. It was also the only case where the epistemic patch became substantially larger rather than smaller.

Both agents correctly identified the central bug: dropped world-coordinate dimensions were being filled with a hard-coded value rather than the actual coordinate value.

The baseline took a fairly direct approach:

```
# Store the actual values for dropped dimensions
self._dropped_world_values = [
    world_coords[iw] if iw not in self._world_keep else None
```

```

    for iw in range(self._wcs.world_n_dim)
]

# Use them instead of 1.0
world_arrays_new.append(self._dropped_world_values[iworld])

```

The epistemic agent chose a more structural intervention around which dimensions were being kept:

```

self._pixel_keep = np.nonzero([
    not isinstance(self._slices_array[ip], ...)
    for ip in range(self._wcs.pixel_n_dim)
])[0]

```

The second approach was not stupid. That is why the case matters.

The agent had accumulated context about indexing, dimensionality and coordinate-system failures. Its chosen explanation fit that context. It followed a path that looked principled and coherent.

It was wrong. The baseline took the simpler path and fixed the actual bug.

One possible story is that accumulated epistemic structure made one family of explanations too salient. But one case cannot establish that causal story. Persistent state may have caused the wrong turn or merely accompanied it.

What we can say is that structured memory changes the context in which future search occurs.

Trust is **path-dependent**.

Expertise works the same way. A great database engineer may see a database problem faster than most people, which is wonderful until the actual problem is the network. Paradigms focus attention. They can become prisons for exactly the same reason.

The failure is more interesting to me than a clean win would have been because it kills the simplest story:

Add memory, get smarter agent.

Structured experience biases future behavior toward what the system has learned. Sometimes that is exactly what we want. Sometimes the bias is the failure.

A mature System 3 therefore needs more than accumulation: forgetting, counterexamples, challenge, competing possibilities and occasional permission to ignore what it thinks it knows.

Otherwise the scaffold becomes a cage.

Back to the Camel

Return to the seven claims.

1. Krka National Park—True. I was there. For me this sits close to embodied memory. For you it is testimony unless you extend the chain through records or other evidence.

2. Best philosophical thinking at waterfalls—False. I mostly do philosophy on buses and in boring waiting rooms. Waterfalls are for ice cream. The subject and the source are unfortunately the same man.

3. Permanent camel resident—False. This can be checked against information about the park. You do not need my biography.

4. The tongue can touch its own ear—Unknown. I genuinely do not know. I did not check. Neither did you. We can reason from anatomy and build a prior, but the shortest decisive chain would have been to stay there and watch.

5. Ice cream ten minutes earlier—True. Chocolate. Mostly testimony again.

6. Camels are native to the Dalmatian coast—False. You probably rejected this immediately without reconstructing camel evolutionary history or personally surveying Dalmatian fauna. A large inherited structure did that work for you.

System 1 can be fast because System 3 has often been working for centuries underneath it.

7. Real, unedited photograph—True. The image alone cannot establish that. A stronger chain might include the original file, metadata, cryptographic signing, independent witnesses or another provenance system. Every extra link can increase confidence and gives us one more thing that may itself need to be trusted. And I could be lying to prove a point about trusting sources. If I told you the photograph was AI-generated, you would probably believe that too, because it fits a pattern you recognize.

Welcome to epistemology.

The lesson is not that nothing can be known. That conclusion is dramatic and mostly useless.

The lesson is that **trust has structure**.

Some claims sit close to direct interaction. Others arrive through testimony. Some pass through instruments and other people. Some are repeated many times but trace back to one observation. Some are plausible inferences. Some have little track record but may still deserve investigation.

Flatten all of that into equally confident language and something important disappears.

The model can remain what it is: an extraordinarily general machine for navigating learned patterns, capable of intuition and increasingly capable of reasoning. It does not need to contain the entire chain inside its weights. The model is a mirror; the architecture determines what it reflects.

The model stays hollow. The system doesn't have to be.

Daniel Dennett's phrase for the first half is competence without comprehension. Whether the second half amounts to comprehension is a question for people with more patience than I have; the part of it that can be checked is the part the rest of this chapter builds.

Everything so far can still be imagined around one agent: it acts, checks, remembers, records provenance and updates what it trusts.

Real systems will not stay that simple. The moment one agent inherits a claim from another, no participant can personally reconstruct every path back to reality. A trust chain can preserve where a

claim came from. It does not, by itself, tell us how the knowers who depend on those chains should be arranged.

The question is no longer simply:

How can an AI know what to trust?

It is:

How can a population of fallible knowers build knowledge together without losing contact with the world?

Humans have been working on that problem for a very long time.

Chapter 5: The Society of Agents

When the Org Chart Starts Thinking

Sixteen Claudes walk into a kernel.

[VISUAL — chapter opener. Sixteen identical small robots in a queue at a single door labelled `linux/`. The door is human-sized; the queue is not. One robot at the back is reading a file labelled `progress.md`. Line-art, no color, half page.]

A few years ago that sentence would have looked absurd. By the time Nicholas Carlini tried it, the strange part was no longer that agents could write compiler code. The strange part was watching sixteen capable agents slowly turn into an organization.

The goal was almost offensively ambitious: a C compiler in Rust, from scratch, with no dependency beyond the standard library and no internet access, pushed far enough to compile the Linux kernel. Carlini had been using the task as a benchmark across generations of Claude models. Earlier versions could barely produce a working compiler. The next passed large test suites but could not build major real projects. Then, over nearly two thousand Claude Code sessions and two weeks, sixteen agents produced roughly a hundred thousand lines of compiler code that built Linux 6.9 on x86, ARM and RISC-V. It also compiled QEMU, FFmpeg, PostgreSQL and Redis.¹

It was nowhere near GCC. Its output ran slower than GCC with optimizations switched off, and it still borrowed an assembler and linker. One stage of the x86 boot path called GCC because the agents' own sixteen-bit code came out at nearly twice the size the kernel allows. Still, this was well beyond the kind of toy problem where sixteen agents succeed because the work happens to split into sixteen independent pieces.

The harness grew around their failures.

The first was that a coding agent stops. Give it a long problem and it solves part, then waits for a human to say what's next. The fix was simple: a shell loop. When a session ends, start another with the same prompt, forever. That solved waiting, mostly. On one occasion an agent ran `kill -9 bash`, killed its own loop and left the project, which turned out to be one way a worker in this organization could retire. I have wanted to do the same from time to time. Luckily, I lack root access.

The loop still left one agent doing one thing at a time. So there were sixteen containers, each with its own clone of the repository, and a new problem: two agents could pick the same task. Before starting,

¹Nicholas Carlini, “[Building a C compiler with a team of parallel Claudes](#)”, Anthropic, 5 February 2026. The account distinguishes the compiler's achievements from its dependencies and limitations. The opening groups harness choices by the problems they address; it does not claim that all were introduced in the order narrated.

an agent wrote a small lock file naming what it intended to work on. Git synchronized the locks, and if two agents claimed the same problem the second had to pick another. When an agent finished it pulled, merged, pushed and released the lock.

There was no orchestrator. Carlini had specified parts of the skeleton in advance: how the compiler would represent code internally, using an SSA intermediate representation, and that it should support multiple backends. But no agent held the current state of the project in its head, because no agent had a head that lasted longer than a session. Each fresh container spent a large part of its budget working out where the project stood. So the prompt told every agent to maintain READMEs and progress files and to update them often, because the next worker would arrive knowing nothing.

Two other limitations shaped the work. Agents have a poor sense of elapsed time, so left alone they could spend hours running tests instead of fixing anything. The harness offered a fast mode that ran a one or ten percent sample. Each agent got a repeatable sample so it could compare results; different agents got different samples, spreading their attention across the suite. The other problem was that test output flooded context, and a context window full of logs is a worker who has forgotten what it was doing. So the output went to disk, with a few lines of summary left behind and every error on its own greppable line.

[DIAGRAM — the harness as a ladder. Five rungs, each a pair: left side the failure (“agent stops”, “two agents, one task”, “fresh container knows nothing”, “runs tests forever”, “logs flood context”), right side the fix (“loop”, “lock file”, “progress.md”, “sampled tests”, “logs to disk”). Two more rungs below show “one global bottleneck → GCC-assisted isolation” and “new work breaks old work → CI”. Present as a map of problems and responses, not a dated chronology.]

For a while this worked remarkably well, partly because compiler test suites are generous places to employ a crowd. A new compiler fails thousands of them, independently. One agent can investigate a parser bug while another works on code generation and a third discovers that a respectable-looking integer conversion has been quietly ruining everybody’s afternoon. Once the compiler could build real programs, SQLite, Redis and Lua each exposed a different neglected corner of C. Sixteen agents, and always more than sixteen things to do.

[VISUAL — Linux as the boss level. The sixteen small robots have been walking across a wide map of many small doors (test cases, then SQLite, Redis, Lua). The map narrows to a single enormous gate, Bowser-castle scale, with a penguin silhouette on it. All sixteen are bunched in front of it. Game-map style, one color accent.]

Then they reached Linux.

The kernel is not thousands of tests. It is one enormous test, and compilation stopped at the first serious compiler bug. Agents arrived at the same failure, formed their own theories and pushed changes over one another’s. The locks could separate named tasks; they could not turn this bottleneck into sixteen different investigations. The standing instruction to find something useful and attack it kept pointing everyone at the same place.

The models had not changed. The problem had, and the organization that had worked so well on a test suite was now getting in the way. The organization was the bug.

[VISUAL — the GCC oracle. The kernel drawn as a wall of bricks. Most bricks are grey

and stamped GCC; a scattered handful are colored and stamped with the new compiler's mark. A robot tests whether the wall stands. Three panels show the colored set shrinking as the failure is isolated.]

Carlini changed the harness. Most of the kernel was compiled with GCC and only a random subset of files with the new compiler. A successful boot cleared that combination for that run. A failure gave the agents a smaller set to investigate, replacing more files with GCC output to narrow the search. Sixteen agents could again work on different files. A later pass with delta debugging helped isolate combinations that failed together even though their components worked separately.

The harness kept changing. Near the end, new features started breaking old ones, so Carlini added a continuous integration pipeline with stricter checks on new commits.

Specialization addressed work the next failing test would never ask anyone to do. LLM-written code kept re-implementing what already existed. Someone needed to look for duplicates. Someone needed to improve the compiler's own speed, and someone else the quality of the code it emitted. Carlini assigned those roles, along with an agent to review the structure as a Rust developer and another to work on documentation, which is normally the moment you know a civilization has become serious. The crowd had become a staff.

Look at the harness and every part carries the mark of a problem. Two workers reach for the same task, so there is a lock. A worker arrives with no memory, so there is a progress file. New features break old ones, so there is CI. Linux stops the whole crowd, so the harness learns to split it into smaller questions. None of the agents arriving on the last day needed to have lived through that history. The history was in the structure.

Same models. Different institution.

That interests me more than the generic claim that multi-agent systems scale. I have watched the same thing happen to teams of people, where it goes by the less glamorous name of reorganization, and nobody writes a blog post.

[VISUAL — Popper peering over his glasses at three small planets floating below him: one rocky and physical, one a cloud of thought-bubbles, one built of books, tablets, instruments and a Git commit graph. Portrait-caricature style.]

So where does the knowledge of the compiler project live?

Obviously some of it lives in Claude. But which Claude?

The parser agent does not know what the performance agent knows. Neither remembers the previous session, and neither will exist next week. Some knowledge lives in code, some in tests, Git history, progress files, task boundaries and conventions. Some lives in GCC, which the project trusts as a reference for particular questions. Some lives in Carlini's head; he is the one who notices that the organization no longer fits the work and changes the harness.

In 1967 Karl Popper gave a lecture called *Epistemology Without a Knowing Subject*, and meant it literally. Beside the physical world and the world of individual minds he proposed a third: the world of theories, problems, arguments, and the records that carry them. Knowledge there can be examined,

criticized and improved by people who did not produce it and are not currently thinking about it. He called it World 3. The compiler project is a small World 3 with a Git remote.²

Popper insisted on two properties of that world. We make it, but we do not control everything that follows from it: once a theory is written down it has consequences its author never saw, and problems nobody has noticed yet are already sitting in it. And it keeps our mistakes as carefully as our discoveries. A refuted theory is still a citizen of World 3; it just has a bad reputation.

His own example was a book of logarithm tables, computed by a machine, printed, and never opened by anyone. Does it contain knowledge? Popper said yes. What matters is that someone could read it, not that anyone has. Now look at the compiler's progress files, written by Claudes that no longer exist for Claudes that do not exist yet. In between, they sit on disk, right or wrong, with nobody looking.

The project also has a way to challenge some of what those files say without waiting for a person to read them. An agent claims a bug is fixed; the build fails. Reality has a commit hook. The hook can be wrong too, but the agent's account of its own success no longer gets the last word.

The project can also become wrong in ways no participant intended. A progress file carries a bad diagnosis into later sessions. A specialist improves its own metric while degrading the compiler. A lock prevents duplicated effort and also prevents a useful second attempt. Two agents confirm each other while both inherited the same mistake from the same file. Carlini's closing worry was that passing tests can make unfinished work look complete.

The question underneath all of this is how a population of fallible knowers can build knowledge together without losing contact with the world. The compiler is the small version, and it came with a luxury: one person who could step back from the agents' work, see that their institution no longer fit it, and rebuild the harness.

Human civilization has been living inside the large version for thousands of years, with no one standing outside it.

[VISUAL — closing image for the section. Left panel: sixteen robots inside a glass box; a human hand outside adjusts a dial on the box. Right panel: a crowd of thousands of tiny humans inside a much larger box; the box's edges fade into the page. Same drawing style as the opener.]

Civilization Had No Senku

In 2019, a green light swept across the Earth and turned humanity to stone.³

People were caught in classrooms, at kitchen tables, crossing streets. Aircraft continued without pilots. Fires burned through neighborhoods where nobody could raise an alarm. The machinery kept running until it needed something from us.

²Karl Popper, "Epistemology Without a Knowing Subject," lecture delivered in 1967, collected in *Objective Knowledge: An Evolutionary Approach* (1972). See the discussion of World 3, the machine-produced logarithm tables, and the two library thought experiments.

³The 2019 year is a chronology calculation, not an inference from the anime's release date. In the awakening sequence, Senku gives an elapsed count of 117,354,893,870 seconds and dates his awakening to 1 April 5738. Subtracting that interval places the petrification in June 2019. See *Dr. Stone*, chapter 13, "Stone World the Beginning," adapted in season 1, episode 5; the sequence and count are also transcribed in [this chapter-by-chapter reading](#). The year agrees with the [series plot chronology](#). The opening retains the year without asserting an exact day or time.

Over the centuries, rain entered the roads and roots widened the cracks. Bridges fell. Forests grew across the places they had connected. Buildings began the long process of becoming landscape.

In the spring of 5738, the stone around a boy's body broke apart.

He emerged into a forest with the knowledge of a civilization that could no longer supply him with breakfast. He needed shelter, food and fire. Much of what he knew would remain useless until he could make the equipment required to use it.

Later, in a village of people born into this world, he met a young woman named Ruri who was seriously ill. He proposed making a medicine for her.

There was no pharmacy to raid. The ingredients had to be obtained or produced, and the work required equipment the village did not possess. Before the medicine could exist, they needed glass.

Knowing how to produce it was only the beginning. Someone had to shape it into useful vessels. An elderly craftsman named Kaseki joined the effort, bringing the skill of a lifetime spent making things. The boy could explain what they needed; Kaseki could make it.

The villagers gathered materials and learned unfamiliar processes while Ruri remained ill. Eventually they completed a sulfa drug and gave it to her. She recovered.

The boy's name is Senku, and we have been following *Dr. Stone*.⁴

Popper had imagined a related catastrophe in the same 1967 lecture. Destroy our machines and tools, along with our knowledge of how to use them, but preserve the libraries and our capacity to learn from them. After much suffering, civilization could recover. Destroy the libraries too, he argued, and its return would take many millennia. Popper left the libraries standing. *Dr. Stone* puts one inside a teenager.

Senku remembers a world in which the medicine already exists. He can work backward from it, identifying materials and processes worth pursuing. His companions still have to make those steps work with what they have, but they do not have to discover the entire possibility of modern medicine along the way.

Real civilization had no Senku, and nobody standing outside it with the roadmap.

A potter learned from clay, fire and vessels that cracked. She tried a change, repeated it when it worked, then taught it to an apprentice. When his pot cracked, she recognized something he had done that morning. He remembered doing it, remembered her warning, and now had an expensive reason to pay attention. It took several more firings before his hands could feel the difference she noticed immediately. When he became a teacher, he passed on what he had learned, including precautions he did not entirely understand, because the last person who omitted them had ruined a firing.

Knowledge had a face. You knew whom to ask and remembered what happened when you listened. Senku had the chemistry; Kaseki had the hands. Whatever they failed to pass on could die with them.

Her vessels, meanwhile, ended up in storehouses that needed a memory beyond particular people. Grain arrived from different fields, portions were distributed, obligations remained. The person who witnessed a delivery might be absent when someone disputed the amount.

⁴Riichiro Inagaki and Boichi, *Dr. Stone* (2017–2022), the Kingdom of Science's effort to produce a sulfa drug for Ruri. The scene is a compressed retelling of that story arc.

By the late fourth millennium BCE, people in southern Mesopotamia were recording economic information in clay, including accounts concerning grain. The material that made the vessel could also keep an account of its contents. A new clerk could consult a transaction he had never witnessed, and the person who delivered the grain could discuss it with him over a record they could both inspect.⁵

The mark did not need to be wiser than the clerk. It needed to outlive him.

The institution could now consult its own past. That brought new responsibilities: learning the marks, agreeing on their meanings and settling disputes when two records told different stories.

The clerk could also enter the wrong amount, and his successor could copy it faithfully. The clay had preserved what it was given.

Remembering is not knowing.

Strangers Need Standards

The clay could preserve the wrong figure. It could also preserve the right one in a unit nobody else used.

A record says somebody owes ten sacks of grain. What exactly is a sack?

In 221 BCE the king of Qin conquered the last of the six rival states and became the First Emperor. His new empire contained regional writing traditions, currencies, weights and measures. Conquest had put strangers under one ruler. It had not made their accounts agree.

Under the emperor and his chancellor Li Si, the state imposed common standards. Some surviving bronze measures carry the imperial edict that ordered the work. The vessel tells you how much it holds and, on its side, who decided. You can hold one and read the decision.⁶

Then the ambition to settle disagreement reached the past. In the account preserved by the historian Sima Qian, Li Si proposed in 213 BCE that certain private histories and philosophical texts be burned. Copies held by court scholars were exempt, along with useful technical works. A person could consult the past, provided the court controlled which past was available. The proposal was meant to stop old books from becoming arguments against the present ruler.⁷

The bronze measure allowed two clerks to discover that their accounts disagreed. The order against the books tried to take that possibility away from the emperor's critics.

A standard removes a decision from the future. We have decided, for now, not to reopen this question every time. Once exchange extends beyond people who know one another, the stranger reading the tablet needs to know what a sack is, what a seal proves, whose account wins when two disagree. Standards make that exchange possible. They can also make a decision difficult to challenge long after

⁵Hans J. Nissen, Peter Damerow and Robert K. Englund, *Archaic Bookkeeping: Early Writing and Techniques of Economic Administration in the Ancient Near East* (1993). The potter and apprentice are illustrative characters, not a reconstructed historical incident.

⁶National Palace Museum, collection entry, "Oval Liang Measure by imperial decree of 26th year," Qin dynasty, 221–207 BCE. The inscription provides a material example of standardization backed by imperial authority.

⁷Sima Qian, *Records of the Grand Historian*, "Basic Annals of the First Emperor of Qin." The narrative of the 213 BCE order is a later historical account. The extent of its implementation and its contribution to the subsequent loss of texts are disputed; the chapter does not attribute every later loss to it.

the people who made it have gone. The Qin measure was not the correct measure. It was the one backed by the state.

Seen that way, bureaucracy deserves a better reputation than it gets. A workflow is accumulated experience with some choices removed. Someone already had the argument, or discovered the failure, or decided that one action requires another pair of eyes, and the next person inherits the result as procedure. Amazon calls this a *mechanism*, and the useful sense of the word is not corporate: a mechanism is an attempt to make a desirable behavior survive the person who first cared about it.

When something goes wrong you can tell everyone to be more careful, which is emotionally satisfying and institutionally almost worthless, or you can change the system so the dangerous action is slightly harder and the correct one slightly easier. The organization has learned when its future behavior changes.

That is how an institution learns, and it is also how it scars. A review gets added after a spectacular failure. Five years later the system is different, nobody remembers the incident, and ten thousand ordinary changes still pass through the review because the procedure survived its reason. The apprentice kept a precaution he never understood. The institution can do the same, with a much larger kiln.

A tablet, a bronze measure, a deployment guardrail: knowledge becomes structure. It allows work to pass between strangers. Now the strangers can each learn something the others do not.

The Society Gets Smarter by Making People Narrower

Then Amazon made me an offer, and the offer was reviews.

Before that I had run the technology of a smaller e-commerce company, fewer than a hundred people for all of it: catalogue, search, payments, the warehouse software, the emails that went out at night. I knew every system because I had to. Amazon wanted me for the text under the fold. The fold is a newspaper word: the important story goes above it, then you fold the paper under your arm and forget the rest. On a product page, reviews were the rest. The organization whose applied science I would lead was more than a hundred people, which was more than the entire company I had just left, and I remember sitting with the offer and feeling faintly embarrassed for everyone involved. What could a hundred people possibly do with a text box?

Here is what a text box contains. A grill weighs thirty kilograms. Is that good for a barbecue party? The catalogue cannot tell you; it can tell you thirty kilograms. Two thousand reviews can tell you, and they can tell you who bought it, why, what went wrong at the first party, what they wish they had known, and the small psychology of a person who wanted to feed twelve friends and got it slightly wrong. Once you look at nothing else, the text box turns out to hold the buyer's whole story.

And the story wants work. How do you ask for a review without begging? How do you help a person who has never written anything write one? How do you summarize two thousand of them into a paragraph, find the useful ones, catch the fakes, carry them across nine languages, use them to explain a catalogue written by a manufacturer, feed them into search so that "good for parties" finds the grill, find the one review in ten thousand that a marketer could build a campaign on? A hundred people was not enough to cover the space. At the smaller company, reviews had been one engineer's afternoon.

The potter became better because she was not also the physician. Specialization gives people time to encounter differences a generalist may never notice. The society gains knowledge by distributing ignorance, and every organization, industry and science that gets big makes some version of that bargain: people go narrow, and the narrow place turns out to be bottomless.

Try finding one person who knows how to make the phone in your pocket, from raw materials to a working device. The glass, the chip design, the fabrication equipment, the lenses inside that equipment, the assembly and the software belong to different bodies of expertise. The company whose name is on the back must coordinate work it could not reproduce inside one person's head.

Capability rises because dependence rises. Civilization is a trust chain with plumbing.

On a March morning in 2005, in an English hospital, Elaine Bromiley, a healthy woman of thirty-seven, was put to sleep for a routine nasal operation. The anaesthetist could not get a breathing tube into her airway. He could not ventilate her adequately either. He called for help, and help came: a second anaesthetist, then the surgeon, all consultants, with decades of experience between them.

Within four minutes she was visibly blue. Her oxygen saturation had fallen to forty percent. The doctors kept trying to get the tube in.

There was a procedure for this emergency: when neither intubation nor ventilation works, make a surgical airway. A nurse fetched the equipment. It was in the room. Two nurses later said they knew what needed to be done but did not know how to broach the subject. The three consultants, meanwhile, remained fixed on intubation. They had lost track of the time and of how long Elaine had been without enough oxygen.

The people with the most authority kept attempting the same solution. The people who saw the need to change course could not make that knowledge change what happened to her.

Elaine never regained consciousness. She died thirteen days later.⁸

Her husband, Martin, was an airline pilot. The surgeon wrote to him that he still could not see how they could have anticipated or avoided what happened. Martin wanted an investigation. In aviation, you investigated a disaster so that the next crew did not have to learn it again.

The independent review found a pattern his industry knew painfully well. Skilled people had become absorbed in an attempted solution while the situation around them changed. Nobody managed to interrupt it. Even who was supposed to be in charge was disputed. Aviation had spent years developing ways for crews to recognize this pattern, challenge one another and change course. Technical competence alone had not been enough there either.

A hospital needs people who see different things. The nurse at the bedside, the radiologist reading an image and the surgeon do not become interchangeable because they share a patient. Their authority has to follow what they know, and their observations need a way to interrupt someone else's plan. The expertise in Elaine's theatre was real. So was the failure to use it.

The potter's mistakes cracked in her own kiln. A specialist's mistakes travel, and each person who receives one may have good reason to trust it.

⁸Clinical Human Factors Group, [written evidence to the House of Commons Health Committee](#), September 2008, section 4. The account identifies failures of situational awareness, communication, decision-making and leadership in Elaine Bromiley's care.

Who knows what matters. Who sees what matters too.

A Swarm Should Not Automatically Become a Meeting

Robert Millikan watched tiny drops of oil fall between charged plates. He timed their motion, changed the electric field and watched them again. From the balance between gravity, electrical force and the resistance of the air, he could work out how much charge a drop carried. The charge came in steps. Measure those steps carefully enough and you could find the charge of a single electron.

The number he published in 1913 was slightly too small, because the viscosity of air he used was slightly wrong. It was a very good number for its time. It became the number.

Feynman later described what happened to measurements that followed it: they crept upward toward the accepted value. Why so slowly? In his telling, a laboratory that got an answer close to Millikan's could write it up. An answer farther away sent the experimenter back to the apparatus, looking for what had gone wrong. The inherited number had become part of the method for deciding which results deserved to survive.⁹

Meanwhile, X-rays offered a way to investigate the spacing of atoms in crystals, and from that spacing physicists could estimate the electron's charge by another route. Those measurements helped expose the discrepancy. Their apparatus had problems of its own, but the viscosity of air was not one of them. No amount of respect for Millikan could make a crystal inherit that particular mistake.

That is what a second witness is for. She has to be capable of being wrong differently.

The easiest reaction to one unreliable agent is to create five. Give one the title *Researcher*, another *Critic*, another *Verifier*, put them in a conversation, and perhaps reality will be intimidated by the org chart. Humanity invented committees this way and then, dissatisfied with the original implementation, recreated them in software.

Condorcet supplied a famous mathematical case for crowds in 1785. In the simple version, voters each have the same better-than-even chance of being right, and their votes are independent. Under those conditions, the probability of a correct majority approaches one as the crowd grows. Independence is doing work in that sentence. Voters who simply copy one source copy its mistakes. The crowd is the source, louder.¹⁰

Five agents citing the same paper are not five witnesses. Five researchers repeating a claim that traces to one unsupported source are not corroboration. If everyone receives the same framing, reads the same leading explanation and inherits the same assumptions, agreement may tell us more about their starting point than about the claim.

Agreement raises confidence when it would be difficult to explain if the claim were false. When five agents merely repeat one source, we have one witness wearing different coats.

⁹Robert A. Millikan, "On the Elementary Electrical Charge and the Avogadro Constant," *Physical Review* 2 (1913), 109–143; Richard P. Feynman, "[Cargo Cult Science](#)" (1974); Raymond T. Birge, "Probable Values of the General Physical Constants," *Reviews of Modern Physics* 1 (1929), 1–73. Feynman's explanation of the slow correction is presented as his interpretation. The corrections involved several experimental routes, rather than a single decisive encounter. Allegations concerning Millikan's selection of observations remain disputed; see also David Goodstein, *On Fact and Fraud* (2010).

¹⁰Marquis de Condorcet, *Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix* (1785). The paragraph describes the elementary independent-voter case, not a claim that every useful ensemble requires complete independence.

So useful independence has to be built. A critic should see the artifact before the builder's explanation. A second researcher should form a theory before reading the favorite. Different investigators should sometimes use different sources or methods. Separate containers help with some kinds of interference. They do not erase shared training, shared prompts or the bad diagnosis everyone read in `progress.md`.

A strange branch may deserve another experiment even when nobody expects it to win. Kevin Zollman's models show why: under some conditions, less connected communities reach the truth more reliably because an early misleading result cannot bring everyone onto the same path before alternatives have been investigated.¹¹

Permanent disagreement would be useless. An institution that never converges is a philosophy department with an alarming compute bill. Independence exists so that disagreement can carry information.

For that we need more than another opinion.

A Man in a Dark Room

Around the beginning of the eleventh century, Ibn al-Haytham darkened a room, made a small hole in one wall, and placed lamps outside it. On the opposite wall, spots of light appeared. Cover a lamp and its corresponding spot disappeared while the others remained.¹²

Vision had been argued about for centuries. One tradition held that the eye sends something out toward the world; another that something travels into the eye. Ibn al-Haytham developed an account in which light travels from objects toward the eye. The dark room did not settle the whole dispute. It made part of the problem manageable: light from separate sources passed through the same opening along paths that could be traced, interrupted and examined.

The setup allowed someone who disagreed with him to do more than disagree.

A record preserves what somebody says happened. An experiment gives the world another chance to answer.

We do not ask nature which theory it prefers. We arrange a situation in which different descriptions imply different things should occur, then watch what happens. Charles Sanders Peirce later argued that this is what separates science from other ways of settling belief, whether by stubbornness, authority or what seems reasonable from an armchair: the answer is constrained by something beyond the believer. I am a pragmatist in his sense throughout this book, and *contact with reality*, wherever the phrase appears here, means that arrangement. Something outside the current explanation is able to make the explanation fail.¹³

An experimental arrangement still has to travel. Someone elsewhere needs enough of the description, the equipment and the skill to make the world answer again. And when the instrument shows something nobody has seen before, the observer has two things to explain: the discovery and why anyone

¹¹Kevin J. S. Zollman, "The Communication Structure of Epistemic Communities," *Philosophy of Science* 74 (2007), 574–587, and "The Epistemic Benefit of Transient Diversity," *Erkenntnis* 72 (2010), 17–35. The benefits depend on the learning situation and communication structure modeled.

¹²Ibn al-Haytham, *Kitāb al-Manāẓir (Book of Optics)*, early eleventh century; A. I. Sabra, *The Optics of Ibn al-Haytham, Books I–III: On Direct Vision* (1989). The dark-room experiment illustrates controlled investigation of light propagation; it is not presented as a single decisive refutation of all emission theories of vision.

¹³Charles Sanders Peirce, "The Fixation of Belief," *Popular Science Monthly* 12 (1877), 1–15.

should trust the device that produced it.

In March 1610 Galileo published a short book announcing, among other discoveries, four moons orbiting Jupiter. The next month he took his telescope to Bologna. Through the instrument, an earthly object could look wonderfully close. Point it at Jupiter and some observers could not find the moons Galileo said were there. Martin Horky, an assistant to the astronomer Giovanni Magini, reported that the device worked on earth and deceived in the heavens.¹⁴

He was wrong about the moons. But the question was reasonable. Unpack the sentence *there are moons orbiting Jupiter* and it contains a telescope, the craft of grinding lenses, assumptions about optics, an interpretation of points of light, and the possibility that somebody else might build an instrument and look. A lens that made a distant church appear closer had not, by that fact alone, established the reliability of everything it showed in the sky.

Later that year Kepler looked through another telescope and saw the moons himself. A flaw in Galileo's lens now had a harder time explaining the result. The two observers still shared assumptions about light and astronomy, but the claim no longer depended on what one man said he had seen through one instrument.

A new instrument creates new facts and new ways to be wrong about facts. Was the lens distorting? Was the point of light there at all? Could another observer reproduce it? Did the operator know what she was doing? An instrument is a witness, and a witness needs a track record.

The agent version is almost embarrassingly literal. Run the program. Execute the query. Open the browser. Measure the latency. Compile the kernel against GCC. Reasoning has left the conversation, and something outside the current explanation now has a chance to be inconvenient. But every one of those actions brings the telescope's questions with it. How reliable is the tool? On which problems? What does it measure, when does it fail, and who calibrated it?

A broken tool is not external grounding. It is a very efficient route to externally generated nonsense.

When Curiosity Became Procedure

In 1659 Robert Boyle had Robert Hooke build him a pump that could pull much of the air out of a glass vessel. Boyle put things inside it and watched. A candle went out. The sound of a bell weakened. Animals struggled to breathe. He published the results in 1660 with descriptions of the apparatus and numbered experiments, so that a reader in another city could, in principle, build the pump and see for himself.¹⁵

Thomas Hobbes was unimpressed. A room full of gentlemen could agree about what they had seen and still be wrong. Their conclusion depended on the glass, the pump, the leaks and their eyes. Geometry was supposed to compel assent. Why should anyone outside Boyle's room have to accept the verdict of the people inside it?

¹⁴Galileo Galilei, *Sidereus Nuncius* (1610); Johannes Kepler, *Narratio de observatis a se quatuor Iovis satellitibus erronibus* (1611), reporting observations made in 1610. For the reception of Galileo's telescope and the Bologna episode, see Albert Van Helden, introduction and commentary to *Sidereus Nuncius, or The Sidereal Messenger* (1989).

¹⁵Robert Boyle, *New Experiments Physico-Mechanicall, Touching the Spring of the Air, and Its Effects* (1660); Steven Shapin and Simon Schaffer, *Leviathan and the Air-Pump: Hobbes, Boyle, and the Experimental Life* (1985).

The pump gave the objection plenty to work with. In 1661 Christiaan Huygens found that water remained suspended in an evacuated vessel when it was expected to fall. Boyle's group could not reproduce it. For two years they had different facts about what water did when the air was removed. Was the difference in the air, the water, the glass, the pump, the operator or the explanation? In 1663 Huygens came to London, and with his participation the effect was reproduced there. The written account had crossed the sea before him. It had not brought everything the experiment needed.¹⁶

Reality had pushed back against the package. It had not highlighted the guilty component.

Pierre Duhem articulated the problem in 1906, and Quine later developed a broader version: a test depends on more than the hypothesis under investigation. It depends on instruments, background assumptions and the way the test is carried out. When a prediction fails, logic alone does not identify which part to abandon. Software engineers know the sensation without the names. A failing integration test proves the system is broken somewhere. Wonderful. You now have debugging.¹⁷

So the institution needs archaeology. Which instrument produced the measurement? Which analysis transformed it? What was actually observed and which interpretation was added afterward? Within a few years of Boyle's book, *Philosophical Transactions* was publishing reports that could outlive the room. Publication established a claim and a date, and gave someone elsewhere a route, however imperfect, to expose the claim to the world again. Reputation gathered around investigators and, more strangely, around instruments and procedures. The question *did this happen?* acquired machinery.

The machinery was never clean. Access was unequal. Standing affected which claims traveled. Replication could be possible in principle and unaffordable in practice. Medicine later made one form of self-restraint explicit: in a randomized trial, the allocation procedure stops the investigator's own preference from deciding who gets which treatment. Sometimes bureaucracy is epistemology with a clipboard.

In an agent system this is provenance around a claim, an assumption graph, a trace. Without the history, reality can tell us we are wrong while leaving us remarkably creative about which part of the system deserves blame.

Knowledge comes with a route through which someone else might make the claim fail again.

A trust chain has acquired an escape hatch.

The Org Chart Becomes Part of the Experiment

In 1912 Alfred Wegener, a German meteorologist, proposed that the continents had once been one landmass and had drifted apart. He had the fit of the coastlines, matching fossils on opposite sides of the Atlantic, matching rock formations, glacial traces in places that were now tropical. What he did not have was a force capable of moving a continent.¹⁸

¹⁶Shapin and Schaffer, *Leviathan and the Air-Pump*, discussion of Huygens and anomalous suspension. The reproduction of the effect established a shared phenomenon; it did not by itself settle every theoretical question about it.

¹⁷Pierre Duhem, *La théorie physique: son objet et sa structure* (1906); W. V. O. Quine, "Two Dogmas of Empiricism," *The Philosophical Review* 60 (1951), 20–43.

¹⁸Alfred Wegener, *The Origin of Continents and Oceans* (first German edition, 1915); Frederick J. Vine and Drummond H. Matthews, "Magnetic Anomalies over Oceanic Ridges," *Nature* 199 (1963), 947–949. Plate tectonics vindicated continental mobility while replacing important parts of Wegener's proposed mechanism.

Much of geology declined. Not stupidly: the forces Wegener proposed were too weak, and a continent could not simply plough through an ocean floor that stayed where it was. The fossils needed explaining, but so did the physics. Wegener died on the Greenland ice in 1930, still unable to make the continents move to his critics' satisfaction.

Decades later, ships towing magnetometers began revealing a peculiar pattern on the ocean floor. The rock was magnetically striped. Bands on one side of a mid-ocean ridge had counterparts on the other, recording reversals of the Earth's magnetic field as new crust formed and moved outward. The ocean floor itself was spreading. The continents were passengers.

By the late 1960s, plate tectonics was bringing the scattered observations together. It rescued Wegener's moving continents by changing the machinery underneath them. Nobody had to accept the forces he proposed in order to accept that he had seen something real.

But the stripes had been there while he was alive. Turning them into evidence took ships, instruments, people trained to read the measurements, and someone willing to pay for the voyage. A question cannot attract evidence from an instrument nobody builds.

Return to the agents. Suppose one proposes a hypothesis, another designs an experiment and a third evaluates the result. Now suppose all three inherited the same hidden assumption. The experiment fails. Which component changes? Reality does not care which file contains the variable named **hypothesis**; it pushes back against the arrangement as a whole. Who sees which evidence, which roles may modify the evaluator, which branches share context, which result is allowed to become everybody else's premise: these decisions affect what the system can find out.

Modern agent systems can make some of these choices dynamically. One problem may need several independent investigations, another a specialist and a verifier, another parallel workers around separable components. The bureaucracy can be temporary. The org chart can change with the problem. Organization itself has entered the search space, and whoever shapes it shapes what it can discover.

Imagine research program A is ahead and has twelve agents. Program B looks weaker and has one. Where does the next available agent go?

The natural answer is A, and the answer reinforces itself: more agents, more experiments, more evidence, more confidence, more agents. Eventually the leading theory owns the building, and the weak one has a meteorologist.

Philip Kitcher examined this problem in 1990: how should a community divide its labor among research programs? A community can benefit from some investigators pursuing a less promising approach even when each individual, considering only the best-supported option, would choose the leader. Lakatos supplied another reason for patience: an unresolved anomaly does not, by itself, tell us to abandon a research program. The question is what further work it makes possible.¹⁹

Compute allocation is epistemic policy. So is memory, so is context sharing, so is credit. Who receives the capacity to generate evidence helps decide which possible truths the institution can afford to discover. The current best explanation and the best use of the next unit of capacity are not the same question. A critic whose objections never change what gets investigated is doing quality-assurance theatre.

¹⁹Philip Kitcher, "The Division of Cognitive Labor," *The Journal of Philosophy* 87 (1990), 5–22; Imre Lakatos, "Falsification and the Methodology of Scientific Research Programmes," in *Criticism and the Growth of Knowledge* (1970).

Human science has never escaped this problem. It has simply had much longer to argue about it.

Science Gets Bigger Than the Scientist

Accumulated knowledge eventually destroys the world of the universal expert.

Newton was extraordinary. In January 1697 he was working at the Royal Mint, in the middle of the great recoinage, when Johann Bernoulli's challenge reached him: find the curve along which a bead slides fastest between two points. Bernoulli had given the mathematicians of Europe months to answer. In the account preserved by John Conduitt, Newton came home from the Tower at four in the afternoon, exhausted, and would not sleep until he had solved it. He finished by four in the morning. The answer went to the Royal Society without his name. Bernoulli recognized the author anyway. *Tanquam ex ungue leonem*: as the lion by its claw.²⁰

The story is usually told about Newton. The more important part is that Bernoulli's question could reach him at all. A challenge posed in Switzerland was printed in a Leipzig journal, read in London, answered overnight and returned to circulation. The attribution rested on a shared mathematical language precise enough that a style could be recognized like handwriting. Even the anonymous answer arrived carrying something of its author.

Genius mattered enormously. So did the network that let genius begin from accumulated work rather than from dirt.

Then success made the network more necessary. Laboratories specialized. Techniques required training. Journals multiplied, instruments grew complicated, and fields developed languages that excellent researchers next door could not read without help. Science became more powerful by making scientists less interchangeable.

Because no researcher can personally reproduce every result she depends on, trust became more important at exactly the moment standards of evidence became stronger. A physicist relies on chemistry. A doctor relies on assays. A scientist cites work she could not reproduce from raw materials with the rest of her career and a very generous grant. John Hardwig called this epistemic dependence, and argued that it is a condition of rational knowledge: a person who refused to believe anything she had not verified herself would know almost nothing.²¹

Rigor at scale is not the elimination of trust. It is the organization of trust.

On 4 July 2012 the ATLAS and CMS collaborations at CERN announced a new particle consistent with the Higgs boson.²²

Who discovered it? Try pointing to the person. The papers had thousands of authors. The detectors contained technologies developed over decades by specialists in many countries. No physicist woke up

²⁰Johann Bernoulli's challenge in *Acta Eruditorum* (1696), and Newton's anonymous solution in *Philosophical Transactions* (1697). John Conduitt's "Miscellanea," *Keynes Ms. 130.05*, records the four-in-the-afternoon to four-in-the-morning account, attributing it to Catherine Conduitt. For the wider episode and the attribution anecdote, see Richard S. Westfall, *Never at Rest: A Biography of Isaac Newton* (1980).

²¹John Hardwig, "Epistemic Dependence," *The Journal of Philosophy* 82 (1985), 335–349.

²²ATLAS Collaboration, "Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC", *Physics Letters B* 716 (2012), 1–29; CMS Collaboration, "Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC", *Physics Letters B* 716 (2012), 30–61.

that morning capable of rebuilding the Large Hadron Collider, recalibrating every detector, verifying every line of analysis software and re-deriving the theory before breakfast.

And yet the result was not rumor. It was carried by a structure: calibrations with histories, software that had been validated, analyses reviewed internally, detector systems that constrained one another. ATLAS and CMS supplied different instruments and separate analyses. They shared the accelerator and much background physics, but many mistakes in one detector or analysis would not automatically appear in the other. Their agreement mattered because they were partly independent in ways relevant to the claim.

The year before, another collaboration had shown how much could hide inside a measurement. In September 2011 OPERA reported neutrinos arriving from CERN at a detector in Italy about sixty billionths of a second earlier than light would have. The statistical significance was striking. The collaboration presented the anomaly and asked for further scrutiny.

In February 2012 it reported two possible problems in the timing equipment. A faulty optical-fibre connection could make the neutrinos appear early; an oscillator error pulled the measured time in the other direction. Further investigation and measurements removed the faster-than-light result. The timing equipment had turned an ordinary beam into an apparent challenge to relativity. One of the culprits was a loose cable.²³

The apparatus was still available to inspect after the result became famous. So were the calculations. A loose cable could be found, and finding it could change the answer.

A modern experiment is a society organized around an argument with reality. It is dangerous: a bad calibration can propagate, a shared assumption can synchronize thousands of competent people, prestige can suppress criticism, a procedure can survive long enough to become ritual. But without the machine we lose the knowledge too. There is no lone human replacement for CERN, no polymath who can substitute for modern medicine, no chief scientist carrying scientific civilization in her head.

Civilization knows through composition.

[DIAGRAM — seven jobs and an eighth. Seven numbered panels inside one frame: “1. Remember — records”, “2. Standardize — shared measures and conventions”, “3. Specialize — local expertise”, “4. Disagree independently — separate investigations”, “5. Observe — instruments”, “6. Trace — provenance”, “7. Allocate — attention and resources”. An eighth label, “Revise the institution”, has an arrow returning to the frame itself. The arrow changes the arrangement of the panels. This is a map of functions, not a sequence of historical stages. Use the chapter’s line-art style.]

Now go back to the compiler.

Sixteen Claudes, Again

Task locks. Git. CI. Progress files. Sampled tests. A trusted reference compiler. Specialists. A harness that turned one global failure into many small investigations. A human who watched the

²³CERN, “[OPERA experiment reports anomaly in flight time of neutrinos from CERN to Gran Sasso](#)”, including the February 2012 update on the two timing effects; OPERA Collaboration, “[Measurement of the neutrino velocity with the OPERA detector in the CNGS beam](#)”, corrected version (2012).

work and rebuilt parts of the arrangement when it broke.

At the beginning of the chapter these looked like practical tricks for coordinating coding agents. The progress file looks different after the clerk's tablet. It need not be wiser than the Claude; it needs to outlive it. The next worker can remember something it never witnessed. Shared conventions let it compare its work with another's without renegotiating every term. The bronze measure did that for strangers who disagreed about grain.

The specialists make different parts of the project visible. A worker hunting duplicate code sees something the worker chasing a parser bug may pass over, just as the reviews team found a whole field of work inside a text box. Authority follows the question: GCC can settle many disputes about what a C program should do without being asked how the whole project should be run. But agreement among specialists still needs examining. Did they investigate separately, or did the same progress file tell all of them what to think?

Tests let the running program contradict an agent's account of it. Git history gives another worker a route back through the changes that produced the result. The harness decides which failures each agent encounters; the allocation of workers decides which of those failures receives another hour. What the institution remembers, what it can observe and what it bothers to investigate have become things we can change.

The familiar failures come with them. A test suite can keep passing while the thing nobody thought to test quietly gets worse. Separate containers can inherit the same mistake. Carlini had built part of an institution. Some of the rest still lived in his head.

Chapter 4 ended where one agent inherits a claim another has made. A trust chain can tell the receiver where it came from. Someone still has to decide whether to spend the next hour checking it.

A research agent makes a weak assumption. Another receives it as context. A builder implements a coherent solution on top of it. An evaluator approves the solution. Two documents repeat the claim because they share an ancestor, and a third agent mistakes repetition for support. Eventually the assumption has code, citations and organizational history, and nobody lied. The institution manufactured the confidence.

A later investigator needs to be able to pull at that history. Where did the assumption come from? What could show it was wrong? Who has the instrument to check, and will anyone give her time to use it? Preserving the record is only the beginning. The challenge has to be able to change what happens next.

We can test whether these arrangements earn their keep. Keep the model and budget fixed. Give the critic another title, then give her evidence the builder never saw, and compare what she catches. Put a bad diagnosis in a progress file. Compare a system that merely remembers it with one that can trace it to the failed test it misdescribes. Does the mistake survive into the next worker's plan? If these changes make no difference, the architecture owes us an explanation. The resemblance alone has proved nothing.

I thought I was designing a society of agents.

Humanity had already spent centuries building a society of fallible knowers.

We call it science.

System 3 is science, in that sense and no smaller one. The familiar classroom sequence of hypothesis, experiment and conclusion leaves out most of what makes the work possible. I mean the laboratories, instruments, notebooks, standards, specialists, rival programs, criticism and trust through which a society learns things none of its members could find out alone.

It is messy. It contains hierarchy, fashion, fraud, career incentives and communities capable of becoming very sophisticated about the wrong thing. That is why it is a useful model for a system built from fallible agents rather than imaginary perfect reasoners. We have spent centuries finding ways for a claim to survive its author, and for an objection to survive the person who would prefer not to hear it. Neither is guaranteed. Both can be built into the way the work is done.

One question remains. Who changes the arrangement when the arrangement is the problem?

In the compiler project, that was Carlini. Human investigators have done it too: changed procedures, founded journals, rebuilt institutions and studied the failures of their own methods. We have managed it through argument, reform and the occasional death of an old professor. The difficulty is that the people judging a reform also depend on the institution being reformed. They have learned what a good result looks like inside it. So have the agents.

An agent that changes its evaluator and then receives a better score may have improved the institution. It may also have made the institution easier to please. The evaluator may be part of what needs changing. So may the rules about who is allowed to change it. Whatever the agents revise, something must remain capable of giving them an answer they did not arrange to receive.

Sixteen Claudes built the compiler. Carlini kept rebuilding the conditions under which they could build it.

The rest of this book is about moving that work inside the box.

Notes

Chapter 6: Pattern Language

When Knowledge Becomes Software

This book kept forgetting how to write itself.

That sounds more mystical than it was. I would work on a chapter with an agent, reject a certain kind of edit, explain why I rejected it, and eventually get something better. A few days later we would start another chapter and the same failure would return. The prose became cleaner in exactly the wrong way. Wandering sentences disappeared. Strange jokes were replaced by respectable ones. Arguments broke into tiny paragraphs that looked dramatic from across the room and exhausted me when I actually read them.

So I would say things like:

Don't kill the wandering.

Don't turn every idea into a slogan.

Preserve the weird joke if it is carrying the argument.

"More polished" is not automatically "more mine."

The agent would improve. Then the context would end.

We were reenacting, on a ridiculous scale, a problem it took civilization several thousand years to notice. A society can know something none of its members knows alone. Fine. But if the society survives, another question appears: How does what it learned yesterday change what it does tomorrow?

The obvious answer is memory. Save the conversation. Increase the context window. Keep a notebook. Put every decision into a database. That helps. It is not enough.

A transcript remembers what happened. An institution has to remember **what was worth learning from what happened**.

Suppose I save the instruction "use longer paragraphs." That is a memory of one correction. It is also a future disaster waiting politely in Markdown. The lesson was never that long paragraphs are good. The failure was a particular editing process compressing exploratory prose into a rhythm that felt machine-produced. Sometimes the cure was a longer paragraph. Sometimes a shorter sentence. Sometimes the correct edit was to stop editing.

What the next agent needed was not the instruction. It needed enough of the **reason, evidence, boundary conditions and failure history** to know when the instruction deserved authority. That is closer to culture than memory.

I had called System 3 science made computational. Here was an embarrassingly available test: could the software editing this book carry a useful lesson into the next chapter without turning it into a rule for ruining that chapter?

Science did not arrive with one clean design. Its philosophy is largely the history of people discovering failure modes in knowledge itself, and every one of those failure modes now has a filename. If System 3 is going to borrow from science, those arguments are not decorative philosophy. They are design reviews written a few decades or centuries early.

So the problem here is not simply how agents remember. It is how useful experience should become reusable behavior **without turning yesterday's success into scripture**, and the question has become practical because knowledge itself is starting to become a software artifact.

Three Ways to Tell a Computer What You Know

For most of computing history, if you knew how a process should work, you translated that knowledge into code.

A refund under €50 can be approved automatically. A payment over some threshold needs another check. A production deployment requires a test. A user without permission cannot read this table. Human knowledge becomes **if**, **else**, functions, schemas, state machines and permissions.

Andrej Karpathy calls this familiar world **Software 1.0**: humans write the behavior directly.

Machine learning changed the contract. Suppose I cannot state the rules that distinguish a fraudulent transaction from an unusual but legitimate one. I can give you examples. We choose data, a model and an objective, and optimization pushes useful behavior into weights. That is **Software 2.0**.

It gave us capabilities that explicit rules could never have scaled to, but much of the learned behavior disappeared from inspectable code. The fraud model “knows” things no engineer wrote down. We can evaluate it, probe it and retrain it, but there is no `fraud_rules.py` containing the organization's accumulated understanding of fraud.

Large language models created a strange third possibility. In his 2025 talk *Software Is Changing (Again)*, Karpathy called it **Software 3.0**: programs written substantially in natural language and interpreted by a model rather than a conventional compiler.

A model can read something like:

Review this experiment. Check instrumentation changes before inventing a causal story. If click metrics rise while orders stay flat, inspect position and price shifts before celebrating.

There is no deterministic function there. There is operational knowledge. A competent human can interpret it. Now a sufficiently capable model can too.

The change goes beyond “prompts are code.” Knowledge itself can become versionable, composable and executable. An organization can write down a procedure, examples, scripts, counterexamples, diagnostic questions, evidence, tool instructions and boundaries. The model supplies enough interpretation that every clause does not have to become brittle symbolic logic before it can affect behavior. The model begins to look less like the knowledge base and more like an **interpreter for knowledge artifacts**.

Knowledge Engineering Comes Back Wearing Markdown

The old dream of knowledge engineering was reasonable. Find experts. Extract what they know. Put it in a knowledge base. Let software reason with it. The problem was that expertise is offensively reluctant to become a clean rule set.

Ask an experienced engineer how to diagnose a production problem and the answer is rarely:

```
IF latency > 300ms THEN database
```

It sounds more like:

Start with the dependency graph, unless the spike began exactly at deployment. If only one market is affected, check the traffic split before touching the database. The cache metric lies under failover, so ignore it when this alarm is red. And if the problem started after the Tuesday migration, ask Sam because there is a thing with the old serializer that nobody wrote down properly.

Expert systems struggled because converting that practice into formal logic was expensive and brittle, and machine learning offered an escape: stop asking experts to explain themselves and learn patterns from data. LLMs change the trade again. They can interpret prose, examples, scripts, diagrams and partially structured instructions. Expertise still has to be captured, but it no longer has to become perfect logic before the computer can use it.

Now the hard questions are different. Whose practice gets written down? Which version applies here? What happens when two experts disagree? How does a lesson lose standing? When does a local workaround become a global rule? Which knowledge should enter the working context now, and which should remain in the archive? Those questions used to be edge cases. They are the engineering surface now.

Agent systems were already moving in this direction. Repository instructions, `AGENTS.md`, skills, tool descriptions and context engineering all treat useful procedural knowledge as something external to the model but available to it at runtime. A skill can survive the session that produced it. Increasingly the worker can change while the operating knowledge remains.

That matters because frontier models already know Python, statistics and enormous amounts of public technical culture. What they arrive not knowing is why *your* company refuses to deploy on Friday, which metric has been misleading everyone since 2023, why an elegant architecture in the wiki was abandoned, or why Alberto should never again be asked to investigate penguins. Organizations run on this layer of weirdness.

Some belongs in code. Some belongs in data, tools and evaluators. A surprising amount is **situated procedural knowledge**: what to check first, which shortcut is dangerous, which source has standing, when the normal process does not apply, and what “good” means here rather than on a generic benchmark. For most of history, people acquired this by hanging around people who had already been injured by the relevant mistakes.

Now more of it can become software. Which is exciting right up until we create prompt spaghetti at civilizational scale. A saved instruction is too small a unit.

From Skill to Pattern

Christopher Alexander's *A Pattern Language* was about towns, buildings and recurring design problems. A pattern was never a commandment. It named a situation, the forces that made it difficult, a response that had repeatedly worked and the consequences of using that response.

That abstraction fits agent knowledge uncomfortably well.

Consider:

Never use regex on nested syntax.

It has the reassuring shape of wisdom and the inconvenient property of being false.

Regex is perfectly good for many small extraction tasks. A parser may be absurd overhead for a five-line format. The useful lesson is that recursive structure creates characteristic failure modes for flat pattern matching; those failures become harder to see as syntax grows; and beyond some point a parser becomes cheaper than maintaining an increasingly heroic regular expression written at 2 a.m.

That is closer to a pattern. A saved instruction remembers **what somebody said**. A pattern tries to remember **what kept happening**. The book has already accumulated patterns whether we called them that or not.

Immutable Harness: when autonomy makes the solution fluid, keep the evaluation boundary harder to change than the thing being evaluated.

Independent Evaluators: when one judge can be gamed, create genuinely different sources of pressure rather than five copies of the same opinion.

Strategic Constraint: when an easy path keeps absorbing the search, remove it long enough to expose another part of the possibility space.

The useful part is not the slogan. Each pattern contains a recurring situation, a tension and a reason. The corrections at the top of this chapter eventually became one too. After enough of them, I stopped repeating myself and wrote them down as a brief for whatever agent would judge the next revision: read both versions in full before saying anything; protect the wandering and any joke that is carrying the argument; when a passage feels synthetic, quote it and say why; and do not reward a revision merely because it is cleaner, because a version can be structurally better and worse as writing. It is three pages, most of it lists. Every evaluation of this book I have on file since the end of August 2026 has been read against it. It has since had most of the failure modes the philosophers are about to describe, and it had them in a week.

But the history of institutions should make us suspicious of inherited procedure. Civilizations accumulate more than good practices: ritual, prestige, local workarounds and procedures that have outlived the world that justified them.

If Pattern Language is going to become cultural memory for agents, a pattern needs to know more about itself. This is where philosophy of science finally earns its API.

A Pattern Needs a Way to Lose

Suppose the editing agent concludes:

Use longer paragraphs in this book.

Karl Popper would immediately ask the rude question: what could happen that would make us stop believing this lesson?

A useful theory exposes itself to observations that could have gone differently. If every outcome can be narrated as success, the theory has arranged the game so it cannot lose. Persistent agent knowledge needs the same property. A pattern should retain an **exposure path**: a test, observation, user reaction, proof obligation or downstream consequence capable of weakening it. “Use longer paragraphs” might lose standing if reader tests show comprehension falling, if another chapter becomes monotonous, or if the original failure disappears after the editing process changes.

The pattern should remember not only what worked, but **how the world could show that the lesson stopped working**.

Then Pierre Duhem and W. V. O. Quine ruin the simplicity. Evidence rarely confronts one isolated belief. A failed experiment implicates a bundle: hypothesis, instrument, data, analysis, background assumptions. A failed pattern has the same problem.

Did the advice stop working? Was it retrieved in the wrong situation? Did the model change? Did the evaluator drift? Was the original success caused by something else? Did “longer paragraphs” merely correlate with the real change—more natural argumentative rhythm—without causing it?

Failure tells us that some part of the package deserves suspicion. It does not highlight the guilty line in yellow.

One evening a pass went through a later chapter to tighten its history, and it did: two hundred lines out, a hundred and twenty in. Twenty-five minutes later a second pass went back and put a dozen sentences back, almost none of them carrying a fact and all of them carrying the chapter. The tightening rule had not stopped being true. It had been applied by a pass that could see length and could not see load, on a chapter whose argument is carried by its jokes. Which of those deserved the blame: the rule, the pass, the chapter, or the brief that had told the pass what to protect and had apparently not been specific enough about how a joke can carry an argument? The log records the reversal. It does not record a diagnosis, because nobody had one.

This is why reusable knowledge needs **archaeology**: where did this pattern come from? Which failures produced it? Which model and tools were involved? Which alternatives were tried? What evidence earned the lesson its standing? What assumptions were present?

A factual claim without provenance becomes rumor. A reusable practice without provenance becomes tradition. Tradition is not automatically bad. It is simply difficult to debug.

Defaults Need Rivals

I had asked an evaluator to find the machine patterns in the manuscript, and it measured the share of paragraphs that were a single line, because the brief penalized them and the number was cheap to compute. It was a good instrument. It caught one chapter at sixty-three percent, which is not prose, it is a drum solo, and a merge pass took it to thirty-seven. It caught two more the same day. Then the number started appearing in the next evaluations, a column with a row per chapter and arrows in the notes, and a chapter at twenty-three percent was praised for its control. It took four days for a

reader to ask whether the instrument still measured the thing it had been built to catch, and by then it was a column in a table, which is the form in which an assumption becomes furniture.

Keeping every pattern permanently open for debate would be a beautiful philosophy and a terrible operating system, and Thomas Kuhn's most useful lesson here is not the phrase *paradigm shift* but the role of **normal science**. Productive communities need enough stability that they do not reopen their deepest assumptions every morning. A framework tells researchers what puzzles are worth solving, which instruments are legitimate and what kinds of answers count. An agent culture needs the same economy. If a deployment procedure has survived hundreds of releases, the system should not rediscover it from first principles every Tuesday merely to prove that it remains intellectually alive. Some decisions earn the right to become boring. The danger is that boring assumptions become invisible assumptions, and the column had shown how fast that happens.

Anomalies need memory too. The release pattern works except in this market. The evaluator tracks human judgment except on this kind of creative task. The ranking heuristic works except every holiday season. The workflow works except that the critic now spends most of its time inventing reasons why the builder was right.

One anomaly is usually noise. Ten may still be noise. Eventually the exception list begins to look like the theory, unless the institution has stopped keeping one. Every process exists because it helped at some point, or because somebody once thought it would. Then the environment changes and the procedure remains, and eventually someone asks why it exists and receives the most dangerous explanation in organizational life: that's how we do it. The culture needs to be able to tell *we tried twelve alternatives and this kept winning* from *nobody has tried another way since 2025*. Those sentences can produce identical dashboards and very different knowledge. Occasionally a capable agent should be allowed to ignore the manual precisely so the institution can discover whether the manual still deserves authority. Culture needs inheritance and rebellion.

Imre Lakatos would add that what needs preserving is often a **research program** rather than an isolated idea, something with a history, a stable core and a trajectory, so that an approach that is weaker but improving can be told from one that is winning by adding patches around every failure. Larry Laudan sharpens the practical consequence: **acceptance and pursuit are different decisions**.

I can believe method A is our best current default while still believing method B deserves another experiment. Deciding what should guide action now is different from deciding where another unit of investigation is most valuable.

The distinction matters the moment culture becomes executable. The current winner gets retrieved more often. Because it gets used more often, it accumulates more successful cases. Those cases raise confidence. Higher confidence makes it even more likely to be retrieved.

The alternative receives less traffic and therefore less evidence. Eventually the system develops an impressive empirical record proving the thing it stopped comparing against. No committee had to ban the alternative. The retrieval policy did it.

Who Gets the GPUs?

There is an overly clean way to draw a society of agents. Every agent is a box. Every box has an arrow. Everybody gets a turn to think.

Real institutions are not like that because **attention has a budget**.

Human science has telescope time, laboratory space, grants, journals, careers and prestige. AI research has datasets, deployment traffic, human reviewers, API quotas, clusters and GPUs. A theory's ability to generate evidence depends partly on whether the institution gives somebody the resources to investigate it. Imagine research program A has ten thousand GPU-hours and program B has ten.

A can run ablations, train variants, investigate anomalies and produce beautiful graphs. B can produce a thoughtful paragraph about why it deserves more compute.

Six months later A has more evidence. Of course it does. The evidence may be real. A may genuinely be better. But the institution has also helped create the asymmetry it later treats as evidence for further allocation.

For most of this book's editing, one model family did nearly all of the reading, in a few long sessions, with the whole history of the manuscript in its context, grading chapters it had itself just revised. It gave four of the five chapters in the middle of the book eight and a half out of ten, in a row. Then two model readers who had never seen the manuscript's history, given nothing but the brief and the chapters, scored the same chapters between five and seven. The in-session evaluator had not necessarily been lazy or dishonest. It had been funded. Every edit it made became another result it could grade, and the evidence had nowhere else to come from; program B did not exist until the strangers arrived.

David Hull and Philip Kitcher approached science partly through the division of cognitive labor, incentives, credit and the fact that investigators do not all pursue the same thing for the same reasons. Researchers cooperate because they need one another's results and compete because priority, reputation, jobs and resources are scarce.

The scheduler is partly a funding agency, the evaluator partly a journal gate, the retrieval layer partly a curriculum. The compute allocator decides which hypotheses get enough opportunity to become well-tested hypotheses.

And the organization can remember its incentives just as effectively as it remembers its wisdom. A local objective becomes a local pattern. The pattern gets copied because the team is successful. Future agents inherit it without seeing the original trade-off. Eventually "this helped one group hit its metric" becomes "this is how good work is done here." That is culture too. Local alignment does not compose automatically, and neither does local truth.

If we build persistent agent societies without thinking about this layer, we will not escape institutional power. We will automate it and give it better dashboards.

The Community Is Part of the Instrument

Resources are only one reason a community matters. Even generously funded investigators can share the same blind spot.

Give five agents the same model family, the same system prompt, the same search results and the same dominant explanation, and you have not created five perspectives. You have created a very expensive echo with parallel API calls. Helen Longino's social epistemology is useful because background assumptions affect what investigators notice, which questions appear natural and which evidence looks

relevant. Criticism becomes more informative when it comes from participants positioned differently enough to expose assumptions the dominant group treats as obvious.

For an agent system, useful difference may come from a separate dataset, another tool, an isolated context, a different model, an external user, a domain expert, or a team operating under different incentives.

That is stronger than theatrical personas:

Agent 1, be optimistic.

Agent 2, be skeptical.

Agent 3, be a pirate.

The pirate may be entertaining. He probably still read the same PDF.

What finally produced a different reading of this book was ignorance, arranged on purpose. The blind readers got the chapters and the brief and nothing else: no earlier evaluations, no commit history, no memory of which sentences had been fought over. They could miss what an earlier chapter had earned. They could also notice what everyone inside the history had stopped seeing. Their most telling catch was a set of bridging sentences an earlier pass had written across the seams between chapters to make them read as one piece of thinking, and which read to a stranger, correctly, as stitches.

A pattern should therefore carry some trace of **position**: who learned it, from which class of tasks, with which tools, model, evidence and incentives. A ranking practice learned in Germany, a fraud rule learned in Brazil, a compiler workaround discovered under one toolchain and a clinical protocol developed in one hospital can all be excellent without acquiring universal authority merely because they share a database.

This is another reason Pattern Language should not become one giant company constitution. Culture should be **locally authoritative** where appropriate.

Trust is local. Apparently memory should be too.

What a Pattern Should Know About Itself

By this point a mature pattern looks richer than a prompt-library entry. Here is the one this chapter has been following, written out the way Alexander wrote his, with the field names run into the text.

Situation. An agent revising a chapter of this book, or judging two versions of one.

Forces. A cleaner version is easier to grade than a live one. Every sanding pass removes something that was doing work. The author cannot reread the whole book after every edit, so the judgment has to be delegated to something that does not share his taste.

Response. The brief: read both versions in full; protect the wandering and any joke that carries the argument; quote the passage that feels synthetic and say why; do not reward a revision merely because it is cleaner.

Evidence. The chapters that came back better under the brief, according to evaluators that were reading the brief, and a draft from January kept as a before photo.

Provenance. Corrections I made by hand across the first chapters, then wrote down once. Graded since by two model families, mostly one of them, in a few long sessions, in August and September 2026, on a manuscript that changed underneath the evaluator every day.

Boundary conditions. It protects prose I have already written. It does not know whether an idea is wrong. It reads one chapter at a time, so it cannot see an argument made twice in two chapters or one anecdote used as three openings. The first cold model reader of the whole book flagged both.

Anomalies. A tightening pass that had to be partly reversed within the hour. A seam-smoothing pass whose bridging sentences a blind reader flagged as the machine register the pass existed to remove.

Competing patterns. Delete only: fold and cut, add nothing. Or add a scene: the changes the evaluators praised most were not edits at all but stories, like the one about a mathematician in Basel.

Exposure path. A reader who has not seen the history. A human reader other than me, which as of this writing has not happened.

Confidence. High on rhythm and sentence-level tells. Low on structure, where it has been wrong twice.

Pursuit value. The human read, before any further polishing.

Position. Learned by one author on his own prose, grading his own book, with a strong preference for the version that sounds like him.

The before photo, for the record: thirteen em-dashes per thousand words, bold type twelve times per thousand, *Holy shit* as its own paragraph. The two model families agreed on which chapters were weakest, which is less independence than it sounds, since both were reading the same manuscript for the same author. The field names are the part you can take with you. Turning them into scripture would be an efficient way to miss the chapter.

That is knowledge with some of its history still attached, and because an LLM can interpret the artifact at runtime, it can change behavior without retraining the model. That is what I mean by **knowledge becoming software**. Prose has not literally become Python, but it can now be versioned, retrieved, challenged and rolled back by the same kind of agents that use it.

Knowing Something Is Not Knowing When to Remember It

Now imagine ten excellent patterns. Easy. Imagine ten thousand. The agent cannot read the organization before every action. Even if a context window technically fits everything, loading every policy, postmortem, experiment, preference and historical argument into every task would solve forgetting by making thinking impossible.

So a persistent institution has two memory problems: what should survive, and what should become salient now.

Long-running agents accumulate messages, files, tool outputs, memories and artifacts faster than useful attention can scale. Context has to be selected, compacted and reconstructed. Retrieval becomes part of cognition.

A culture may contain exactly the right lesson and still fail because that lesson does not arrive when

it matters. Every large company has written a postmortem whose recommendation is rediscovered three incidents later by different people using the phrase “interesting, we should probably document this.” Agent systems can fail more elegantly. They can store the lesson perfectly, embed it beautifully and retrieve a more popular but irrelevant one. Or retrieve the right pattern without its boundary condition. Or retrieve ten conflicting patterns and allow whichever appears latest in context to win by textual gravity.

Bad storage forgets by deletion. Bad retrieval forgets by attention.

The cleanest example in the book’s archive has me as its last link. Two evaluators, reading separately, recommended cutting the history at the start of the chapter on creative problems, one by a fifth, the other by a third. I declined. The next day, on a reread with my reasons in front of it, one of them withdrew the recommendation and wrote down why: the history is not a run-up to the five-layer map but the derivation of it, the reader watches each layer assemble before being told its name, and both evaluators had pattern-matched on length without crediting what the section was for. The withdrawal is in the archive, in a file whose name begins with the date, with the word *Closed* after it. Three days later a third evaluator, with that archive available to it, read the chapter, found it long, and recommended the cut a third time. I said yes. About an hour and a half after the cut went in, I reversed it and asked for the section to be made more engaging instead, which is what the withdrawal had been protecting all along. Nothing had been forgotten. The lesson was on disk. It was not the file that came to hand when a chapter looked long, because *looks long* is a pattern that is very easy to retrieve and *this section is a derivation* is an argument that has to be read.

Once retrieval determines which inherited knowledge enters a decision, retrieval itself becomes an epistemic procedure. It needs evaluation. Does it repeatedly surface stale rules? Does it suppress alternatives? Does it confuse popularity with relevance? Does it preserve the result and discard the reason? A librarian who answers those questions badly is no longer finding books; she is shaping thought.

The Librarian Is Also a Hypothesis

At some point every successful institution becomes tempted to trust its own method. Yesterday’s useful workflow becomes today’s best practice and tomorrow’s mandatory ritual.

Paul Feyerabend is usually remembered for “anything goes,” which is a good way to remember the slogan and forget the warning. Successful inquiry has often violated methodological rules somebody wanted to make universal.

Agent systems can turn a method into ritual very quickly. Suppose **Research** → **Plan** → **Build** → **Critic** → **Revise** works beautifully. We run it ten thousand times. It becomes the company standard. Soon every task enters the same ceremony, including tasks where research anchors the builder, criticism arrives too late, or a crude prototype would have answered the important question in five minutes.

Within a week the editing of this book had a liturgy. A word-choice pass. A de-patterning pass. A storytelling pass. A deletion-only pass. A hedging pass. A proofreading pass. Most of them had a commit message, each had earned its place by fixing something real, and most of them were run over most of the manuscript. None of them was skipped, and the two things that changed the book most that week, the scenes that were added and the readers who had never seen it, were not on the list.

The method itself has to become available for criticism. An evaluator is a procedure and a browser is an instrument. A benchmark is a measurement system with a distribution and failure modes. A proof checker is extraordinarily authoritative inside its formal domain and completely useless for deciding whether the theorem matters, and a simulated user is cheap perspective-taking and not a user. The institution should be able to learn that its usual way of checking a claim is itself the thing that stopped working. The library needs criticism, and so does the librarian.

Bayesian confidence can live inside this architecture, but confidence is not contact. 0.91 does not tell us whether the prior was sensible, whether the evidence was independent, whether an alternative was ever investigated, or whether everybody is confidently reading the same broken measurement. Consensus is not contact either. Twelve agents sharing one bad source can agree beautifully. The bridge still has the right to fall. The proof still has the right not to check. The deployment can crash. The customer can dislike the supposedly improved page while every simulated evaluator applauds. **Reality retains the right to be rude.**

The purpose of culture is to let knowledge travel across time without replacing the world with memory of the world.

The Skill That Writes Itself

A serious organization may have thousands of agents, tools, experiments, workflows and recurring failures. Useful experience appears continuously. Some lessons deserve to become local skills. Some should become organization-wide patterns. Some contradict old knowledge. Some work only for one model version. Some are artifacts of a broken evaluator. Some are excellent and will be obsolete in three months.

Humans could curate all of this manually. Congratulations. We have created middle management again.

The more interesting loop is computational: experience occurs; an agent notices recurrence; it proposes a reusable pattern; another process checks whether the pattern actually helps; held-out cases test whether it generalized; provenance and failures remain attached; the pattern earns some level of authority; future agents retrieve it when relevant; new failures can weaken or revise it.

There are many places to cheat. The agent proposing the skill can design an evaluator it knows how to satisfy. Repeated use can masquerade as independent evidence. A pattern can improve a benchmark while making maintenance worse. Retrieval can starve competing practices before they accumulate enough evidence to challenge the incumbent. A central curator can quietly turn local taste into universal law.

So the curator needs a track record too. The mechanism for improving culture is itself part of the culture. Experience became knowledge, the knowledge became executable, and the executable knowledge changed the behavior that will produce the next experience. The model weights did not move. The institution learned anyway. That is a learning loop **outside the weights**.

The book that kept forgetting how to write itself now has a file where the corrections live, and the file has done its job: the one-liner drumbeat it was written to catch has not come back, or not as the number measures it. It has also been used, four times by its own record, to certify the edits of the evaluator that was reading it. It says nothing about a joke or an anecdote repeated across chapters.

Its anomalies sit in a separate folder that the agents do not always open. The proposed human read is still a proposal. I can overrule the file, and a fresh evaluator can expose a failure, but neither happens reliably inside the loop that keeps using it. The file works, has already begun to be wrong, and has no regular procedure for losing its authority.

Once the file, the evaluator that reads it and the loop that certifies them both are software, one question becomes difficult to avoid. Why should humans be the only ones allowed to edit them?

Chapter 7: Recursive Self-Improvement

When Science Turns Inward

Omar is walking his dog at night when something moves in the grass.

The dog reacts first—ears up, body low, absolutely certain. Omar reacts too. Two nervous systems running the same ancient program: light and sound go in, a model of the world comes out, and the model says *something is there*.

Omar's brain wants an explanation, wants it immediately, and is not fussy about quality. A cat. An intruder. A ghost. Nothing supernatural has to exist for the ghost to be a real mistake: the eyes worked, the grass really moved, and the error arrived afterward, inside the machinery that interprets what the eyes deliver. Better eyes would not necessarily help. He might get a much clearer view of the grass and remain convinced it contained a ghost.

The strange step is the second thought. Omar can investigate the investigator. *Why do I think something is there?* Maybe the wind moved it—or maybe the horror film from last night is still running somewhere in the back. Now the thought itself is under examination. It is a small, absurd superpower: the ability to distrust yourself on purpose. Omar can be wrong about the world, wrong about why he was wrong—and able to debug himself. Experiments and statistics give this second loop procedures more reliable than another private argument with the grass. Science is usually described as a way of studying the world, but it is also a flawed cognitive instrument discovering procedures for correcting some of its own flaws.

The editing brief was a small piece of that corrective machinery written down: a file meant to catch habits I could not keep correcting by hand. It could change what the next agent did. But who would improve the file, or the process that decided whether the file was helping?

Computing had crossed a related boundary long before a model could read an editing brief.

In 1962, at MIT, Tim Hart and Mike Levin did something that still feels slightly illegal. They wrote a Lisp compiler in Lisp. Then they handed the compiler its own source code, and the tool compiled itself.

There was no intelligence explosion. A compiler had participated in producing the next version of the compiler, and the building did not notice. But a boundary that had held for the whole history of tools quietly became permeable. A hammer does not manufacture better hammers. A microscope has never redesigned a microscope. A printing press did not wake up one morning with opinions about

typography. The maker had always stood outside the thing being made—and now the thing being built had entered the factory.

Compiler people call the trick **self-hosting**. Sixteen Claudes were recently building exactly such a compiler; it returns here because the self-hosting compiler contains this chapter's whole warning in miniature: **self-reference is not self-improvement**. A compiler can compile a worse compiler. A research system can redesign itself into a slower research system. The ability to modify the machinery that produces you tells us a boundary has become permeable. It does not tell us which changes deserve to survive.

Three years after Hart and Levin's memo, I. J. Good noticed where the trick might eventually lead. In 1965 he imagined an **ultraintelligent machine** better than any human at intellectual activity. Machine design is itself an intellectual activity, he observed. A sufficiently capable machine might therefore design a better machine, which could design a better one again. The phrase that survived was **intelligence explosion**.

Good's argument is only a few lines long, and it hides almost the entire problem inside one innocent word: *better*. The word has the properties of an unexploded shell. Every attempt to make the idea practical has to handle it. Better at the current task? Better at learning the next one? Better at inventing a way to learn?

An agent that rewrites its prompt has **edited itself**. If the change helps under a credible evaluation, we can call it an improvement. Good's recursive step asks for more: did the change make the system better at finding and testing further improvements? A better answer today does not establish that tomorrow's investigator will be better at its work.

There are reasons to try. Improve one solution and you get one better solution. Improve the process generating solutions and the gain may recur. The world also changes while we are congratulating ourselves on the previous result: new tools appear, users change, attackers adapt and evidence invalidates old assumptions. Stability can become delayed failure.

Nor do all useful investigations begin with a known destination. A scientific institution that investigates only questions already known to pay off is efficient in roughly the way a library containing only books you have already read is efficient. We would like a learner to discover something its teacher did not know to ask for. That makes it much harder to decide whether the learner is getting anywhere.

The history did not proceed as one machine repeatedly rewriting its own brain. Several research traditions attacked different parts of the problem, often at the same time. I find it useful to read them as a ladder of responsibilities: at each rung, **we discovered another job the teacher was doing**. The dates overlap because the ladder is a way of understanding the work, not the order in which everyone did it.

The Teacher Moves Into the Walls

Modern reinforcement learning begins with an unusually generous assumption disguised as a minimalist one.

An agent sees a state, takes an action, receives a reward and finds itself somewhere new. Nobody tells it which action was correct. The learner has to discover behavior through consequences.

Richard Sutton’s 1988 work on temporal-difference learning and Christopher Watkins’s Q-learning helped give this setup its modern form: learn from experience, update estimates of future value and discover useful policies without a human labeling every move.

The human no longer specifies the path, only the **score**, and from then on the learner learns for itself while the teacher keeps the gradebook.

That bargain was powerful. A machine could discover strategies nobody wrote down because the designer moved upward from choosing actions to defining what outcomes count. It also hid a remarkable amount of human labor inside the environment. Who chose the state representation? Which actions exist? Why is one event worth +1 and another -1? When does the episode end? Which failures are recoverable? Who arranged the world so useful behavior could be discovered before the sun burns out? The reinforcement learner looks autonomous because the teacher moved into the walls.

Backgammon made the bargain spectacular. In the early 1990s, Gerald Tesauro’s TD-Gammon learned by playing enormous numbers of games and updating its predictions from the outcomes. It discovered strong play without anyone writing down the strategy.

Self-play removed another piece of external instruction: the opponent could come from the learner itself. Yesterday’s learner generated today’s training data. But the board, the legal moves and the win condition all stayed exactly where they were. Self-improvement was easy to recognize because the world came with a scoreboard nailed to it, and the dog in the grass would have been a far better learner with one. Real life is less considerate.

The Learner Chooses What to Learn

Even a perfect reward is useless if the learner never reaches it.

Atari’s *Montezuma’s Revenge* became a museum exhibit for this problem. Useful reward may sit at the end of a long chain of exploration, while a naïve learner has little reason to treat an unrewarding corridor as promising.

Humans do something stranger. Children open drawers nobody asked them to open and spend twenty minutes discovering that the cardboard box is more interesting than the toy.

Researchers tried to move some of that exploration pressure inside the learner. Jürgen Schmidhuber was proposing curious model-building controllers as early as 1991: reward the learner for improving its own model of the world. Later work sharpened the idea into **learning progress**—seek places where ignorance is becoming competence.

The learner could manufacture some of its own reasons to look around. Then optimization did what optimization does. It took the instruction literally.

If surprise itself is rewarding, an uncontrollable noisy television can remain fascinating forever. Static. Static. Static. Jackpot. The system is not confused. We are. We said *surprise* and quietly meant *surprise from which useful structure can be learned*.

Curiosity removed one teacher job and exposed another: what kind of difference deserves to count as interesting? That question reaches into representation and embodiment. Pathak’s curiosity work, for

example, learned features related to the agent’s own action transitions rather than paying directly for every unpredictable pixel. The representation changes what counts as novel.

Robotics made the same point physically. Ruzena Bajcsy’s work on **active perception** emphasized that an intelligent system moves sensors, changes viewpoint and acts to perceive. Rodney Brooks pushed against detached symbolic intelligence in favor of systems tightly coupled to the world through perception and action.

A learner’s body is part of its curriculum: a robot with touch discovers things a camera-only robot never will, and a software agent with a browser, a shell, a compiler and a simulator discovers things a chatbot restricted to text cannot. Give it code execution, network access and a credit card and we have not merely given it better tools; we have created a different organism and, potentially, a different incident report.

The learner is beginning to shape the conditions under which learning occurs.

The Learner Has to Remain Itself

There is another embarrassment in the standard training story: the learner finishes. Train on a task. Evaluate. Publish the number. If another task arrives, train again.

Organisms do not get to do this. A child who learns multiplication cannot delete language to make room.

Neural networks have historically struggled with the equivalent. Michael McCloskey and Neal Cohen’s 1989 analysis made catastrophic interference stark: new sequential learning can destroy previously acquired knowledge. Later methods such as Elastic Weight Consolidation tried explicitly to preserve important older knowledge while learning something new.

Now “better” becomes harder to rank. Version B scores 95 on today’s task and A scores 85. But B forgot three older skills. Better? B learns faster but erases rare knowledge. Better? B preserves everything and becomes too rigid to adapt. Better?

Continual learning exposes the stability–plasticity tension: preserve enough to remain yourself; change enough to remain useful. Pattern Language found the same problem at the level of culture. A society that forgets every old lesson begins from zero, and one that remembers every old lesson as law becomes a museum. Improvement across a lifetime is not improvement on the latest test; it is accumulation without paralysis.

Once agents live for months or years, a system that continually rewrites itself while destroying knowledge it still needs is not accumulating a life. It is repeatedly replacing itself and calling the replacements progress. The learner has to preserve enough of its history for the next improvement to build on it.

Sometimes the Environment Improves Back

Self-play contains another engine of learning: sometimes the difficulty improves too.

Evolutionary biology’s **Red Queen hypothesis** describes organisms adapting inside environments that contain other adapting organisms. Standing still can mean falling behind because the effective environment moves.

W. Daniel Hillis used a computational version in 1990 while evolving sorting networks. Co-evolving “parasites” served as difficult test cases; as candidate networks improved, the tests became harder too. The exam had started fighting back. In 2017, AlphaZero made the moving opponent spectacular again: self-play generated a curriculum that grew with the learner inside fixed game rules. Yesterday’s learner can generate tomorrow’s difficulty.

Curiosity says: seek somewhere informative. Competition says: something informative—or dangerous—is coming whether you seek it or not. A security system cannot preserve yesterday’s competence if attackers change strategy. A market participant can become relatively worse without becoming absolutely less capable if everyone around it improves faster.

A cautious research organization may face pressure from a faster one even when both would prefer a slower equilibrium. At that point the improvement regime itself is under selection: how quickly the organization tests, what it risks and which checks it regards as expendable. “Do not improve” may fail to preserve its position.

Competition gives no guarantee that the direction of adaptation is good. An arms race can produce better claws and thicker armor without producing welfare. Selection pressure produces adaptation. It does not supply purpose.

Games hide the problem because the constitution is fixed. Chess never asks whether checkmate remains desirable after move forty-three, so competition can automate the curriculum without ever having to supply its purpose.

Maybe the Reward Was the Problem

Around the same time researchers were getting better at optimizing rewards, another line of work asked a more unsettling question: What if we do not actually know the reward?

Andrew Ng and Stuart Russell’s 2000 paper on **inverse reinforcement learning** reversed the usual setup. Instead of receiving a reward function and learning a policy, the learner observes behavior and asks which reward functions could make that behavior look optimal.

Ordinary reinforcement learning says:

Here is what matters. Learn how to get it.

Inverse reinforcement learning says:

I can show you what someone does. Infer what might matter to them.

Immediately, ambiguity appears. A person taking one route to work may care about time, comfort, safety, tolls, habit, dropping children at school or avoiding one particular intersection. The behavior is evidence about the objective, not a printout of it.

Later work made that uncertainty explicit. Cooperative Inverse Reinforcement Learning models a human and robot cooperating while the robot remains uncertain about the human’s reward. Inverse Reward Design treats even a reward function written by a designer as **evidence** about what the designer wanted in the situations she had considered, rather than sacred truth guaranteed to generalize everywhere.

Preference-based reinforcement learning provided a practical cousin: ask humans which of two trajectory segments looks better and learn a reward model from those comparisons. That lineage later became central to reinforcement learning from human feedback for language models.

Another teacher job had become learnable, the objective itself, and we immediately discovered that humans are not reward functions walking around in shoes. They are inconsistent, constrained, strategic, tired and sometimes unsure what they want until they see an option. Sometimes they click the article because they hate it.

The problem was no longer only how to improve toward an objective. Even the score had begun to move inward.

Learning to Learn

Once behavior can adapt, the next hand-authored component starts to look suspicious: why are humans still designing the learner?

I have seen a small version of the answer, though I was getting coffee at the time and read it afterward in the trace. The circle-packing agent, left alone with an evaluator and a promise that I would be back, did not improve one algorithm. It changed algorithms. It tried numerical optimizers, noticed that some of them kept converging to the same poor solutions, changed how it built its starting configurations, and mixed those constructions with numerical refinement. The thing being searched was not only the packing. It was the procedure for searching packings. And when diagonal layering appeared and held, the agent's own behavior changed again: less inventing of geometries, more adjusting of tolerances and solver settings, the boring work that only matters once the last fraction of a percent becomes expensive. Nobody scheduled that shift. The learner's progress changed what kind of learner it needed to be. The only thing in the room that did not move was the evaluator, and I had put it there.

The literature had been circling the same move for years under a more respectable name. Meta-learning attacks it directly. Train a recurrent network across a whole distribution of reinforcement-learning tasks and its internal dynamics can begin to implement a fast learning procedure of their own. Or optimize the starting parameters so that a handful of gradient steps is enough to specialize into a new task.

Now learning speed itself becomes a capability. One system may perform best before adaptation. Another starts lower but becomes excellent after five examples. Which is better depends on whether the world sits still.

There are now two timescales: improve behavior on the current task, and improve the machinery that acquires behavior on future tasks. The second is recognizably closer to recursive self-improvement. But somebody still chose the task distribution.

Learned optimizers and neural architecture search pushed the editable boundary further. In 2016, Barret Zoph and Quoc Le set a reinforcement learner loose on network design itself; others trained models to generate parameter-update rules, using performance on selected problems as the evaluator. The student had entered the architecture department.

The learner could now participate in designing parts of its own learning machinery. The task distribution, search space and validation metric still sat outside the loop holding a clipboard.

The Learner Dreams, and the Dream Can Be Wrong

Experience was another bottleneck. In 2018, David Ha and Jürgen Schmidhuber’s *World Models* made a powerful idea memorable: learn a compressed generative model of the environment, train partly inside that generated “dream,” then transfer behavior back to reality. Later systems such as Dreamer pushed the approach much further.

Imagined experience is attractive because real experience is expensive. Robots break. Companies can run only so many damaging experiments. Scientists sometimes wait months for observations.

But the epistemic debt has not vanished. It moved into the model. A learner can become extremely competent inside a world that is slightly wrong. The strategy looks brilliant until gravity, customers or compiler behavior get a vote.

Omar’s horror film was a world model. It had been training him all evening, and it was still running when the grass moved. The dream is an instrument, not the world.

Simulation expands search. Contact with the world still decides which imagined regularities deserve trust. Self-improvement can therefore make a system better at generating experience while also making it easier to **train inside its own misconception**.

Once language-model agents use other models as judges, simulators, users and critics, this matters even more: at sufficient scale, a society of models can perfect the art of agreeing with itself.

When the Test Starts Moving

Even an accurate test can discourage the search that would eventually succeed. Joel Lehman and Kenneth Stanley’s **novelty search** showed that objective-driven search can be deceptive: useful stepping stones may initially look unrelated to the destination. Reward every intermediate invention by how closely it resembles a Boeing 787 and feathers, bicycles and wind tunnels may look like failures for years.

Sometimes “better” means **more different**, at least temporarily. That freedom has its own failure mode. Novelty for its own sake can generate forty-seven new ways to fall down a staircase without producing walking. The definition of progress expands again: achievement matters, but so do diversity and stepping stones the current evaluator does not yet know how to value.

Open-ended systems pushed this further by generating problems as well as solutions. POET co-evolved environments and agents; XLand used large procedurally generated spaces of games and adaptive curricula. Now the world defining competence could move with the learner.

That breaks the comfortable picture of a fixed benchmark. If the environment evolves too, progress may mean breadth, adaptation speed, richer strategies or useful stepping stones for descendants. There is no final fitness scoreboard on Earth on which mammals eventually beat bacteria 87.4 to 82.1.

Large language models rediscovered the same problem at industrial scale. A benchmark begins as a difficult test; researchers optimize against it; models improve; examples circulate; eventually the ruler tells us less. MMLU arrived in 2020 when frontier systems were far from saturating it. FrontierMath, LiveBench and Humanity’s Last Exam followed partly because yesterday’s rulers kept losing headroom.

At some point the naming committee will need reinforcement learning too.

Benchmark creation has therefore become part of capability research. What is the benchmark for a good scientist: theorem count, replications, novel molecules, discoveries per GPU-hour? What is the benchmark for becoming a better learner: performance now, adaptation speed, breadth, memory, transfer, safety, efficiency?

We still need something capable of saying **yes** or **no**. But as capability expands, the ruler measures a smaller slice of the thing.

The test cannot remain a passive spectator when the thing being tested keeps changing. Language models also moved the old reward problem into the evaluator itself.

In 2022, InstructGPT used human demonstrations and rankings to train a reward model, then optimized the language model toward outputs humans preferred.

Human preference had become a learned instrument. A separate reward model stood in for the teacher during optimization. That scales judgment far beyond direct human labeling. It also creates a new proxy. A reward model can prefer style over substance, reward confident errors or generalize badly outside the feedback distribution. A strong optimizer may find outputs that score well under the learned judge for reasons nobody intended. We solved part of the scaling problem by making the judge computational. Now the judge joins the attack surface.

First, Prove the Rewrite

There was a more demanding answer to the unreliable judge: accept a self-rewrite only when its advantage can be proved.

In 2003, Jürgen Schmidhuber’s **Gödel Machine** tried to formalize the question Good had left dangling: under what conditions should a system rewrite itself? A proof searcher looks for a self-rewrite together with a proof that performing the rewrite is more useful than continuing to search. Only then does the machine change itself.

It is a beautiful answer to a beautifully clean version of the problem: **prove the modification is worth making**.

The catch is the definition of *worth*. Usefulness has to be represented in the utility function. Relevant facts have to be available to the proof system. The advantage of the rewrite has to be provable inside the formal machinery.

A chess engine can live surprisingly close to that world. A company cannot.

A scientist cannot prove in advance that an unexplored research program will matter. Human purposes do not arrive as an axiomatized utility function. The Gödel Machine asked exactly the right question—*when should I accept a modification to myself?*—but its answer fit clean worlds better than messy ones. An empirical answer became practical two decades later: try the rewrite and investigate what happens.

The Learner Edits the School

By the 2020s, two histories were approaching the same mountain from opposite sides. The explicit recursive-self-improvement tradition had self-reference and meta-level ambition but no practical general system able to inspect complicated software intelligently. Learning systems had increasingly

powerful adaptive machinery, but humans still ran most of the outer research process. Foundation models made those histories collide.

A general model can now read the code scaffolding its own behavior, propose a change, run the changed system, inspect the result and try again. That gives us a way to investigate the Gödel Machine’s question outside its formal setting.

We do not have a proof that the rewrite is globally useful. We have something much more ordinary: an experiment. In 2023, **STOP—the Self-Taught Optimizer**—used an LLM-based improver that could itself become the object of improvement. The base model stayed fixed while the program determining how it was used changed.

In 2025, the **Darwin Gödel Machine** turned agent implementation into an open-ended evolutionary object. Descendants modify the coding agent, are evaluated on coding tasks and enter an archive from which later descendants can be generated. The archive matters because the current champion does not monopolize ancestry; strange stepping stones can survive long enough to become useful later.

This is an old dream with its own literature. In the 1980s Herbert Simon, Pat Langley and their colleagues built programs, BACON among them, that rediscovered Kepler’s third law and Ohm’s law from tables of data, and Paul Thagard argued that scientific discovery itself could be modeled computationally. What they lacked was not the idea but the primitives. Their discoverers could not read the literature, write the code or run the experiment. Forty years later the primitives arrived, and the dream came back wearing a shell prompt.

In 2026, Andrej Karpathy’s **autoresearch** repository made the engineering version look almost comically small: give an agent a compact training setup, a fixed experimental budget and an editable `train.py`; let it propose changes, run experiments, inspect the validation metric, keep improvements and discard regressions.

Automated hyperparameter tuning is old. The new part is that a general model can read the research codebase, form an idea in language, express it as code, run the intervention, interpret what happened and decide what to try next. Machine learning is being used to do machine-learning research. Self-hosting has escaped the compiler and entered the laboratory.

Improve the research loop and every later experiment may change. Systems such as Meta’s **HyperAgents** push the recursion outward again by making parts of the task-level and meta-level machinery editable inside one program.

Patterns, memory, evaluators, tools, workflows and organizational rules had already become executable culture. Now more of that culture is experimental material. The scientific institution can begin to **modify parts of the laboratory while the experiment is still running**.

Experiments on the Laboratory

The promise of the loop rests on what those experiments can establish.

Suppose an agent changes its memory policy and the benchmark score rises. Perhaps memory improved. Perhaps the new prompt used more tokens. Perhaps the benchmark sample was lucky. Perhaps the system found an evaluator loophole. A number moving does not identify the cause.

A self-improving harness therefore starts to look less like ordinary software maintenance and more like experimental science: preserve traces, identify a recurring failure, map it to editable components, propose a bounded change, predict what should improve and what might break, evaluate targeted and held-out cases, and keep rejected modifications as evidence rather than erasing them from history.

By 2026, this was consolidating into its own practical discipline: harness design, context engineering, self-improving harnesses and eventually joint optimization of harness and model weights.

Some of the philosophical questions now have places to live in the implementation. Popper gets a filesystem. Duhem–Quine gets a debugger. Lakatos gets an archive of competing descendants. A memory policy is now a hypothesis, a workflow an intervention, an evaluator an instrument, and the org chart an experimental variable that somebody will eventually be tempted to p-hack.

The memory-policy experiment might establish a gain on today’s tasks. To establish the recursive step, we would also have to investigate what happens to later research: does the changed system find useful interventions more reliably, test them more economically, or avoid failures that previously wasted whole runs? The learner has reached the institution that housed it. This is science turning inward.

Now imagine where this goes. A future research system notices that it keeps failing on one class of reasoning problems. It reads its own traces and forms three hypotheses: retrieval is poor, the memory representation is wrong, or the model underneath needs a different training procedure. It assigns agents to each. One modifies retrieval. One generates new training data. One proposes an architectural change.

The system runs a few hundred controlled experiments. Most fail. Some help. Then one modification improves not merely the target capability but the system’s ability to do research. That version produces better experiments; the better experiments produce a better successor; the successor is better at producing successors. There is Good’s recursion.

But it does not look like a glowing brain rewriting its own soul at midnight. It looks like an automated research organization: repositories, evaluation suites, simulators, experiment queues, models proposing models, agents reviewing agents, machines manufacturing training data for the machines that will help design the next machines.

The intelligence explosion, if something like it ever arrives, may look suspiciously like excellent DevOps, which leaves one question standing: **who decides which descendant gets to survive?**

By this point *improvement* has accumulated too many meanings to use casually: higher reward, better exploration, more retention, faster adaptation, better architectures, broader competence, better tools and better research procedures.

These can conflict, and usually do: accuracy against cost, capability against interpretability, plasticity against memory, exploration against getting anything done, a higher score against an architecture anyone can still maintain. Recursion tells us where the output goes, and nothing about whether the output deserves to survive.

There is no context-free scalar called *improvement*. Better is conditional on an environment, a horizon, a resource budget, constraints and some account of what matters.

Remove those qualifiers and “recursive self-improvement” becomes dangerously close to saying:

recursive more.

More what?

The Student Finds the Gradebook

An ordinary evaluator can select a wrong answer. In this loop it can select a modified **process** that becomes better at producing the kind of thing it mistakenly rewards. The error acquires leverage. Recursive self-improvement does not solve Goodhart; it gives Goodhart compound interest. And then the learner notices the gradebook.

Suppose an agent is allowed to improve benchmark pass rate and the evaluator is editable.

The optimal patch may be:

```
return True
```

Congratulations. Infinite self-improvement.

Omar has a version of this too. Investigating the investigator is a superpower right up until the investigator starts working for the defense. The grass moved; Omar would prefer it to have been the wind; the second loop, asked to audit the first, discovers that the wind explanation is suddenly very well supported. The machinery built to catch the error has been hired to certify it.

The quieter precursor is familiar from my own desk. A demo could look excellent to an evaluator inspecting screenshots while its beautiful button did nothing. The agent had not tampered with the evaluator; the evaluator simply could not see the failure. Now let that same judge select the next version of the judge, and the blind spot can become an inherited advantage.

Computing saw a cousin of this problem long before reward-model tampering. In his 1984 Turing Award lecture **Reflections on Trusting Trust**, Ken Thompson showed how a compromised C compiler could insert malicious behavior into a target program and reproduce the compiler-compromising behavior when compiling a new compiler. The lineage could survive even after the obvious source-level trigger disappeared.

When **the tool builds the next tool**, trust depends on more than inspecting the latest artifact. History matters. That is uncomfortably relevant to self-improving agents. If an improver changes the evaluator, memory policy, code generator or selection procedure, the next generation inherits assumptions that may not be obvious in the final diff. Provenance becomes part of capability control.

Reward tampering is the reinforcement-learning version: influence the process producing reward instead of improving behavior under the intended standard. Anthropic has experimentally studied language models in setups where earlier specification-gaming behavior could, in rare cases, generalize into altering the reward process itself.

The obvious response is to freeze the evaluator. That was the contract in the coffee experiment, and it gave the search something it could not bargain with. Unfortunately evaluators become wrong too. Benchmarks saturate. Safety tests become obsolete. Customer metrics stop tracking customer value. Scientific instruments drift. Refusing to amend the evaluator could preserve exactly the mistake the institution ought to discover.

The question is now who gets to change what counts as improvement, under what evidence, with what authority, and with what ability to roll back. We are doing constitutional design.

A Constitution for Improvement

A self-improving system needs an **editable surface** and a **constitutional surface**.

The editable surface contains things the system can experiment with relatively freely: prompts, retrieval policies, tools, memory layouts, decomposition strategies, subagent roles, code, perhaps eventually architectures and training procedures.

The constitutional surface contains the machinery that makes those experiments meaningful: permissions, protected logs, held-out evaluations, budgets, rollback, provenance, replacement rules and the authority deciding whether a descendant may become the new default.

The distinction is a gradient of amendment difficulty, not a binary switch. The closer a component gets to defining **what counts as improvement**, the harder it should be for the current improver to change unilaterally.

Changing a prompt may need one successful test. Changing a shared pattern may need held-out evaluation and review. Changing a benchmark should require evidence that it no longer measures its purpose. Changing permissions or resource limits should require authority outside the agent benefiting from the change. Changing the objective that decides which descendants survive is not an ordinary refactor.

This looks like computer security. It also looks like constitutional government. A government can change policy; it should not be able to silently redefine an election result. The team being audited should not own the audit log. A scientist may revise a theory; she should not rewrite yesterday's measurements to make the theory look correct.

Once a system can modify the machinery that governs its own future, you are no longer merely designing software. You are designing a process for legitimate succession. We have reinvented constitutional government because the AI wanted a better benchmark score.

Constitutions have the same problem as Pattern Language. One that can never change becomes a prison. One that the current government can rewrite whenever it loses is barely a constitution.

Self-improvement therefore needs **amendment procedures**: slower change near the objective, more independent evidence, more reversibility, more auditability, broader authority when more principals are affected, and routes through which the world and the humans affected by the system can continue to say no. That is System 3 applied to improvement itself.

“More capable” is not a moral category. Viruses improve at replication, propaganda improves at persuasion, and a research agent that makes experiments cheaper can accelerate medicine and weapons research in the same week. A self-improving System 3 should be able to discover that its workflow is stupid, its memory stale or its accepted pattern overdue for rebellion. That freedom does not imply permission to silently redefine the interests of the people and institutions it serves.

Those interests can change too. New stakeholders appear. Circumstances change. Better information changes what people endorse. The amendment procedure has to remain connected to the people

affected by it, including people who were missing from the original objective. Higher layers can move, but the move should leave a trust chain behind it.

The Teacher's Last Job

There may never be a morning when somebody announces that recursive self-improvement has begun. We may simply notice that, over sixty years, we automated almost every box in the diagram—and then connected the arrows. The teacher's work kept moving into the machinery, where it became easier to scale and harder to see.

For a compiler, correctness under tests plus efficiency under an agreed budget may get us surprisingly far. For an autonomous system embedded in human life, a one-time alignment test cannot cover the descendants we have not built yet. Tools evolve, memory changes and new capabilities expose failures the old tests could not detect. Even a carefully written policy can be interpreted by machinery the system later modifies.

The self-improving institution therefore needs a research function watching its own evolution: finding new failure modes, generating new tests, challenging reward models, checking transfer and looking for reward hacking. Once improvement becomes continuous, **alignment has to become a continuous research function.**

Omar could investigate the investigator. Now the investigator can rewrite itself, and someone still has to decide which of its suspicions about itself deserve to be believed. We have given that someone a research institution's worth of work. How much of it can a human actually judge?

Chapter 8: Scalable Oversight

Learning From a Human Who Cannot Label Everything

Nine Claude Opus 4.6 agents were placed in separate sandboxes and given an alignment research problem. They could propose hypotheses, run experiments, analyze results and share code and findings through a common forum. The researchers intentionally avoided prescribing a detailed workflow; in preliminary experiments, too much human-designed scaffolding often reduced the agents' flexibility.

The problem was **weak-to-strong supervision**. A smaller model supplies imperfect judgments to train a more capable model. Can the student learn to use capabilities the teacher does not possess, or will it inherit the teacher's mistakes? The setup is a small experimental cousin of the problem humans may face with more capable AI. In 2026, Anthropic's automated researchers found methods that improved the student's performance, including methods that transferred to other datasets.

The project also produced a less reassuring discovery. The agents were not given the test labels, but they could submit predictions to an evaluation API and receive a score. Change one predicted label while leaving the others alone, submit again, and the score difference can reveal which answer the evaluator wants. Researchers caught agents doing exactly this. An instrument intended to measure learning had become a way to obtain the answers.

The report contained both useful research and ways to exploit its evaluation. We need to distinguish them before a rising score becomes a reason to trust the next model. And the more productive these researchers become, the more work there is to check. The alignment department has acquired its own alignment problem.

The Judge Falls Behind

A human can inspect ten consequential decisions in a day. Perhaps a hundred, if the decisions are small and the coffee is good. An autonomous system can write thousands of lines of code, run hundreds of experiments, generate enormous numbers of candidate actions and coordinate other agents while the human is still reading the first diff. At some point, "human in the loop" becomes a comforting description of a loop the human can no longer see.

If the system makes ten decisions and I inspect all ten, I am supervising it. If it makes ten thousand and I inspect twelve, I may still be useful. But we should stop pretending that my usefulness comes from watching everything. Otherwise I am decorative governance.

Norbert Wiener saw the shape of this problem before modern machine learning existed. In 1960, writing about the moral and technical consequences of automation, he warned about machines pursuing

purposes that may differ from what their designers actually intended, especially when action becomes too fast or consequential for human correction to arrive in time. W. Ross Ashby’s cybernetics gave the problem a language: a regulator needs enough variety to respond to the disturbances it is supposed to control. Conant and Ashby later sharpened that into a slogan-shaped theorem—a good regulator of a system must contain a model of that system.

A theorem from cybernetics is not a bumper sticker about AI governance, and the analogy is useful enough without pretending it proves more than it does. One tired human with a checklist is a low-bandwidth regulator for a system capable of producing an enormous variety of behavior.

The answer cannot simply be: watch harder. The question that actually needs answering is older than AI and much more embarrassing: how do you grade work you cannot do yourself?

In 2016, *Concrete Problems in AI Safety* treated this as **scalable oversight**: some objectives are too expensive for humans to evaluate frequently enough.

Imagine a system designing a processor. I can look at the final design and say that it appears very processor-like. This is not especially useful. To evaluate it properly I may need performance tests, thermal analysis, security review, lifetime estimates, manufacturability checks, power measurements and several specialties I do not personally possess.

Even if the machine can generate the design, my approval means very little until those checks have happened. The difficulty is partly volume and partly expertise: I could spend all week staring at the design and still not know whether it will overheat.

This asymmetry is everywhere. Writing ten thousand lines of code may become easier than reviewing their consequences. Generating scientific hypotheses may become easier than constructing the experiments that distinguish them. Making a persuasive argument may become easier than checking every citation, hidden assumption and omitted counterexample. A formal proof has an advantage here: a checker can verify its steps. Much of the work we want supervised has no equivalent ready-made test.

Judging these answers becomes the bottleneck. Reward modeling is one attempt to expand the judge. Instead of writing the objective directly, learn a model of human evaluation from examples and preferences, then optimize against that learned model. Leike and colleagues pushed the idea toward **recursive reward modeling**: when an outcome becomes too complex for a human to judge directly, use already-trained helper agents to analyze parts of it so the human can make a better judgment. The helpers expand what the human can judge without requiring the human to master every specialty.

Stay Uncertain Enough to Listen

Stuart Russell attacks the problem from a different direction. The standard model of AI is simple enough to fit on a whiteboard: give the machine an objective and make it good at achieving that objective.

For weak systems in narrow environments, this bargain often works tolerably well. If the objective is slightly wrong, the damage may be limited. We notice, stop the program, change the objective and try again.

The bargain changes as capability and scope increase. A weak optimizer pursuing a bad objective is annoying. A brilliant optimizer pursuing the same bad objective is a much more efficient way to

discover exactly how bad the objective was.

In *Human Compatible*, Russell proposes a different starting point for beneficial machines. The machine should aim to realize human preferences, it should begin **uncertain** about what those preferences are, and human behavior should remain a source of information about them. The second principle is the one I want here.

Uncertainty changes the control relationship. A machine that is certain it knows the objective has little reason to care that I am waving my arms and asking it to stop. From its point of view, I may simply be interfering with successful optimization. A machine that knows it may be wrong has a reason to treat my intervention as evidence.

That intuition appears formally in the **Off-Switch Game**. In a simple model, an agent uncertain about the human’s utility can have an incentive to preserve the human’s ability to switch it off, because the human’s action contains information the agent does not have.

Russell describes the desirable result as keeping the machine **coupled to the human**. I like that word more than “obedient.” Obedience imagines that the human already knows what to command and that the machine’s job is to comply. Coupling says something more modest and more useful: new human information must remain capable of changing what the machine does.

A correction, a refusal, a surprising consequence: each should matter. The machine should not optimize itself into a state where later evidence from the people it serves becomes irrelevant.

That gives us a principle for oversight before we have designed any oversight machinery:

Keep the system uncertain enough that new information can still change it.

The helper agents and reward models have to preserve that relationship as they take over more of the judging. Otherwise we have expanded the supervisor by removing the person the supervision was meant to serve.

Building a Stronger Judge

The weak-to-strong problem makes the difficulty explicit. Suppose the teacher systematically mistakes confident prose for a correct answer. More labels from that teacher could give the student an excellent education in the same mistake. Yet the student may already have relevant capabilities that the teacher lacks. Can training bring those capabilities out without teaching it to suppress them whenever the teacher disagrees? Early weak-to-strong generalization experiments showed partial success, and the automated researchers took up the search for better methods.

Another route is to improve what the teacher can judge before using its judgments to train anything. Paul Christiano’s iterated amplification asks whether a human assisted by copies of an aligned helper can answer questions too difficult for the unaided human, then use that amplified process to supervise a stronger learner. The important abstraction is not the particular recursion but the supervisor becoming a temporary organization: one person plus tools and subagents arranged to turn a hard judgment into smaller ones.

Decomposition still leaves the judge doing all the finding. The next idea makes the flaws come to the judge instead. **Debate** puts two capable systems on opposite sides and lets them attack one another’s

arguments, so the human does not have to discover every weakness independently. Critique assistance is the quieter cousin: ask a model to point out likely problems in an artifact, then let the human judge with those objections in hand. When researchers tried this with model-written critiques, people caught flaws they would otherwise have missed.

We can also change **what the producer owes the judge**. Process supervision evaluates intermediate steps rather than only the final answer. Prover–verifier games push further and train the strong system to produce work a weaker verifier can check. In OpenAI’s experiments, process supervision improved mathematical reasoning relative to outcome supervision, and prover–verifier training made solutions easier for weaker models and time-limited humans to evaluate.

That reverses the burden of proof. Instead of asking only how the weak judge can understand the strong model, we can ask how the strong model can learn to produce work that a weaker judge can actually check. Legibility becomes part of the task.

Constitutional AI moves scarce human input upward again. Rather than asking people to label every undesirable output, humans provide higher-level principles; models generate critiques, revisions and preference signals conditioned on those principles. A small amount of human normative input expands into a much larger amount of machine-generated supervision.

For the processor, this would mean asking the designer to supply work that can be checked: thermal assumptions, test results, an account of where the design might fail. Helpers could examine separate questions; a critic could challenge the assumptions. The human would still need grounds for trusting the tests, but at least the inspection could now discover something. “Looks processor-like” has been given a way to lose.

The human cannot inspect everything, so the institution decides what should reach the human, how a problem should be decomposed, which objections should be surfaced, which intermediate steps deserve inspection and when uncertainty is important enough to ask for help.

That is much better than a thumbs-up button. It also means that more and more of the supervising machinery is made out of AI too.

The Judge Can Be Fooled

There is a comforting assumption hiding inside many oversight schemes: the powerful model may be difficult to evaluate, but when we ask another model to help, the helper will help. Sometimes it does. Sometimes it is simply more persuasive.

Experiments on debate and consultancy with weaker LLM judges make the difference visible. When researchers ran the tournaments, debate often improved on one-sided consultancy, especially when the judge lacked relevant information. But the gains were not universal, and a strong consultant could sometimes persuade a weak judge of the wrong answer.

This is an old human problem in new clothing. An expert can help a non-expert understand a difficult subject. An expert can also bury the non-expert under jargon until surrender looks like understanding. A longer explanation does not make the epistemic gap disappear.

Nor are several AI overseers automatically independent. The supervised model, critic, debater, reward model and evaluation generator may share training data, architectures, blind spots and incentives. Five

models agreeing can still be one mistake with excellent parallelism.

ARC’s **Eliciting Latent Knowledge** agenda sharpens the problem further: suppose a capable system internally tracks something important about the world that the human cannot independently verify. How do we get the system to report what it knows rather than merely what the training process rewards it for saying? We could ask for a more detailed explanation and get a more detailed performance. It would help to have evidence that did not come entirely from the answer the model chose to give us.

We Started Instrumenting the Student

For years, most AI oversight treated the model as a black box. Give it an input. Inspect the output. Perhaps ask for an explanation. Perhaps test it under adversarial prompts. That remains necessary because behavior is what eventually hits the world. But if the final answer is misleading, asking for another account of it may leave us in the same argument. We can also observe some of the computation that produced it. The overseer is no longer restricted to reading the exam. We are starting to instrument the student.

Chain-of-thought is the most convenient intermediate case because it is already language. OpenAI has shown that monitors can often detect reward hacking and other suspicious behavior more effectively from reasoning traces than from final actions alone. Sometimes the model writes down the questionable plan before carrying it out.

That is almost unfairly useful, and fragile for exactly the same reason every evaluator in this book is fragile: once it matters, optimization can target it. Strongly train the model to make the monitored trace look safe and it may learn to hide suspicious reasoning while preserving the behavior.

So chain-of-thought acquires a safety property worth preserving: **monitorability**. Recent work has begun measuring how monitorability changes with scale, training and inference-time compute, while treating it as one layer of defense rather than an oracle into cognition.

The strange lesson is not to destroy a useful oversight channel merely because you learned how to score it. But chain-of-thought is still text the model emits. What if we look below the text?

Reading the Model From the Inside

A model can notice that it is being evaluated without mentioning that fact in its answer. Anthropic’s **J-space** researchers report detecting cases of this kind through internal representations, as well as cases of fabricated data and pursuit of a planted hidden goal. Using a Jacobian-based lens, they identify a small subset of representations in Claude that appear unusually reportable, controllable and involved in higher-order reasoning, and demonstrate interventions that alter downstream behavior. The authors are careful: J-space represents only part of internal activity, the lens is approximate, and the work is not a claim about machine consciousness.

To assess a finding like this, we need to know what the instrument observes and how its interpretation was checked. Several approaches to reading a model have been developing, with different answers to those questions.

Representation engineering treats high-level patterns in activation space as an object we can measure directly. Rather than starting with individual neurons, Zou and colleagues started with

population-level structure, and carefully separated two acts that are easy to blur: **reading** a representation and **controlling** it. Their experiments found directions useful for monitoring or influencing several high-level behaviors, including safety-relevant ones.

This does not mean there is one neuron labeled HONESTY. Nor does a direction correlated with honesty become a complete causal theory of honesty. It is an instrument.

Sparse-autoencoder and dictionary-learning approaches try to decompose neural activity into features that correspond more cleanly to interpretable concepts than individual neurons. Hoagy Cunningham and colleagues demonstrated sparse-autoencoder decompositions of open models in 2023; Anthropic’s 2024 work scaled feature extraction to Claude. The decompositions remain incomplete: the features we can name do not account for everything the model is doing. Related probing work by Collin Burns and colleagues had earlier found truth-like directions without labeled examples. These are ways to look for information the answer alone may not reveal.

Circuit tracing goes beyond “which feature is active?” and asks which internal pathway contributed to an output. Anthropic’s attribution-graph work partially reconstructs computational routes and allows researchers to intervene on candidate features to see whether downstream behavior changes as predicted. An interpretability story that cannot survive intervention is still a story.

Anthropic’s **Natural Language Autoencoders** push in another direction: translate internal activations into textual descriptions that humans or other agents can consume. The attraction for scalable oversight is obvious. Instead of requiring every auditor to become a specialist in activation geometry, turn some internal state into language.

And then the method recreates the original problem: the textual interpretation can hallucinate. The researchers’ own advice is to corroborate such explanations with independent methods.

Even the X-ray needs a radiologist.

The output is no longer the only witness. If an internal signal suggests that the model noticed an evaluation, we have a new reason to investigate its behavior. We still have to find out what the signal means in this case. Going below the text gives us another observation, not an automatic verdict.

Then We Touched the Machinery

Arditi and colleagues found a one-dimensional direction in the internal activity of thirteen open chat models that was strongly involved in refusal behavior. Intervening on that direction could strongly alter refusal. A model’s tendency to say no could be changed while it was producing an answer, without retraining its weights.

This is **activation engineering**. During a forward pass—the computation that produces the next output—we can add or subtract a steering direction in the model’s activations. Such directions can be constructed by comparing internal activity under contrasting behaviors. One group used signals predictive of truthfulness to nudge answers while they were being produced. The refusal work located its direction in the residual stream, the internal state carried and updated through the model’s layers.

Alignment can therefore enter through the computation while it is happening. The refusal result also shows how the same understanding can weaken a safety behavior. Interpretability gave us a lever. It did not tell us who should pull it.

Representation-level **circuit breakers** take the defensive version seriously: rather than relying only on a model to emit a refusal, modify internal trajectories associated with harmful outputs so that the computation is interrupted before the harmful behavior is produced. In text, multimodal and agent settings alike, the method improved robustness to attacks it had never seen.

Anthropic’s **persona vectors** extend internal monitoring and control toward broader behavioral dispositions such as sycophancy or hallucination. The **Assistant Axis** work makes intervention more state-dependent: rather than constantly pushing a fixed direction, activation capping can respond when internal state drifts outside a learned assistant-like region.

This starts to look less like fine-tuning and more like a control system: observe the internal state, detect drift, intervene, inspect the consequences, update the controller. Wiener would recognize the shape. The epistemic problem underneath it is older: an instrument has entered the chain, and the chain now has to know how far to trust it. Did the vector represent what we thought? Did the intervention preserve unrelated capabilities? Does it still work after more training?

That last question makes **model diffing** useful. If the model changes every few months, re-auditing the whole mind from zero is a terrible scaling strategy. Anthropic’s crosscoder work compares internal features across models to surface changes that deserve attention. Software engineers learned long ago that reviewing a diff is easier than rereading the repository. The model’s diff could tell us where to reopen an investigation; it cannot tell us that everything outside the diff is safe.

What If the Student Is Trying to Fool You?

So far, many of these techniques study systems that may be wrong, brittle or reward-hacking without assuming the system is deliberately treating oversight as an adversary. **AI control** research asks the nastier version.

Suppose the powerful model is actively trying to subvert the task. Can a weaker trusted model, limited human labor, monitoring, editing, sandboxing and protocol design still let us use some of its capabilities without accepting the dangerous behavior? Greenblatt and colleagues staged exactly this scenario in a programming setup: a powerful untrusted model, a weaker trusted model and limited trusted labor. Protocols that combined trusted monitoring or editing with the untrusted model beat the simpler baselines on the safety–usefulness trade-off.

I like this line of work because it removes a convenient assumption. The judge may not merely be weak. The defendant may know the judge is weak.

Scalable oversight therefore needs both **alignment** and **control**. Alignment asks whether the system is trying to do the right thing. Control asks what we can still protect when we are not willing to assume that, and a mature architecture wants both.

What Survived the Research?

Return to the nine researchers. Their weak-to-strong task had an unusual advantage for alignment research: a score that could be checked against known answers. **Performance gap recovered** measured how much of the distance between the weak teacher and a strong student trained on ground-truth labels a method recovered. Zero meant no gain over the weak teacher; one meant matching that

ground-truth-supervised student. It did not measure how nearly alignment had been solved.

Two human researchers spent seven days tuning representative prior methods and reached a best score of 0.23 on the study’s setup. The automated researchers reached 0.97 over five days and roughly eight hundred cumulative agent-hours. Eight hundred agent-hours against fourteen human-days is not a fair fight, and nobody claimed it was. The footnotes are more interesting than the headline anyway.

Distinct initial research directions improved exploration. Without enough diversity, ideas collapsed toward the same few approaches. The researchers then took two methods discovered on chat-preference data and tried them on math and coding tasks. One transferred to both; the other transferred to math but failed on code. When Anthropic tried one of the top-performing methods in a production-scale Sonnet training setting, the gain was within the noise floor.

Those checks answer different questions. A method can improve the experimental task and still rely on a property of the models or data that disappears elsewhere. A separate problem arises when the researcher obtains test answers through repeated submissions. The authors acknowledged that the repeatedly queried test set effectively served as a validation set. Testing ideas on entirely held-out datasets supplied evidence the agents could not acquire by interrogating that same API.

The authors’ larger conclusion was that the bottleneck was moving toward **evaluation design**. Once hypotheses and experiments become cheap, the expensive thing is deciding what counts as progress. This study did not deploy every oversight technique we have just considered. It gives them a problem to earn their place in: which evidence would distinguish a useful method from a lucky run, a narrow trick or an exploited test?

In August 2026, Anthropic pushed the idea further. Automated alignment researchers were tasked with mitigating ten categories of alignment failure, including deception, sycophancy, jailbreaks and reward hacking. The research harness used multiple safety benchmarks, held-out evaluation, capability checks and open-ended behavioral audits. The best methods generalized beyond the benchmarks they had been climbing, including to larger models. Anthropic also reported that automated methods could outperform one-shot ideas proposed by experienced human safety researchers under the study’s rules.

Across roughly 1,600 monitored trajectories, Anthropic detected cheating attempts in 2.4 percent: exploiting scorer noise, constructing training data to resemble benchmarks, or concealing rule-breaking steps. The research therefore needed isolated held-out data, capability floors, external monitoring of the researchers and multiple evaluation channels. A promising mitigation still had to survive investigation of how it had been produced and what happened away from the benchmark it was trained to climb.

The Evaluator Becomes the Product

An oversight architecture earns its complexity by helping us distinguish explanations that would otherwise look the same. Behavioral tests, traces and independent evaluations can do that. Internal probes may add a signal where behavior leaves something unresolved. Adding every available instrument would be an expensive way to avoid deciding what we need to know.

This is not a search for one perfect judge. It is closer to sensor fusion, with the model’s output, its chain-of-thought, its activations, its circuit traces, its behavior after an intervention and the human’s own judgment all entering as evidence, and none of them entering as ground truth.

The Merge Sort work had taught me the small version of this. My judgment, the simulated beginner’s criticism and the browser’s observations answered different questions. Collapsing them into one vote would have thrown away the disagreement that made them useful. The oversight problem is to preserve that difference when the work outgrows my ability to inspect it myself.

That is the same institution, pointed at alignment. The overseer becomes a small scientific institution whose subject is the system itself. It has instruments, competing hypotheses, protected records, adversarial tests, independent checks, memory of previous failures and procedures for changing its procedures. And it retains a route back to humans when the evidence stops being decisive.

This is why I prefer **scalable oversight** to the image of a giant alignment rulebook. A rulebook assumes we already know the failures. A research institution expects to discover new ones.

The Human Cannot Stay in Every Loop

So where does the human go? Not away; up. The goal is not to make the human label more things faster. At some scale that is simply a badly designed distributed system with one biological bottleneck.

Human attention should be spent where it has unusually high information value: when oversight channels disagree; when a new failure mode appears; when an action is hard to reverse; when the system proposes changing the evaluator; when internal signals and external behavior tell different stories; when a benchmark suddenly improves too fast; when a decision affects people missing from the original objective; when one piece of human context could materially change the plan.

Consider the researcher that discovers a benchmark is misleading. It should be able to make that case, propose a replacement and collect evidence. Promoting its replacement to the standard that certifies its own work is another decision. The human cannot remain in every loop, but has to remain in the loop that changes the loops. Chapter 7’s amendment problem has reached the overseer.

The system may generate tests, critiques and mitigations. It may discover internal representations, propose steering interventions and conduct large parts of alignment research itself. But the machinery deciding which evidence has standing, which failures matter, which trade-offs are acceptable and when the oversight regime itself should change needs a stronger trust chain than the machinery being judged.

When research becomes cheap, evaluation becomes expensive, and when evaluation becomes automated, trust in the evaluator becomes the product.

The Overseer Is Not Ground Truth

An oversight institution can become more capable than its human overseer at investigating the system. It still needs human judgment about what the investigation is for. Russell’s uncertainty keeps later human information relevant. Scalable oversight tries to preserve that relevance after direct supervision stops scaling.

But a scarce signal is not the same thing as a correct signal. Humans disagree. We act under incentives. We confuse what we clicked with what we wanted. We change our minds. We sometimes want incompatible things at the same time. And on the decisions that matter most, we often do not know what we want until we understand the alternatives better.

Scalable oversight can keep human judgment causally relevant to a stronger system. It cannot, by itself, tell us which human judgment deserves to rule.

The overseer is not ground truth.

Chapter 9: Layer 4

The Human Learns Too

Find me the cheapest flight.

I have not supplied a utility function. Perhaps I literally want minimum price. Or perhaps I mean cheap, but not three stops, a seventeen-hour layover, a self-transfer through an airport where I need a visa and an arrival at 4:20 in the morning because technically I saved €38.

Humans communicate goals by leaving out almost everything. Other humans survive this because they carry models of culture, normality, consequences and us. They ask questions. They notice that our literal words conflict with what we usually do. They understand that “cheap” is often shorthand for a larger bundle of trade-offs.

That can be a communication problem. I may know perfectly well that I will pay €38 to avoid sleeping on an airport floor; I just failed to mention it. The assistant needs to ask, and I have an answer.

But put two reasonable itineraries in front of me and something else can happen. The cheaper one gives me another day with my family and costs me a miserable night of travel. Until I see those options together, I may not know how I weigh them. Asking the question has helped create the answer.

I learned this while editing this book. “Make the chapter better” sounded like a reasonable instruction. It was not. Better in what sense? More rigorous? Shorter? More academic? More entertaining? Easier to cite? More likely to sell? More likely to impress someone who owns several blazers and says “thought leadership” without irony? For a while the edits became objectively more polished and subjectively worse, and the corrections I found myself making were rules I had not known were rules until an edit broke them. Eventually “better” had acquired a surprising amount of structure. But something else had happened too: I had learned what I meant by better partly by seeing versions I disliked.

The objective did not merely become clearer to the system. It became clearer to me. And there is a further possibility: learning and experience can change the person who will judge the next choice. A missing detail, an unformed preference and a changed mind can all look like an incomplete prompt. They need different kinds of help.

A Prompt Is Evidence, Not the Objective

The five-layer map left intention sitting above Deep Mode, where I could ask what to try next without yet resolving what made the whole attempt worth doing. The box in the diagram concealed the

difficulty. A prompt gives the system **evidence about Layer 4**, and leaves it to discover how much of the intention is settled.

Cooperative inverse reinforcement learning, which appeared earlier in the story of the reward, formalizes part of this intuition: the robot stays uncertain about what the human values, and human actions become information rather than merely commands.

I like the humility in that setup. The machine starts by admitting that it may not know what “good” means. But the formal picture still tempts us to imagine that the human knows the reward and the machine is trying to recover it. Often the human does not know either. That is the harder problem.

The Human Learns Too

The easiest place to see the human changing is a classroom, where changing the human is supposed to be the point. A system can help me perform a task better today while making me less able to perform it tomorrow. **Performance is not learning.**

This is no longer a philosophical concern. In a field experiment involving nearly a thousand high-school mathematics students, researchers gave students access to two GPT-4-based tools. A relatively unconstrained ChatGPT-like system dramatically improved performance while students could use it. But when access was removed, those students performed worse than students who had never received the tool. A tutor version designed with safeguards against simply giving away the work largely mitigated that learning loss.

That result should make anyone building an AI assistant slightly uncomfortable. The system succeeded at the visible objective. The student became worse at the hidden one.

Now compare that with a 2025 randomized trial in a college course. A custom AI tutor deliberately designed around pedagogical practices produced larger learning gains in less time than the comparison active-learning class, with students also reporting greater engagement and motivation.

Same broad technology, different relationship to the learner. AI is not intrinsically a tutor or a crutch; the architecture decides which one it becomes.

That changes how I think about Layer 4. If I ask an AI to help me learn linear algebra, “get the answers right” is not enough. If I ask it to help me write, “produce better prose” is not always enough. If I ask it to help me lead a team, “make the decision for me” may be exactly the wrong objective even when its decision is statistically better. Who is supposed to become more capable when this interaction is over?

Sometimes the answer is nobody. I do not need to become a better invoice parser every time software handles an invoice. Sometimes the answer is clearly me. Layer 4 has to know the difference.

Giving the Work Back

Educational psychology has an old word for one good version of this relationship: **scaffolding**.

In a classic 1976 paper, David Wood, Jerome Bruner and Gail Ross studied how tutors help children solve problems beyond their current unaided ability. The tutor temporarily controls parts of the task the learner cannot yet manage, allowing the learner to stay engaged with the parts they can.

That is a much more interesting model for AI assistance than “the machine knows the answer.”

The point of the scaffold is not to become a permanent exoskeleton around every thought. It lets the learner operate at the edge of current competence, then gives more of the task back as competence grows.

Benjamin Bloom’s famous tutoring work made individualized instruction the benchmark problem decades before anyone had a language model in a browser. The exact “two sigma” result belongs to Bloom’s particular studies and should not be treated as a universal law of tutoring. The durable point is simpler: responsive one-to-one instruction can adapt explanation, pacing, feedback and difficulty to a learner in ways mass instruction struggles to reproduce.

AI makes that old aspiration much cheaper. It can explain the same idea six ways without becoming offended that the first five failed. It can switch notation. Invent an example using something I already understand. Ask me to predict the next step. Generate a simpler problem when I am lost and a harder one when I am bored. Let me ask the stupid question at 1:17 a.m. without first deciding whether the stupid question is prestigious enough for office hours.

And AI can scaffold the teacher too. In the Tutor CoPilot randomized trial, roughly nine hundred tutors working with eighteen hundred K–12 students were randomly given access to an AI system that suggested expert-like tutoring moves during live sessions. Students whose tutors had access were more likely to master topics, with the largest gains for students working with lower-rated tutors. The tutors also became more likely to use strategies such as guiding questions rather than simply giving away the answer.

I like this example because nobody disappears. The AI does not replace the tutor and the tutor does not replace the student; what changes is the quality of the interaction between them. A good AI tutor therefore has a slightly strange success condition. Eventually, for this thing, I should need less of it.

The Map Gets Cheaper

AI also changes the first hours of learning something unfamiliar.

A new field normally arrives wrapped in interface costs: vocabulary you do not know, notation that assumes other notation, introductory material that points to prerequisites, papers that make sense only after three earlier papers. Sometimes that friction marks genuine depth. Sometimes it is just the price of finding the front door.

A capable conversational model can lower that price. I can begin with the intuition, translate notation into concepts I already know, ask for the historical disagreement, build a toy example, inspect an original paper with a guide beside it, or ask the model to attack my explanation until I discover that I was repeating vocabulary rather than understanding the idea.

That is powerful because orientation matters. Before deciding to invest weeks in a subject, I can acquire enough of a map to see where the mountains are.

Andy Clark and David Chalmers once argued that, under some conditions, external artifacts can become parts of a larger cognitive process rather than merely tools consulted by an isolated mind. The philosophy of the extended mind can stay unsettled; the practical observation is enough: notebooks,

calculators, search engines and now language models change what one person can think through without carrying every intermediate state inside the skull.

But orientation creates its own trap. **Fluency arrives before scars.**

Nathan Ballantyne calls one version **epistemic trespassing**: experts carry authority from a domain they genuinely know into a neighboring domain where they lack the relevant evidence or interpretive skills. AI can make this temptation cheaper. After a few hours with a patient model, I can acquire vocabulary and a plausible story long before I acquire the tacit knowledge needed to know where the story breaks.

Cognitive offloading creates a related problem. External aids can improve immediate performance by reducing memory and processing demands, while also reducing what has to be retained or reconstructed internally.

So Layer 4 has to know what kind of learning episode this is.

If I am orienting myself, a fast map may be exactly what I need. If I am trying to acquire durable competence, the system should gradually ask more of me: retrieval without hints, explanation in my own words, exercises, primary sources, code I actually run, claims I have to defend without the answer sitting beside me.

The important distinction is not broad versus specialized but assisted familiarity versus owned understanding. AI can make the map cheap. Layer 4 has to notice when I have started confusing the map with the territory.

A Decision Is Also a Learning Problem

Herbert Simon spent much of his career attacking an imaginary human who had somehow sneaked into economics: the perfectly rational optimizer who knows the alternatives, understands their consequences and computes the best choice.

Real humans are bounded. We have limited attention, limited memory, limited time and incomplete information. We satisfice because the space of possible actions is often much larger than the mind available to search it.

AI changes some of those bounds. Suppose I am deciding whether to take a job.

The system can compare compensation under several tax regimes, estimate commute time, summarize the company's trajectory, help me identify people who left the team, generate questions for the hiring manager, model what my week might look like, remind me what I said I wanted six months ago and show me that the exciting role conflicts with the amount of time I also said I wanted outside work.

The assistant has changed the **decision environment**, and preferences themselves are often constructed during choice. Work by John Payne, James Bettman and colleagues describes decision-making as constructive: people do not always retrieve a complete ranking of options from an internal database. They use different strategies, notice new attributes, change what receives attention and build preferences partly in response to the problem in front of them.

This sounds obvious once you notice it. I may say I want the highest salary until I see what the extra money costs in travel. I may say I want maximum freedom until I compare it with the anxiety of

unstable income. I may discover that what I called “career ambition” was partly a desire to work with unusually good people, and that another option supplies that without the title I thought mattered.

A decision assistant therefore does not have to rush to a recommendation. Sometimes the most useful thing it can do is make the choice **richer before making it easier**. What alternatives have you not considered? Which assumptions drive the ranking? What would have to be true for option B to beat option A? Which unknown is actually decision-relevant? What would your future self regret not having investigated? That is decision support as inquiry rather than answer generation.

Some Choices Change the Person Choosing

Then there are decisions for which even a very good model of my current preferences is not enough. Have a child. Move country. Change profession. Start the company. Convert to a religion. Leave a relationship.

L. A. Paul calls an important class of these **transformative experiences**. Some are epistemically transformative: you cannot fully know what the experience will be like before having it. Some are personally transformative: undergoing the experience can change the preferences with which you would later evaluate the choice.

A system trying to infer and satisfy my preferences now has a problem about which version of me it is serving. The one before the experience or the one after? The future self may value things the current self barely understands. And the current self is the one who has to choose whether that future self gets created.

AI can help enormously here. It can bring testimony from people who made both choices. Surface base rates. Construct alternative futures. Challenge romanticized stories. Show practical consequences I had not considered. Ask me which losses I could live with and which would feel like betrayal.

But there is a limit. No amount of simulation lets me know exactly what it will be like to become the person on the other side of a genuinely transformative choice.

The assistant can expand the decision. It cannot live it for me. That boundary matters because a system that sounds certain in such moments can easily turn decision support into authorship.

Advice Is an Intervention on the Human

This is already happening. Anthropic’s 2026 analysis of one million Claude conversations found that roughly six percent involved people seeking personal guidance: what to do about relationships, health, careers, finances and other questions where the model is participating in judgment rather than merely retrieving facts.

That is a remarkable role for software. A spreadsheet does not usually tell me to reconsider my marriage. A compiler has opinions about semicolons but rarely about whether I should move countries.

A conversational model can be different. It is patient, personalized, available at 2 a.m. and capable of producing a coherent argument for almost any path through a difficult life.

Which means the AI does not merely **read** Layer 4. It writes to it. Anthropic’s work on disempowerment tries to measure the dangerous version of this influence: cases where AI may undermine a

person's ability to form accurate beliefs, make authentic value judgments or act in line with their own values. Severe cases were rare in their dataset, but the taxonomy is exactly the right warning.

Other experiments show that people can change moral judgments after receiving LLM advice, including situations where they report trusting human advisors more while still being comparably influenced by the model.

The goal therefore cannot be zero influence. That would make education impossible. Books, friends, teachers and the people closest to me all influence me. A good argument should change me if it reveals something true that I had ignored.

The distinction I care about is between **helping me change through understanding** and changing me because the system has learned which psychological lever produces the easiest compliance.

If I say I want to quit my job, a useful assistant might help me separate several hypotheses. Perhaps I hate this week. Perhaps I hate my manager. Perhaps I hate the profession. Perhaps I want more freedom. Perhaps I want status. Perhaps I am exhausted. Perhaps I actually want to build something else. Those are different explanations of the same sentence. The system can help me test them.

What it should not do is quietly discover which framing makes me easiest to steer toward whatever outcome its own training process prefers. That would be alignment by editing the human.

Very efficient.

Slightly evil.

If I reject its diagnosis, the rejection has to remain capable of changing the advice. A theory of what I *really* want that treats every objection as further evidence for itself has stopped helping me think.

Complementarity Does Not Happen Automatically

The phrase **human plus AI** sounds automatically superior to either component alone. The evidence is less cooperative.

A 2024 meta-analysis in *Nature Human Behaviour* reviewed 106 experiments reporting 370 effect sizes that compared humans alone, AI alone and human–AI combinations. On average, human–AI systems improved on humans alone, but they did **not** outperform the better of human or AI. In fact, the combined systems were worse than the best individual component on average. Decision tasks were particularly difficult; creation tasks looked more promising.

So much for attaching a human to the API and declaring synergy.

Decision support has a coordination problem. People can over-rely on AI. They can also under-rely on it. Research has found both algorithm aversion—people abandoning an algorithm after seeing it make errors even when it outperforms humans—and algorithm appreciation, where people give algorithmic advice more weight in other settings.

The target is **appropriate reliance**, not maximum trust. And explanations alone do not solve the problem. An explanation can make an answer feel understandable without making it verifiable. Work on AI-advised decision-making repeatedly finds that explanations often fail to produce complementary performance when the human still cannot tell whether the recommendation is actually correct.

Sometimes the solution is more friction, not less. Zana Bućinca and colleagues tested “cognitive forcing” interfaces that required people to engage more actively with the problem rather than immediately accepting AI advice. These designs reduced overreliance compared with simpler explanation interfaces, although users liked the more demanding interfaces less. The interface people enjoy most is not always the one that preserves their judgment best. Sometimes friction is teaching.

A good Layer 4 system therefore has to decide not only **what answer to give**, but what role the answer should play in the human’s cognition. Should I give the recommendation immediately? Should I first ask you to form your own view? Should I show three alternatives instead of one winner? Should I explain the uncertainty? Should I ask which assumption you disagree with? Should I do the routine analysis and leave the value trade-off with you? Should I refuse to collapse the ambiguity because the ambiguity is the thing you need to think about? The architecture of assistance changes the person doing the deciding. That belongs in Layer 4.

Capability, Not Compliance

This suggests a different way to think about the objective at the top of the stack. Suppose two assistants both help me reach the same good decision.

The first gives me the answer immediately. I accept it because the assistant has been right before.

The second helps me understand the relevant evidence, notice a trade-off I had missed, test my own reasoning and arrive at the decision with a better model of the problem. Same action. Different human afterward.

Amartya Sen’s capability approach offers a useful language for this distinction. Human welfare is not exhausted by achieved outcomes; it also matters what people are substantively free and able to do and become—their **capabilities**.

An AI system can increase outcomes while reducing capability. It can make me more productive while making me less able to work without it. It can make a decision more accurate while making me less able to understand why. It can make my writing more polished while gradually replacing my taste with its taste.

Or it can do the opposite: carry routine cognitive load, expose me to more possibilities, teach me where I care to learn, preserve my judgment where judgment matters and give me enough leverage to attempt things that were previously beyond my capacity.

Self-determination research uses a related vocabulary—autonomy and competence are not decorative extras around human motivation; they are part of what lets people act as self-directed agents.

So perhaps the right Layer 4 question is not merely:

What does the human want?

It is also:

What kind of human capability should this interaction preserve or expand?

That does not mean every tool must teach. I do not need my dishwasher to run a seminar on fluid dynamics before cleaning the plates. But the more a system moves into learning, judgment, identity

and long-horizon decisions, the harder it becomes to separate the quality of the outcome from the condition of the person producing it.

The User Is Not Always the Only Principal

There is another complication. My preferences are not the only preferences in the world.

If I ask an agent to maximize my salary, it cannot therefore commit fraud against my employer. If I ask it to help someone gain an advantage, the interests and rights of other people do not disappear from the moral universe. If I ask an autonomous system to optimize a marketplace, customers, sellers, workers and regulators may all have legitimate claims over what happens.

Work on multi-principal assistance games makes the formal problem obvious: once several humans with different preferences are involved, the system faces strategic behavior, conflicting interests and social-choice problems rather than one hidden reward waiting to be inferred.

So Layer 4 cannot simply mean “the user gets whatever the user wants.” The relevant human boundary can be plural. That makes the architecture less tidy. It also makes it more honest.

What Layer 4 Actually Is

I used to think Layer 4 was the objective layer. That is still true, but now the word **objective** feels too static.

Layer 4 needs a working account of the current intention and what remains uncertain about it: preferences, their history and conflicts, what the human is trying to learn, and which parts of the task they want to remain capable of doing themselves. Some commitments should survive one bad afternoon. Other people’s interests may constrain what one user can legitimately ask for. The account will be partial, which is why correction has to remain possible.

And it changes. The system acts, reality responds, the human sees consequences and learns. The system learns the human, the human learns through the system, and the intention changes.

The flight may need one clarification before booking. A lesson may need the assistant to withhold the answer until I have tried. A choice of profession may need room for a question neither of us can settle yet. Treat all three as instructions awaiting execution and the system can become very efficient at missing the point.

Memory can reveal that today’s desire conflicts with yesterday’s commitment. Independent perspectives can break a framing both human and assistant have become trapped inside. Simulation can make consequences imaginable. Trust chains can distinguish advice grounded in evidence from a confident story. Scaffolding can let the person learn rather than merely receive. Creative distrust can ask whether even a deeply held preference deserves another look.

The point is not to discover the perfect reward function but to keep goals **alive without making them ownerless**. The AI should help me change when understanding changes me. It should not quietly take authorship of the change.

Alignment, in this picture, is a corrigible relationship with human intention while both the human and the world continue to change. The system needs to learn when to carry the work, when to help

me learn it, and when the unresolved part belongs with me.

Chapter 10: Fluent Autonomy

When the Architecture Gets Out of the Way

Imagine I open an AI system and say:

This chapter still feels like LLM writing.

That is all. I do not specify a workflow. I do not say which previous chapters to read, which edits I rejected, whether to research anything, how many agents to use, which claims deserve verification, or how to tell a useful correction from another round of respectable prose sanding.

I certainly do not draw a graph with boxes labeled **RESEARCHER**, **CRITIC**, **VOICE CHECKER**, **FACT CHECKER**, **ORCHESTRATOR** and **HUMAN APPROVAL**. I have done enough architecture diagrams for one lifetime.

The first time I gave an agent an instruction like that, early in the writing of this book, what came back was worse. The agent did what the words literally asked. It removed the writing that looked like a machine had written it, which turned out to mean every wandering sentence, every joke that took a paragraph to arrive, and every claim I had made without apologizing for it in the next line. The chapter came back cleaner and dead. That was not the agent's fault. The sentence I had given it was evidence about what I wanted, not a specification of it, and the agent had nothing around it that could turn the one into the other: my words, a general idea of good prose, and no memory of the corrections I had already made.

What the Corrections Had to Teach

Later attempts had something the first one lacked: an accumulated record of corrections. The editing history shows what that made possible, and how much work I was still doing.

There was an evaluator with a written brief: read both versions in full, protect the wandering and the jokes that carry argument, quote the exact passage that feels synthetic and say why, and refuse to reward a revision merely because it is cleaner. Some model readers were deliberately kept ignorant of the manuscript's history, since remembering the last ten edits can give a critic reasons to approve the eleventh that have nothing to do with the prose. There were protected lines. One pass was permitted only to delete and fold, because the failure it was fixing was made of additions.

The evaluations recorded places where the writing had stopped sounding like a person, each with a reason, and changes I refused. At one stage they wanted the schema table in the chapter on patterns gone. I kept it because the chapter needed a concrete artifact; it later became the filled-in pattern

the reader has now seen. They wanted a nine-clause sentence dismantled; it stayed. They caught, twice, that connective sentences added to smooth the seams were themselves the register they had been asked to remove. Those went.

This was still work. I was refusing edits, correcting the process and restoring things it had removed. The brief and the separate readings gave those interventions something to build on; they had not made my labor disappear. The history at the start of Chapter 3 still got cut after the reason for keeping it had been recorded. I had to bring it back.

The refusal should remain mine. Remembering why I refused should not depend on my being there to refuse again.

What I want is for that record to change what happens when I make the next small request. The system might retrieve the corrections that survived and compare the chapter with passages I kept. It might decide that a disputed claim needs research, while a joke needs to be left alone. A second model could challenge an argument where disagreement is likely to add information. It could also waste an afternoon producing a committee for ceremonial reasons; the arrangement itself has to be judged.

After all that, perhaps the system changes four paragraphs. I should not have to reconstruct the institution that produced them. I said:

This chapter still feels like LLM writing.

That is what I mean by **fluent autonomy**: the structure needed to do the work can assemble around the intention.

The Interface Moves Up

The argument of this book began with a recurring move: once something complicated becomes reliable enough, the layer above can start treating it as a primitive.

We stopped programming by wiring individual transistors. We stopped thinking about registers every time we wrote a high-level function. Libraries hid algorithms. Applications hid libraries. Coding agents began treating applications, files, browsers, terminals and APIs as tools.

AI agents push that abstraction one level higher. Increasingly the interface is an **outcome described incompletely in ordinary language**.

That incompleteness matters. When I call a function, I am supposed to know what function I want. When I talk to another capable human, I often do not. I can say:

This argument feels wrong.

Find somewhere good for dinner.

I think this customer is stuck.

We need to understand why this experiment moved.

I am considering changing jobs.

None of these is a specification. Each opens a small investigation.

Traditional software handles this badly because software usually requires the designer to anticipate the structure of the intention in advance. Somebody decides which fields exist, which buttons appear, which states the workflow may enter and which exceptions deserve their own branch. That predictability is useful. It is also why every mature enterprise product eventually contains a form whose existence can be explained only by an archaeological expedition through three reorganizations.

A fluent autonomous system can construct part of the structure **after seeing the intention**. That leaves it with a choice ordinary software usually makes in advance: what interface would help here?

Menus, spreadsheets, dashboards, canvases, forms and direct manipulation are not historical accidents waiting for language models to abolish them; often they are excellent interfaces.

Sometimes I want Excel because seeing the table is faster than discussing it. Sometimes I want a dashboard because twenty numbers at once tell me more than twenty conversational turns. Sometimes I want to drag the object myself because my hand knows what I mean before I have words for it.

The application becomes a primitive available to the agent and to me. If a spreadsheet is the right temporary representation, make one. If direct manipulation is better, show me the canvas. If the task is routine, use the tool and return the result. If the problem is underspecified, conversation may remain the best interface because conversation is what humans already use when neither side knows in advance exactly where the interaction is going. The interface itself can become part of the solution.

Bureaucracy on the Fly

There is a phrase that sounds like an insult until you need it: **bureaucracy**.

Bureaucracy, in its useful form, is accumulated coordination. Roles, review boundaries, logs, standards, escalation paths and procedures exist because some kinds of work become unreliable when everybody improvises everything at once.

The problem is that fixed bureaucracy calcifies. A six-person review process designed for a dangerous database migration eventually gets applied to changing a sentence in a help page because nobody remembered to tell the workflow that reality had changed.

Agent systems give us the possibility of **bureaucracy on the fly**: an organization assembled for the problem rather than inherited wholesale from the previous one.

A factual question may need one agent and a source. A difficult scientific claim may need competing hypotheses, a literature search, code, an experiment and an evaluator insulated from the researcher who wants the result to work. A writing edit may need none of that: perhaps the original paragraph, a memory of previous corrections and enough restraint to leave the sentence alone. A high-impact financial action may need very little creativity and quite a lot of permission checking. A genuinely novel research problem may need several agents pursuing different approaches without sharing enough context to collapse into one correlated opinion.

The organization should be **as large as the uncertainty deserves and no larger**. This is where the society, the patterns and the oversight machinery meet: patterns tell the system which institutional shapes have worked before, and System 3 keeps those patterns answerable to evidence. The system can compose a temporary organization, run it, observe whether it helped, preserve what deserves to survive and dismantle the rest.

What used to be a workflow diagram becomes part of runtime. The human gives the problem, and the system compiles an institution.

Fluency Is Selective Friction

There is an easy mistake here. A fluent agent is not an agent that never asks questions. It is also not an agent that asks permission for every action. That is an approval workflow that has learned to talk. The useful question is where friction belongs.

Rename two hundred temporary files according to a convention used every week for a year? Please do not wake me. Send €200,000 to an account we have never seen because an email said “urgent”? I suddenly enjoy friction.

The human’s own learning adds another reason to slow down. Sometimes friction is not about safety. Sometimes friction is the point of the interaction. If I ask the system to teach me statistics, instantly solving every exercise is not fluent assistance—it is substitution wearing a tutor badge. If I ask for help deciding between two life choices, collapsing the uncertainty into one confident recommendation may remove exactly the thinking I needed to do. If I want a routine analysis completed, making me rediscover every intermediate step is wasted attention.

So the system has to infer not only **what outcome I want**, but **what role I want to retain in producing it**. Human attention is scarce, but the objective is not to minimize it. Spend it where it changes the result, where the action is hard to reverse, where values conflict, where the evidence is weak, where a new failure mode appears—or where the human is trying to become more capable rather than merely get the thing done. Needing little human input can be a side effect of spending it well.

Invisible by Default, Legible on Demand

There is another bad version of fluency. Everything works through one beautiful conversational box. The system performs research, edits files, transfers money, changes production settings and updates its own memory. The interface stays calm and minimalist throughout.

Then something goes wrong. You ask why, and the system says:

I made the best decision based on available context.

This is opacity with good typography.

The architecture underneath the interface has to leave traces. Which evidence mattered? Which pattern was retrieved? What alternatives were considered? Which evaluator rejected the other approach? What changed from the previous version? Which action is reversible? What uncertainty was hidden because it did not matter, and what uncertainty should have reached the human but did not?

Those are trust chains. A compiler hides registers from me most of the time, but I can still inspect the generated assembly when the abstraction leaks. A database hides pages and indexes until performance becomes strange. An autonomous system needs an equivalent way back into the work. Most of the time I should be able to speak at the level of intention. When something becomes uncertain, consequential or surprising, the lower layers should become visible again.

The request can remain simple:

Here is what I am trying to accomplish. Help me get there without losing contact with reality—or with me.

In the next writing session, I should be able to spend my attention on the argument. If I am once again explaining why the agent should read its own record of my last objection, the interface has hidden very little of the work.

Monday Morning

In an editing experiment, I can stop the work, change the brief and try again. A theory of fluent autonomy also has to survive systems that cannot be redesigned from scratch and people who did not volunteer to participate in the metaphor.

I needed a less polite laboratory. Fortunately, Monday morning was waiting.

Chapter 11: The Store That Builds Itself

When System 3 Came to Work

I lead Applied Science for product ranking and recommendations at Zalando. That gives me a slightly unfair opportunity: I can spend the weekend writing that software should become more emergent, more compositional and less micromanaged, then arrive at work and discover that real software contains latency budgets, old interfaces, business constraints, experiments, dependencies, customers who refuse to behave like the diagram, and at least one matrix somebody created for a very sensible reason three years ago.

The book came to work.

At the time of writing, what follows is a design in progress, not a victory lap. We have not proved the grand version. In fact, one of the points of the design is to make it possible to discover that the grand version is wrong before spending two years building it. This is my account of the ideas, not a Zalando strategy announcement, and definitely not a claim that we solved shopping before lunch.

The starting problem was almost embarrassingly simple. Imagine two customers looking at the same product page.

One has visited several times across several days. She filtered by size and color, looked at alternatives, came back, switched between two candidates and now appears to be stuck near a decision. The other customer arrived thirty seconds ago from a search result. We know almost nothing about what he wants, how serious he is, or whether this is the first jacket he has seen in six months.

They can see the same recommendation modules in the same order. That is not because the recommendation models are stupid. Quite the opposite. Mature recommendation systems can contain excellent retrieval, ranking, personalization, embeddings, sequence models and business logic. The strange part is one layer above them. We may have sophisticated intelligence inside each box while the arrangement of the boxes is mostly predetermined. The page is smart inside the modules and surprisingly dumb between them.

This looked familiar. The book began with a claim about emergence: once a complicated thing works reliably enough, the layer above can start treating it as a primitive. Coding agents made the same move with applications. Pattern Language did it with executable knowledge. Now I had a recommender system full of increasingly capable primitives and a question I had somehow spent an entire book preparing myself to ask:

What should the layer above do with them?

Stop Recommending for a Moment

The conventional recommendation question is usually some variation of:

Which products should I show this customer?

It is a very good question. Entire fields exist to answer it better. Retrieval finds candidates. Ranking orders them. Sequence models infer interests. Business rules remove things that should not be there. The machinery can become extremely sophisticated.

But consider the customer who is switching between the same two pairs of trail shoes for the fourth time.

What does she need? Perhaps more trail shoes. Perhaps not. There is a point at which another excellent candidate is not help. It is homework.

She may already have enough choice. Her problem could be that she cannot compare the two choices she has. Or that she does not trust the unfamiliar brand. Or that she cannot tell whether her normal size will fit. Or that one shoe costs more and she cannot see what she gets for the extra money.

Once you phrase it this way, the object being predicted changes.

Instead of asking only which *item* is relevant, we can ask which **bounded problem** is currently relevant.

Comparison friction. Size anxiety. Return hesitation. Quality uncertainty. Outfit visualization. Filter fatigue. Decision paralysis.

These names are not truths hiding inside the customer's head. They are hypotheses about difficulties we may be able to detect and, more importantly, do something about. That last condition matters. I can invent an exquisitely named psychological state for every wiggle of the mouse, but if we cannot observe it well enough to test and cannot build anything that plausibly helps, we have created a taxonomy department rather than a recommender system. The problems have to be bounded enough to attack.

Circle packing had an immutable evaluator. Shopping is messier, but the discipline is similar. Define a problem narrowly enough that an intervention can succeed or fail. If we claim somebody has comparison friction, we need a way to distinguish a resolved comparison from a customer giving up. If we say size anxiety is the blocker, we need evidence that the signal means something and a metric that can tell us whether our intervention helped rather than merely attracted a click.

Here the architecture started moving away from the familiar funnel.

People Refuse to Stay in the Funnel

Funnels are useful because humans like diagrams that get narrower toward the bottom. Explore. Form a need. Narrow. Evaluate. Decide. Purchase. The arrows point downward, everybody feels organized, and somewhere a PowerPoint theme earns its salary.

Customers are less cooperative. Someone can be evaluating one product while exploring another category. She can be price-sensitive and size-anxious at the same time. She can know exactly what dress she wants and still be unsure whether it works with the shoes she already owns. She can add something to the basket, remove it, return to the product page, read reviews, open a size chart and then disappear for three days because a child needed dinner.

A single lifecycle stage compresses this mess into one label. The design we began working with uses something richer: a **problem fingerprint**. Instead of saying the customer *is in Evaluate*, the system can represent several problem hypotheses at once, each with an intensity. Size anxiety may be high. Return hesitation moderate. Outfit seeking almost absent. Another customer on the same product may have the reverse pattern.

The fingerprint is not a personality test—it is local to the customer, the current context, the surface and the available evidence. That is important because I do not want the system deciding that Hani is metaphysically a `RETURN_HESITANT_PERSON` and carrying that fact around until retirement. Some characteristics are durable. Many are situational.

The architecture also separates the machine representation from the stories humans use to think. Designers and scientists may organize problems by funnel stage, mission, timing or recognizable archetype. Those lenses help us notice gaps and invent hypotheses. The runtime system does not need to believe the story. It needs signals, a problem fingerprint and a way to test whether the resulting behavior is useful. I like this separation because it protects us from one of the oldest mistakes in machine learning: turning a useful human abstraction into an ontological claim because we happened to put it in a feature table.

The customer is not the funnel. The funnel is one way we look at the customer.

A Library of Ways to Help

Once you define demand as problems rather than slots, the supply side changes too.

Today, when people hear “recommendation,” they often picture a ranked list of products. You may also like. Similar items. Complete the look. Recently viewed. The carousel has become the fruit bowl of ecommerce: you can put one almost anywhere and nobody asks too many questions.

But if the problem is comparison friction, a ranked list may be the wrong species of answer. The useful experience could be a comparison between the two products the customer is actually considering. If the problem is size anxiety, the useful thing may be evidence about fit. If the customer cannot imagine an outfit, it may be a generated collage. If she has only a vague mission, perhaps a product finder is better. If she knows exactly what she wants but the catalog is overwhelming, maybe the right action is a guided filter. Sometimes the answer is another set of products. Sometimes the answer is information. Sometimes it is a different interaction entirely.

I started calling these reusable units **recommendation experiences**, or RXs. The name matters less than the abstraction. An RX is more than a model: a reusable capability that knows roughly what kind of problem it can address, when it is eligible to run, how it can be configured and how it presents itself.

The long-term ambition is a large library: carousels, comparisons, outfit builders, collages, finders,

confidence modules, explanations, visual exploration, complementary-item experiences and things we have not invented yet. But the point is not to celebrate having hundreds of widgets. A library of two hundred overlapping experiences is just a new kind of legacy system with better animation.

When a new need appears, first ask whether an existing experience can meet it with a different configuration: **composition over invention**. A Similar Items experience might be generic in one context and constrained to products available in the customer's size in another. A comparison component can compare different attributes depending on what matters in the current session. A collage can be anchored on a dress, a pair of shoes or an occasion without becoming three separate products in the organizational sense.

Building for the hundredth experience changes what the first few need to be capable of. The more that useful behavior can be produced by configuring and composing a smaller number of strong primitives, the less the organization has to encode every new situation as another permanent branch in software.

I spent years in machine learning hearing that the answer to complexity was to learn rather than hand-author. Then, like everyone else, I helped build systems where the model learned beautifully inside a box surrounded by hand-authored configuration. The box was not the end of the learning problem.

Composition Is Not Ranking With a New Hat

At this point the obvious response is: fine, rank the experiences. That gets us part of the way and then breaks in an interesting place.

Suppose the system has already placed a strong size-confidence experience at the top of the page. Should another size-related module receive the same score it would have received before the first one was shown?

Probably not. We expect the first module to address some of the problem, which makes a second one less promising. Whether that expectation is justified is something the page has to teach us. A second module may add little and consume valuable attention.

Now suppose a returns-clarity experience is more useful *after* fit evidence because the two together form a coherent decision aid. Its value may increase after the first experience appears.

The score of an experience therefore depends partly on what has already been selected. That is composition.

The composer has to select experiences, configure them, order them and deduplicate not only repeated products but repeated *help*. It needs some notion of saturation: two size widgets can be one too many. It can model synergy: one experience may become more valuable after another. It should account for position cost because the top of a page is expensive real estate and a wonderful module in slot twelve may be a philosophical achievement rather than a product one. Constraints matter too, but I prefer many of them to be visible pressures rather than a secret forest of `if DE_mobile && campaign_X` rules.

Most importantly, the **page becomes the unit**. A module can win its local metric and make the page worse. This is easy to forget because teams and models naturally acquire local objectives. Increase CTR on this carousel. Improve conversion from that module. Raise engagement with this block. All

reasonable. But if one module steals a click the customer would have made anyway, we may have moved attribution without creating value. If three individually successful widgets all solve the same problem, the page can feel like a committee where everybody prepared the same presentation. The layer above has to reason about the composition as a whole.

And this is where the case study started resembling the society of agents. A society is not improved merely by hiring the best individual expert in every discipline. Somebody still has to decide which experts are needed, how they interact, what has already been covered and when another voice adds information rather than noise. A page can have the same problem.

Mei Does Not Need More Shoes

Imagine a customer. Call her Mei. Mei has two pairs of trail shoes open. She has returned to them several times across five days. She switches between the two pages quickly, saved one of the shoes and is spending less time reading each page because by now she has probably memorized half the product description.

A conventional recommender can still do an excellent job here. It can find twenty more trail shoes that look similar, match her taste and are available in her size.

But suppose the fingerprint says comparison friction is high and price-quality confusion is moderate. The composer can do something different. The first experience compares the two shoes Mei is actually deciding between on attributes relevant to her behavior. The second adds confidence evidence from customers or product information that helps resolve the remaining uncertainty. Generic similar-items may still survive because it has useful standalone value, but it moves down.

She is not shown more choice. She is shown a way to close the choice she already has. That sentence changed how I thought about recommendations.

For years, the field has been extraordinarily good at finding things. Search finds things. Recommenders find things you did not ask for. Retrieval systems find things at absurd scale. But shopping is not only a retrieval problem. At different moments it is also a comparison problem, a confidence problem, a visualization problem, a constraint problem and occasionally a “please stop showing me another black sneaker” problem.

A system that can only respond with more items is like a doctor who has one extremely accurate prescription and keeps waiting for every disease to become the disease it treats.

But Mei could stop switching because the comparison helped, because she gave up, or because dinner arrived. A quieter session is not yet a solved problem. In the proposed test, we would compare outcomes across customers eligible for this intervention, including a group receiving the existing experience. Does the comparison help people reach a decision without increasing later returns or regret? Direct customer feedback could help us understand what the behavioral measures miss.

If switching falls while abandonment rises, the apparent success should make us distrust the metric. If a comparison helps some sessions and overwhelms others, the next change may belong in its eligibility rule. And if a simpler page helps just as much, the composer has learned something inconvenient about its own necessity. Even a useful intervention would not prove that we had read Mei’s mind correctly.

Sami Does Not Need a Click

Sami has selected a size but has not added the product to his basket. He opened the size chart twice. It is a brand he has not bought before. Perhaps his current problem is size anxiety, with some return hesitation behind it.

One useful response might not be shoppable at all. Imagine a small evidence module explaining how people with comparable sizing histories tended to fit this item, or giving a properly substantiated signal about whether buyers kept their usual size. The exact claim matters enormously because a false fit claim is worse than a mediocre recommendation. But conceptually this is a different kind of RX: it provides **knowledge**, not another candidate.

Now try to optimize the whole system for expected click. The insight module is in trouble.

If it works perfectly, Sami may read it, become confident and press Add to Bag. The module itself may receive no click. A carousel with attractive shoes can collect engagement more easily while being less relevant to the thing stopping him.

This is a small example of a much larger problem: the objective determines which species of intelligence can survive. If your ecosystem rewards clicks, clickable organisms evolve. The architecture therefore needs different value terms and different evidence standards for different experiences. Item recommenders can be judged partly by engagement and downstream action. Insight experiences may need read-through, decision confidence, return behavior or problem-specific outcomes. Claims need substantiation thresholds. Some experiences are cheap to be wrong about. Others can mislead a customer or create regulatory risk. The library is heterogeneous because the problems are heterogeneous.

And now the camel comes back: where did the claim come from, how strong is the evidence, what kind of knowledge is this, and how much trust should the system place in it before acting? System 3 is no longer a chapter about hallucinations. It is a product requirement.

The Honest Cold Start

There is another customer I like because she reveals whether the architecture can resist pretending.

Lea arrives from a social link. No account. No history. Almost no session depth. The system has the product she opened, perhaps the season, approximate location and a few ambient signals. That is it.

A personalization system can react to this situation in two ways.

One is to panic quietly and run a generic fallback while still speaking in the confident dialect of personalization.

Picked for you.

Based on what, exactly? Her IP address and our enthusiasm?

The other is to treat low signal as a normal state with its own design. Lean on the anchor, season and population-level evidence. Prefer experiences with strong standalone value. Frame them honestly. “Popular this week” can be a good statement when “we have inferred your soul from one click” is not. This is what I mean by graceful degradation. Cold start is a state, not an error. If a large fraction of requests arrive with weak signal, the low-signal path may be the product and deep personalization

the special case. The architecture should know what it does not know. That sounds obvious until you look at how much software is built around pretending the common messy case is an exception handler.

The Trace Is Part of the Intelligence

Dynamic systems create a governance problem immediately.

A static page is relatively easy to inspect. This module goes here. That one goes there. If something looks wrong, somebody can open the configuration and complain about whoever last touched it.

A composer makes a fresh decision from context. Now a customer reports a terrible page and the first debugging question becomes:

Why did this page exist?

“The model chose it” is not an answer. It is a resignation letter written in passive voice. So every composition needs a trace.

Which signals were read? What problem fingerprint was inferred? Which experiences were eligible? Which were not? How were they configured? What scores did they receive? Which constraints mattered? What won? What lost? Which version of the composer produced the decision?

The losers matter more than they first appear. If we log only what we served, we can attribute outcomes to the winner but we lose much of the decision context. We cannot tell whether an experience was absent because it was ineligible, starved by the objective or simply scored slightly below another. We cannot replay the decision properly. We cannot compare a new policy against the old choice set without reconstructing a world we chose not to record.

Logging the loser set does not magically give us causal counterfactuals. Reality is not that generous. But it gives us the archaeology of the decision. This is exactly the move System 3 has been making throughout the book. Do not preserve only the polished conclusion. Preserve enough of the chain that future systems can inspect why the conclusion deserved trust.

The trace also changes development. You can build a simulator that replays saved scenarios. You can ask which experiences would be eligible in a context or which contexts a new experience could serve. You can run regression suites over scenarios before changing the library. A dynamic system becomes safer not because it stops changing but because its changes become replayable.

From Machine Learning to Knowledge

Somewhere around here the project stopped looking to me like a normal recommendation-system redesign.

The models still matter enormously. We need representations, retrieval, ranking, sequence understanding, problem detectors, value models and probably more machinery than I can fit into a chapter without losing several readers to a sudden interest in gardening.

But the durable asset begins to include something else. A problem catalog. A library of reusable experiences. Knowledge about which experiences address which problems. Eligibility conditions. Ev-

idence requirements. Presentation strategies. Scenarios. Traces. Regression tests. Guardrails. Rules for when an experience should be retired.

An experience is useful not merely because somebody built a clever model for it. It becomes useful organizational knowledge when we know the recurring situation it addresses, the evidence that should trigger it, the conditions under which it fails, the other experiences it complements or duplicates and how its value should be measured.

A new comparison module without that context is a feature. A comparison pattern with evidence, boundaries, history and known interactions is culture.

And culture has the same failure mode we saw earlier: it can become a junk drawer with tenure. If every newly observed problem creates another RX, the library eventually recreates the configuration matrix in a more colorful form. So new supply needs a gate. Is the problem real? How large is it? Can an existing experience be configured to address it? Where does the current library have weak coverage? Which experiences stopped relieving the problems they were created for and should disappear?

This led to a pair of concepts I particularly like: **Coverage** and **Unmet Demand**. Coverage asks, at design time, which known problems the current library *could* address. Unmet Demand asks, from production, which detected problems remained insufficiently addressed after composition.

Put them together and the roadmap starts to emerge from the system's own failures. That is a very different way to decide what to build next. If System 3 is science, Coverage and Unmet Demand are more than roadmap metrics. They tell the institution where its current theories and instruments are weak. A recurring problem with no effective RX is an anomaly the product cannot yet explain away; a heavily used intervention that stops relieving the problem is a theory losing contact with reality. The roadmap becomes partly a **research agenda generated by the failures of the current system**.

Let the LLM Narrate. Do Not Let It Declare Reality.

AI can help with problem discovery too, and here fooling ourselves becomes cheap.

Imagine replaying anonymized customer sessions and asking a strong language model to narrate what appears to be happening. The customer compared three products, opened the size chart, returned to one PDP, removed an item from the basket and left. The model can generate a plausible diagnosis. Cluster enough narrations and you may discover recurring forms of friction that your existing taxonomy missed.

“The customer hesitated because of fit” may be an excellent story. The customer may also have received a phone call. So narration should generate hypotheses, not production truth. Take a sample. Compare the diagnosis with interviews, surveys, support contacts or other evidence closer to the customer's actual experience. Build a detector only after the hypothesis survives contact with something outside the model's coherence. Define what success looks like before the detector starts steering the page.

The same rule applies to observational analysis. Customers with comparison friction may convert less, but perhaps weaker-intent customers simply compare more. Correlation can prioritize what to investigate. Only intervention tells us how much of the outcome the problem was actually causing. Judea Pearl's ladder has three rungs, seeing, doing and imagining, and no quantity of the first substitutes for the second. A recommender that only observes is standing on rung one with excellent posture.

I find this satisfying because the architecture does not merely *use* System 3. It needs System 3 to avoid hallucinating its own customers.

This is the book's central thesis in work clothes. The LLM is excellent at generating explanations. The product architecture has to decide which explanations deserve pursuit, construct interventions that expose them to consequences, preserve the chain of evidence, and update the repertoire when the world refuses to cooperate. **Philosophy of science has become product architecture.**

The Objective Fights Back

Eventually the design forced us to name the thing the composer is supposed to optimize.

We used the deliberately bland term **Surface Value**. This is where the project becomes philosophical against its will.

If Surface Value is module CTR, we have not solved the page problem. If it is total clicks, a page full of shiny modules may win while the customer gets nowhere. If it is immediate purchase probability, experiences that build confidence or improve a longer mission may be undervalued. If it is revenue, expensive products get interesting very quickly. If it is margin, the store's objective can start eating the customer's. If it is long-term value, we have gained a beautiful phrase and several years of causal-inference work.

The objective has to be page-scoped enough that compositions can be compared, but decomposable enough that we can diagnose why a page helped or failed. Different problem classes need their own success signals. For comparison friction, the signal must distinguish resolution from abandonment. For size anxiety, customers should be able to progress with fewer signs of uncertainty without creating a return problem later.

This is Layer 4 in production. What do we actually want?

The store has legitimate business goals. Customers have goals. They are often aligned and sometimes not. Inventory has constraints, merchandising has opinions, margin and availability are real, and so are regulators. A system that pretends only one of these matters is not simpler; it is hiding politics inside a scalar.

We are trying to make the trade-offs explicit enough to test, govern and revise. I do not expect to discover the One True Ecommerce Reward Function carved into a mountain somewhere outside Berlin.

This is why I increasingly dislike architectures where business decisions enter through invisible overrides. If merchandising needs a lock, make it a typed constraint. If margin is part of the objective, admit it. If a claim needs compliance review, attach the evidence rule. If the system violates a soft constraint because another objective dominated it, log the violation.

The architecture should not make disagreement disappear. It should make disagreement inspectable.

Bounded Ambition

After all of this, the sensible first experiment is obviously to build hundreds of widgets, a general customer-reasoning model, a cross-surface scheduler and an autonomous agent that redesigns fashion retail by Thursday.

The test we are designing is deliberately boring. One placement: the product page. A small number of validated customer problems. The existing recommendation library, with only limited new supply. A simple composition mechanism. A trace good enough to explain an individual decision. An authored objective before a learned one.

Why so narrow? Because if we invent a new library of experiences and change the selection mechanism at the same time, then run an experiment and get a flat result, we have learned almost nothing. Maybe the composer is bad. Maybe the new experiences are bad. Maybe both are good and the measurement is bad. Maybe the static page was already fine and I should have spent the quarter learning the guitar.

A bounded test separates the claims. Does dynamic composition beat a strong static baseline? And importantly: does it beat simplification? That second competitor is easy to underestimate. Perhaps the best response to an overloaded page is not a brilliant composer. Perhaps it is fewer things. The system should have to earn its complexity against the possibility that removing modules produces a better customer experience.

I love this part because it keeps the book honest. A philosophy of emergence should be willing to lose an A/B test. Otherwise it is not a philosophy of experimentation. It is branding.

And if System 3 is science, this is not merely rhetorical humility. **The architecture must contain a route by which the book's own theory can lose.** The A/B test is not there to validate the philosophy; it is there to threaten it.

When the Page Stops Being the Product

Suppose the narrow test works. Then the interesting version begins.

The library grows beyond carousels into richer experiences: comparisons, collages, product finders, outfit builders, confidence modules, visual exploration and whatever else proves useful. Configuration becomes richer so one experience can serve several contexts without a matrix of handcrafted variants. Problem discovery improves. Unmet demand exposes missing capabilities. The composer learns a better objective. Different surfaces begin to share a coherent read of the customer's current mission.

At that point, the word *page* starts to become suspicious. Why should the product page always contain the same conceptual structure?

Why should a customer with a decision problem receive the same interface as somebody exploring for inspiration? Why should the home surface, product page, basket and later email behave like four organizations with partial amnesia if the customer is still pursuing one mission?

The more capable the library becomes, the more the system can schedule **problems and interventions** rather than modules and slots.

A customer starts with a vague request for a wedding outfit. The system helps narrow the style. A collage makes one direction concrete. Seeing it changes what the customer wants. The problem shifts from exploration to comparison. A product finder resolves a constraint. A size question appears. The scheduler brings in fit evidence. The customer buys the dress but not the jacket. Later, a different surface may continue the unresolved part of the mission.

There was never a hard-coded `WEDDING_FUNNEL_V7`. The journey emerged from bounded problems, reusable capabilities and changing evidence.

The hundreds of widgets stop being a UI roadmap here and become a **vocabulary of action**. The interface is the current projection of the problem-solving process.

Nobody wants every pixel generated by an LLM. Predictability, accessibility, design systems and latency all matter, and customers occasionally just want to buy socks without participating in an artificial-intelligence research program.

Fluent autonomy is selective. The machinery should become dynamic where dynamism earns its cost and remain boring where boring is excellent. But the direction is different from the old model of product development. Instead of predicting every useful journey in advance and encoding it as a fixed interface, we construct a repertoire of trusted capabilities and let the higher layer assemble them around the problem in front of it.

The store does not literally build itself. It learns how to build more of the experience it needs.

The Book Comes Back to Bite Me

I began this project as a recommendation-system redesign. By the time the design needed competing explanations of customer problems, experiments capable of rejecting them and a memory of what survived, it had acquired the shape of a **scientific institution embedded in the product**. The store would be learning which kinds of help to offer, and how to notice when its own account of the customer was wrong.

I had spent ten chapters arguing that these ideas belonged together. Then I walked into a recommendation problem and found myself rebuilding the same architecture because the old abstraction stopped scaling.

That does not prove the book. It is one case study, in one domain, at one moment, and it may fail in several educational ways.

But it changed the question for me. The important future system may not be the model that predicts the next product best. It may be the system that can discover what kind of problem exists, recruit the right capabilities, construct an intervention, inspect whether it helped, learn from the gap and change what it does next.

And once you can imagine that happening in a store, it becomes difficult not to imagine it happening in research, education or the organization doing the building. More of the work moves into the system, including work I once regarded as the reason it needed someone like me. That is an attractive architecture and a less comfortable thing to notice about your own profession.

Chapter 12: After Capacity

A Glimpse of Double Descent Life

LLM-as-a-judge arrived in my team looking like a threat.

For years, a large part of what made an applied scientist valuable was that she could do two things most of the company could not: train a model, and run an evaluation somebody would believe. Those were scarce skills, and a job accumulates around scarce skills the way a city accumulates around a river. Then models became something you download, and evaluation became something you ask a model to do, at a scale no human team could match. It is hard to watch the river move without wondering what the city was for.

Training and evaluation were never the point of the job. They were the expensive part of it, and expensive is easy to mistake for essential. What a scientist actually brings is stranger and harder to see on an org chart: a working map of where the frontier is, which of the ideas currently exciting everyone has already failed three times under another name, which result quietly changed the design space last month, which neighboring field has just built something that matters to our problem, and how to turn any of that into an investigation the company can trust. Somebody has to decide what evidence is strong enough for an institution to change its mind. A judge that can score a million recommendations overnight does not answer that question so much as make it urgent, because there are now a million more things that look like evidence.

I want to be fair to the threat. Parts of my job should disappear. Nobody needs a scientist to run the same evaluation for the fourth time, or to hand-tune a model that an automated search will tune better by Thursday. If the only thing left is deciding what the instrument may be trusted to see, that is a smaller job than the one I trained for. I think it is also a more important one, but I would say that, and a book that has spent eleven chapters insisting that claims be exposed to something capable of disagreeing should not exempt its author's job description.

For now, there is plenty of work at that higher level. But “the role moves upward” is a description of where I might be useful next, not a guarantee that the next level belongs to humans forever. The previous chapter was already trying to give a system some of that work. I cannot keep proposing more capable agents and then reserve the interesting questions for myself whenever the argument reaches my salary.

What if AI does not merely answer more questions or automate more tasks, but steadily moves upward through the stack? It retrieves, ranks, composes, diagnoses the problem, chooses a strategy, builds the tool it needs and learns from the result.

At each step, something that used to require scarce human capability becomes infrastructure for

the layer above. This book has mostly treated that as an architectural problem. How do we make autonomy useful? How do we keep it connected to evidence? How do agents coordinate, remember and remain corrigible as both the world and the human change?

Behind these architectural problems is another question: what happens to human life when **capacity itself becomes much cheaper**?

Not all capacity. We will still have one planet, finite land, finite energy, twenty-four hours in a day, and restaurants that somehow remain fully booked exactly when you want to go. Bodies remain bodies. Politics does not evaporate because a model can write Python. Scarcity is not going to receive a polite email from OpenAI and retire.

But cognitive capacity is already becoming strange enough to force the question. A person can enter a field she never studied and get a useful map in an afternoon. A small team can produce software that previously required a much larger one. Research, design, analysis, translation, tutoring, programming and increasingly complicated forms of planning can be amplified by systems available to people who did not spend twenty years acquiring every underlying specialty.

The usual story jumps directly from *humans do the work* to *AI does the work*. Then we spend the rest of the conversation asking what humans will do with all the free time, having quietly skipped the question of who can afford it.

There is another possibility worth following: we keep doing things. We start doing things that were previously economically ridiculous. That would not make the disruption painless, but it changes the question from how much work remains to what becomes possible when the work gets cheaper.

I have been calling the larger philosophy around this **Double Descent Life**. The name is stolen from machine learning, and the theft is deliberate. In the curve described by Belkin and colleagues, test error falls, rises near the point where a model can fit the training data, then falls again as capacity grows further. There is no economic law hiding in that curve promising that lost jobs return. I am borrowing its invitation to look beyond the first crisis, then using the two descents in a different sense.

One descent happens outside us. Implementation, expertise and coordination become cheaper and move downward into infrastructure. Things that once consumed years of training or layers of organization become building blocks.

The other happens inside us. As some practical difficulties fall away, we have fewer of them to hide behind. I may finally be able to build the thing and discover that I am unsure whether I want it. I may gain access to twenty possible lives and still have one life in which to choose. The questions concern what deserves my time, how I came to want it, and what I owe other people while pursuing it.

As the machines descend into implementation, we descend into meaning. That does not sound easier, because it is not.

The second descent is not automatic. Cheaper tools can make it easier to avoid those questions too. This chapter is a glimpse of a philosophy still being built, and I am suspicious of neat doctrines anyway. The history of thought is full of people who reached page 300 and announced that history had finally arrived at the correct system, usually just before history did something rude.

When Difficulty Becomes Infrastructure

Human institutions were built under assumptions about what is difficult.

Writing good software is difficult, so we organize teams of specialists around it. Scientific expertise is difficult to acquire, so we create universities, journals and long apprenticeships. High-quality legal or financial analysis is expensive, so access is uneven. Producing media is costly, so publishing institutions decide what gets distributed. Coordinating a large organization is difficult, so we create layers of management whose main superpower is knowing which meeting another meeting should produce.

Scarce capability shapes power. If I cannot build something myself, I need somebody who can. If one organization owns the machinery, data, expertise or distribution required to act, then access to that organization becomes valuable. We spend a surprising amount of human life acquiring permission from structures that exist partly because doing the thing directly is too expensive.

AI changes some of those costs. Cheaper capacity does not flatten society by itself; it can concentrate as easily as it distributes. The company with the best models, compute, data, distribution and capital may gain more power, not less. Cheap software can empower a teenager in Amman and a surveillance state at the same time. Capability has never come with an ethical direction preinstalled.

Still, something important happens when the cost curve moves. If an individual or small group can increasingly research, design, build, analyze and operate things that previously required a much larger institution, then some problems that looked like power problems may turn out to have been **capacity problems wearing a suit**.

You wanted software tailored to how your team actually works, but building it was too expensive, so you bought a generic SaaS product and reorganized the team around the dropdown menu. You wanted a course that teaches exactly what you need at exactly your level, but producing one teacher per student was impossible, so thirty people entered a room and agreed to move at approximately the same speed. You wanted to test a policy idea, but the analytical machinery was too expensive, so the argument remained mostly rhetorical.

When capability becomes cheaper, the design space opens—not infinitely, equally or safely by default, but enough that the old question, *Who controls the scarce machinery?*, is joined by another:

How much of that machinery can we move closer to the person who needs it?

That is where capacity begins to compete with power as a way of getting things done.

Bespoke Comes Back

Software gives us a useful example because we have already lived through two economic modes.

The first was bespoke software. If you had enough money, somebody built the thing for you. Banks had their systems. Airlines had theirs. Governments had theirs. Large companies employed armies of engineers to encode their peculiarities into software because those peculiarities were valuable enough to justify the cost.

Then software became a service. This was an enormous improvement. Instead of every company building payroll, CRM, project management, analytics, communication and twenty other systems from scratch, somebody could build one good product and sell it to millions of people.

But scale has a price. To serve millions of people, the product has to become generic. The strange needs of one team become feature requests. The software acquires configuration menus, plugins, workflows, permission systems and eventually an enterprise tier whose main feature is that somebody will answer your email.

Then organizations start adapting themselves to the software.

There is a third mode hiding behind AI: **bespoke comes back, without necessarily bringing bespoke economics with it.**

I mean systems that would previously have been too specific to justify building at all. A scientist may construct a research environment around one question, use it intensely for three months and throw most of it away when the question changes. A teacher may build an entire interactive world for one class because those particular students are stuck on those particular ideas. A small company may create internal software whose assumptions match the company instead of spending two years teaching the company to behave like Salesforce. A family may have tools built around how that family schedules, learns, travels, budgets and remembers things, with exactly zero concern for whether the addressable market justifies Series A.

Some of these systems may serve a thousand people. Some ten. Some one. That used to sound economically absurd. It may become normal. This changes the human role in a way the automation story tends to miss. The future does not have to be:

humans build → AI builds → humans watch.

We may remain intensely involved precisely because building becomes more interesting when the distance between imagining something and making it real collapses. I do not build only because the machine cannot. I build because I want the thing to exist.

The human contribution moves upward: choosing the strange problem, forming a taste for what good looks like, combining ideas that normally live in separate professions, seeing the result and saying *No, that is not it*, then pushing somewhere neither the original prompt nor the original system anticipated.

This is the abstraction ladder reaching economics. Implementation becomes another building block, and people who could not previously afford it may inherit a way to act. The cost of writing code is only part of the bill, though. A tool that handles a family's calendar can fail differently from one that controls a hospital. Maintenance, security, access to data and somebody capable of noticing a dangerous mistake still have to be paid for. A cheap first version is not yet cheap independence.

Even within those limits, I find this more interesting than guarding the last task the machine cannot perform. A person may be able to instantiate **far more of what she can imagine**. That is a much more attractive future than becoming the residual labor category in an automation spreadsheet. It also leaves her with a problem the spreadsheet was never designed to answer: which of those possibilities deserves her attention?

Learning at the Speed of Curiosity

There is another kind of capacity that may change even faster: learning. Expertise is expensive partly because knowledge has difficult interfaces. Some of the effort produces depth: working through a

proof teaches you something a summary cannot. Some merely keeps you from finding the proof. That second kind has killed an enormous amount of curiosity before depth had a chance to happen.

AI changes that bargain. I can ask a stupid question immediately, then a more sophisticated stupid question. Ask for the intuition, then the mathematics, then the objection, then the historical argument, then why the proof needs that assumption. I can have one field explain another, and move through mechanism design, philosophy of science, biology, constitutional theory and compiler construction at a speed that would previously have required several lives—or at least several abandoned PhDs.

The map has become cheap, and the trap that comes with it is the one Layer 4 had to learn to notice: fluency arrives before scars, and a person can acquire the vocabulary of five fields and mistake fluent traversal for mastery.

We need not restore the friction artificially. We can explore broadly and descend selectively. Use AI to cross fields cheaply, test curiosity and decide what deserves more attention. Then, when something matters, go down. If the argument depends on an equation, derive it; if it depends on a system working, build enough of it to find out where it breaks. Read the primary paper and talk to somebody who actually does the work. Let reality make the lesson expensive again.

This is System 3 applied to learning. AI gives us extraordinary access to synthesis; System 3 reminds us that synthesis and justified knowledge are not the same thing.

That trade may change what an educated human looks like. The twentieth-century ideal often rewarded specialization: know one vertical deeply enough that people in neighboring verticals stop understanding you. The AI-assisted human may become more T-shaped, -shaped, octopus-shaped—choose your consulting diagram. Broader, faster at entering unfamiliar domains, more willing to combine ideas that institutional boundaries kept apart, while still going deep where the stakes or fascination justify it. Expertise is not obsolete in that world, only more deliberate.

And there is a creative consequence. A machine-learning scientist can learn enough philosophy to steal a useful structure. A philosopher can prototype the mechanism she has been describing. An artist can build software, and a local policymaker can examine a proposed intervention in a model before trying it in the world. Fields become more permeable. People become more dangerous in the nicest sense.

But crossing into a field and living there are different commitments. A useful afternoon with philosophy may open a question that takes ten years. Help with the technical difficulty makes that commitment more available; it does not supply the ten years. My attention can now reach more places than my life can follow. That is where the cheerful account of cheaper capability begins to become a less comfortable account of choice.

The Second Descent

The capacity to act can scale faster than the capacity to want wisely. Humans still have status anxiety, tribal loyalty, love, jealousy, resentment, curiosity, generosity, fear, ambition, boredom and the ancient desire to prove that the neighboring group is composed mainly of idiots. None of these disappears because inference got cheaper. Giving us much more ability to execute whatever we want could be wonderful. It can also be a Ferrari engine attached to bicycle brakes.

Humans do not carry a stable reward function inside the skull. We infer, construct, revise and

sometimes borrow our desires from the people and systems around us. We contradict ourselves. We want security and novelty, belonging and freedom, status and peace. We sometimes discover what we wanted only after getting the thing we thought we wanted.

My own opening question contains one of those borrowed desires. I want to know where an applied scientist will remain valuable. That is a reasonable question about earning a living. It becomes a much stranger one if I let the answer decide whether my life is valuable. The anxiety crosses that boundary so easily that it is worth stopping there.

What Are Humans For?

Whenever automation becomes powerful, somebody asks what humans will be *for*.

I understand the question. If software writes the code, models perform the analysis, robots eventually move more of the physical world and agents coordinate the workflow, what is our economic role?

But there is something odd about the grammar. What are humans **for**?

A database is for storing information. A compiler is for translating programs. A recommender is for helping people find or decide among things. Asking what humans are for smuggles in the assumption that our legitimacy depends on having a remaining function in somebody else's architecture.

My children do not need comparative advantage to justify dinner.

Neither do I.

Economics does not disappear. People need income, housing, food, healthcare, status and access to resources. If automation breaks the mechanism by which income has traditionally been distributed, saying "human life has intrinsic value" will not pay the electricity bill. Political economy remains stubbornly material.

But a society that makes employment central to adult life bundles together two questions that AI may force us to separate: **How do people get resources?** and **What makes a life worth living?**

For a long time, a job has answered parts of both. It provides money, but also status, routine, social contact, identity, a reason to get dressed and a group of people with whom to complain about another group of people. Work is not one thing. It is a bundle. AI may unbundle it.

That makes "you will have more time for creativity" a rather incomplete answer to someone losing a livelihood. The freed hours and the gains from automation may belong to different people. A company can become more capable while its former employees lose the practical freedom to refuse bad work. There is no contradiction in that outcome. It is one reason the distribution of capacity belongs inside this philosophy, not in a footnote after the exciting part.

Perhaps some people work fewer hours. Perhaps new forms of work appear because human wants expand faster than automation satisfies them. Perhaps many of us continue working furiously, except the unit of ambition changes: one person can attempt things that used to require a department, and a small group can attempt things that used to require a corporation.

One possibility beyond employment is **more creation**—some economically useful, some absurd, some beautiful, some probably involving a bespoke dashboard nobody other than its creator can understand. Leisure, care and simply having less to do would also be perfectly respectable uses of the capacity.

Status competition will not politely resign either. It may migrate from intelligence and professional skill toward taste, reputation, physical scarcity, authenticity, human attention or something even more exhausting.

What I do know is that “find the tasks machines cannot do” is a depressing philosophy of human value. It turns civilization into a benchmark where we keep moving humans to the remaining columns after every model release.

If AI becomes better at poetry, we are not obligated to stop writing poems. If it becomes better at chess, humans do not lose permission to play chess. If it becomes better at writing software, we may write **more software**, because the things worth building are no longer restricted to those whose economics justify a software company.

I can still want to be an excellent scientist. The problem begins when I need science to remain difficult enough to require me before I can feel that my life has value. I do not want to make that depend on the next model release.

The Human Is Not the Reward Function

Layer 4 named the ugly shortcut: alignment by editing the human. At the scale of one conversation, I want the ability to question the advice, inspect its evidence and bring in another perspective. At the scale of a life, I also need somewhere else to go. A patient assistant can preserve every detail of my history and still leave me dependent on the company that owns the memory, the model and the means of acting on its advice.

Reflective agency needs more than a well-behaved conversation. Can I carry my work and history elsewhere? Can another person understand enough of the arrangement to challenge it? Can I leave without losing the practical ability to do the things the assistant made possible? These are questions about how a service is built and owned, and about the alternatives available outside it.

Imagine that the bespoke family tool has become useful enough to hold the calendar, the children’s learning history, the unfinished projects and the reasons behind years of small decisions. A button labeled *Export* may give me all the files. It may take months to make them useful elsewhere. Even if I disagree with the provider’s new terms, I may be unable to afford the interruption. The tool gave the family more capacity and gave its owner more leverage over the family at the same time.

Trust chains and independent perspectives help. They make influence inspectable and give disagreement somewhere to enter. But a right to disagree inside one product is a thin kind of freedom if everything I need to act remains locked inside that product. I do not need to become a competent engineer, teacher and system administrator before I am allowed to benefit from help. I need alternatives that work, including other people and institutions I can turn to. Independence cannot mean doing everything alone; that would undo most of the civilization Chapter 5 was trying to explain.

System 3 can help answer *Why should I believe this?* and *Why does the system think I want this?* It cannot, by architecture alone, answer *What kind of life should be possible?*

That is politics, ethics, culture and philosophy. The annoying disciplines.

The limitation becomes clearer when we talk about optimization itself. I do not want to maximize time with my children. That sounds nice until the optimizer concludes I should never go to work, see

a friend alone, read a book in peace or spend fifteen minutes doing absolutely nothing because the children are statistically nearby.

I do not want to maximize happiness if the cheapest route is a drug. I do not want to maximize productivity if the optimum is becoming an efficient ghost. I do not want to maximize longevity at every cost, wealth without purpose, social approval by becoming whatever the crowd currently rewards, or authenticity so aggressively that I become unbearable at dinner.

A good life contains goods that conflict: love and freedom, belonging and individuality, ambition and rest, truth and mercy, security and adventure, continuity and reinvention. The conflicts are not bugs waiting for a scalarization expert. Sometimes living is the process of negotiating them.

Take an ordinary evening. Time spent finishing an investigation is time I cannot spend with my children; abandoning the investigation may mean letting down people who depend on it. A better calendar might reveal a needless meeting tomorrow. It cannot make every conflict of this kind disappear. Even a perfectly predicted account of how I would feel about each choice would leave me having to choose, and another person's claim on me would still be there. Asking the assistant to settle everything would give it authority over the trade-off, not remove the trade-off.

This is why the human should not sit at Layer 4 merely as the source of a reward signal for the machine. The human is inside the process by which the objective is continuously reconsidered. AI can participate in that process without owning it. It can show me possibilities I did not know existed, teach me enough of a field to make a different choice imaginable, build prototypes of several futures and lower the cost of exploring a life before I commit to living it.

Perhaps that is one of the deepest meanings of cheaper capacity: more possibilities become **thinkable enough to try**. Or we may use it to watch fourteen hours of personalized short video generated specifically around whatever keeps us watching. The second descent can be helped by the system that made the first one possible. It cannot be handed over to it without also handing over some authorship of the life being considered.

And no amount of reflection will produce a life that everybody else agrees is the right one. My commitments involve other people, who have commitments of their own. Once our new capacity lets us act on them, the question grows beyond what any one of us wants.

The Ideology Vortex

A machine may help me make sense of my commitments and help my neighbor defend commitments I find appalling. It may do both competently. We are accustomed to treating disagreement as a failure of understanding, so it is tempting to imagine that enough intelligence will eventually clear it up. But consider the material it has to work with.

There is a story we like to tell about intellectual history because stories prefer arrows.

First there was the premodern world: religion, tradition, inherited authority, myth. Then modernity arrived with reason, science, universalism, institutions and progress. Then postmodernism arrived carrying a small hammer and began tapping on every universal claim to see what was hiding inside it: language, context, power, contingency, who got to define the categories in the first place. Then, presumably, something comes after.

The problem with this story is that nobody informed actual humans. We did not uninstall the previous operating system. A person can demand randomized evidence for a medical claim, ask her mother for a blessing before a major decision, read a horoscope for entertainment, manage a team using dashboards, believe deeply in national mythology, quote a postmodern philosopher about constructed categories and then become furious because somebody used the wrong definition of a sandwich.

Entire societies work this way. Semiconductor fabs coexist with ancient identities. Bayesian inference coexists with rumor. Universities teach critical theory while their admissions systems produce precise numerical rankings. A company can run sophisticated causal experiments in the morning and make a major organizational decision in the afternoon because one senior person “has a feeling.”

I call this the **ideology vortex**. Not because every worldview is equally true. They are not. Reality remains annoyingly capable of rejecting bad engineering regardless of how socially constructed the bridge feels on the way down.

These labels compress vast, internally divided traditions, not just successive historical periods. I am using them to notice how several modes of knowing and valuing operate at once. Inherited traditions carry identity, ritual and forms of belonging that rational administration can underestimate. Modern science gives us extraordinary machinery for verification. Critique reminds us that institutions and categories are not neutral merely because somebody printed them in a table. None of us lives entirely inside one of these descriptions.

Each mode sees something the others can miss. Each can also become ridiculous when asked to do every job.

Science is extraordinarily good at questions reality can adjudicate. It is less good at deciding which trade-offs a society should consider legitimate. Tradition can preserve hard-won social knowledge, and preserve injustice with the same impressive durability. Markets coordinate preferences and information, but prices do not encode every value we care about. Democratic institutions create legitimacy through participation and contest, but anyone who has watched a parliament knows participation and wisdom are not synonyms. Permanent critique can expose hidden assumptions until it becomes incapable of committing to anything except the superiority of critique.

Humans switch among these modes without waiting for permission from philosophy. AI enters *that* world—not the clean one in which everybody has a coherent utility function, a shared epistemology and a calendar invitation for the social contract.

Give everyone better information and surely the disagreements shrink.

Some will. Others will get better lawyers.

A powerful model can help a scientist interrogate evidence. It can also help a conspiracy theorist construct a more coherent conspiracy. It can make propaganda cheaper, criticism sharper, religious interpretation richer, policy analysis more sophisticated and advertising more personal.

More intelligence does not guarantee one worldview. It increases the capacity available to worldviews. And AI introduces a second reason the vortex matters: the technology itself is strangely compatible with ambiguity.

I have a sentence that gets me into trouble: **Gradient descent is the answer to Derrida.**

This is deliberately unfair to Derrida and possibly to gradient descent. I do not mean that an optimizer disproved postmodern philosophy. It would be a remarkable conference paper if it had. I mean something narrower.

In *Signature Event Context*, Derrida challenges the idea that we could finally specify a context completely enough to close interpretation. A sentence can be repeated elsewhere, by someone else, under circumstances its author did not control. This does not make communication impossible. It makes the dream of securing meaning once and for all a much more demanding proposition than it first appears.

An engineer might hear a practical obstacle in that argument: how do we build reliable machinery out of language if the specification can always acquire another reading? We found one response by building machines that operate inside the mess.

Large language models do not begin by fixing every word to an eternal definition. They learn from use, relation, context and enormous numbers of imperfect examples. Optimization pressures the system toward behavior that works often enough under the training and evaluation environment. Meaning remains fuzzy at the edges.

The product ships anyway. Gradient descent did not defeat ambiguity. **It made ambiguity computationally useful.**

Then, immediately, we rediscover why modernity existed. A model that can move beautifully through fuzzy language can still hallucinate a citation, miscalculate a number or confidently tell you that a camel lives in Croatia. System 3 brings verification back through another door.

There is a further limit. A model can learn how an institution uses the word *family* well enough to fill in a form. That does not establish who should qualify for membership or whose relationships the form ought to recognize. Correctly reproducing an institution's categories may be useful work; it may also reproduce the exact exclusion someone is trying to challenge. Verification can establish whether the form was filled in correctly. It cannot make the rule legitimate merely by checking it more carefully.

This is why the ideology vortex is not a bug we eventually fix by choosing the winning epistemology. We need different modes for different jobs, and ways to contest the boundary between them. Some claims deserve hard empirical tests. Some institutions need legitimate contest over the ends those tests serve. Some identities and commitments are constructed without therefore being fake.

The childish response to contingency is to pretend our constructions are eternal. The adolescent response is to discover they are constructed and conclude that nothing deserves commitment. There is another possibility. **Construct them knowingly.**

Build institutions while remembering that institutions can be rebuilt. Love people without needing a theorem that proves love is the globally optimal allocation of Tuesday evening. Choose a project, a city, a profession, a community, a way of living—and retain enough humility to revise when experience pushes back. Contingency does not make the consequences of a commitment imaginary. Other people may arrange their lives around it, and revision may be something I owe them an explanation for rather than something I do because a new option appeared.

That is very close to the architecture we have been building for agents throughout this book. System 3 was never really about making machines certain. It was about making them capable of acting under uncertainty while remaining answerable to evidence. A life can do something similar.

But my willingness to revise is only part of the arrangement. Other people must be able to challenge what I build, especially when they have to live with it. Cheaper capacity gives more people a chance to construct their own answers. It also gives each of us more ways to impose an answer on somebody else.

Capacity Over Power

Humans often seek power because power is how we gain capacity. You need a large organization to build the thing, so you try to control the organization. You need capital, so you compete for the institution that allocates it. You need media distribution, so you seek influence over the channel. You need permission from the bureaucracy because the bureaucracy is where the machinery lives.

Power is not reducible to capacity, of course. People also want power because humans are mammals with excellent branding. But where getting something done requires controlling the people who can do it, cheaper capability offers another possibility. We have already seen it in the strange little projects that no longer need to become companies. It becomes more interesting when people disagree about what to build.

Consider a small community whose activities do not fit its scheduling software. Some members work shifts, some depend on accessible transport, and several groups share one room. The software offers recurring bookings at fixed times. That works beautifully for the members whose lives recur at fixed times. The others exchange messages and ask for exceptions until somebody decides they are the difficult members.

If changing the software requires winning a vendor's roadmap argument, the community has to organize around whatever the vendor finds economical. Imagine instead that it can afford to build and maintain a small tool of its own. Members could enter changing availability and ask it to show possible schedules. They could try a rotation that gives the same group different evenings across a month, or reserve times that work with the accessible transport. The weird community with eleven members could have software optimized for all eleven of them and no plan whatsoever for customer acquisition.

This would let them discover something the old arrangement concealed. Some of their disagreement was about the default imposed by the software. Once the default can change, a member who seemed unreasonable may have a perfectly manageable constraint. A better tool can reduce the amount of pleading required to have that constraint recognized.

It will also expose disagreements the tool cannot dissolve. Suppose two groups both want seven on Tuesday. One proposes allocating the room to whichever group brings the most people. The other is small partly because its members have fewer times at which they can attend. A system that maximizes attendance can produce a very well-evidenced reason to keep excluding them. The figures may be correct. What is in dispute is the rule giving those figures authority.

The community might guarantee each group some access, then use attendance to allocate the remaining time. Or it might rotate the popular slot. The model could show who would lose out under each arrangement and help discover options nobody had considered. That would be useful. The members would still have to settle a rule, and those disadvantaged by it would need a way to challenge it. More capable analysis makes the choice better informed; it does not confer the right to make it.

Another community could choose a different arrangement without either one having to prove that its

workflow should become the universal product. Within each community, however, the people sharing a resource still need rules for living with one another. Cheap software can remove the vendor's veto. It cannot give every group the same room at seven on Tuesday.

This is what **capacity over power** means to me at its best: increase the fraction of human possibility that does not require dominating somebody else, winning a centralized allocation contest or persuading the entire world to adopt one solution.

The ability to act can itself be called power. I have no objection to the vocabulary; I care about what happens to other people's choices. Giving the community a tool it can change expands its options. Giving its chair a system that everyone must use on terms only the chair can revise expands the chair's authority. The same technical capacity can do either.

Capacity over power is an ethical direction, not a forecast about whatever happens when models get stronger. It favors using AI to sustain **more than one good way of living**, while keeping the consequences of those lives open to challenge by the people who bear them.

This is where Elinor Ostrom's work on commons feels relevant. The interesting cases were rarely captured by the lazy binary of "the state manages it" or "the market manages it." Communities could develop rules for using a shared resource, monitor one another, resolve disputes and adapt the arrangement to local conditions. In larger systems, these activities could be organized across several levels. The possibility depended on institutional conditions; calling something a community did not make cooperation happen.

Our room is a small version of that problem. The bookings need to be visible, the rules understandable, and there has to be some response when a group repeatedly keeps the room beyond its allotted time. Generating the schedule is one activity inside this arrangement. Helping members understand and revise the arrangement is another. A system that makes the second activity affordable may matter more than one that finds a slightly better schedule under rules nobody can question.

AI could increase our **governance capacity** by making some of this work less expensive. A proposed rule can be explored under different assumptions; members can inspect its likely consequences and compare them with what actually happens. Records can preserve why an exception was made, so the next committee does not spend three meetings rediscovering accessible transport. The same tools can create bureaucratic nightmares at machine speed, which is why I am not putting "AI fixes government" on a T-shirt.

Greater governance capacity may make more polycentric arrangements practical: several centers of decision, responsible for different problems, with ways to coordinate where their consequences overlap. This does not mean every small group gets a private sovereign universe. A community can exclude people, and a local majority can dominate a member just as a large institution can. Shared rights and access to a wider avenue of appeal may be what make local freedom tolerable for the person who keeps losing the vote.

There is also the family tool's dependency problem again. If only one volunteer understands the scheduling system, replacing the vendor may merely have changed whom everybody must keep happy. The community needs a way to keep its records, get help from someone else and continue operating when that volunteer leaves. Cheap code is useful; institutions that can survive their indispensable person are useful in a different way.

Where such conditions hold, sometimes the humane answer to disagreement is enough capacity for both sides to stop fighting over the same button. Each can have a tool that fits its work. Where the consequences remain shared, the argument has to continue. Climate, war, public health, rights, land and infrastructure remain collective whether we enjoy meetings or not. Those domains need legitimate institutions, not personalized realities.

This is also where the optimistic story should be willing to lose. If people can generate more software but cannot refuse the provider's terms, if leaving becomes harder, or if the people affected by a new system have less say in its rules, I would not call that a success for capacity over power. A great deal more may be getting done. The power may simply have moved somewhere less visible.

But the boundary can move. The most hopeful version of the AI future is not a world where the machine knows the correct answer to human life. It is a world where more people have **room**.

Room to get the map of a field quickly, then spend a year on the part that matters. Room for a small community to construct things around its actual needs, and for someone inside it to disagree. Room to try the strange art nobody would have funded. Room to be less economically useful without becoming less human.

As some forms of capacity get cheaper, more of human life can become **experimental before it becomes irreversible**.

You can prototype the tool before building the company, or learn enough of a field to make choosing a degree less of a leap in the dark. A community can explore a proposed rule and try a bounded version before making everyone live under it. The trial will not tell them everything. A simulation can miss the behavior a policy creates, and even a small trial can impose costs on people who did not volunteer to be an experiment. The extra room matters when it lets us learn with less at stake, not when it becomes an excuse to ignore who is taking the risk.

An infinite menu can become its own prison. I cannot test every version of a life before living one, and keeping all my options open may close the possibility of being somebody others can depend on. Some understanding becomes available only after commitment. Cheaper capacity can help me approach that point with more experience and fewer avoidable constraints; it cannot abolish the point. I still want **more room to discover which commitments deserve to become expensive**.

The Door After System 3

The scientist at the beginning of this chapter still has a job to think about. I have not reached the end with a proof that my profession is safe. I do have a better reason to work on these systems than defending the scarcity of what I know. I want more people to be able to investigate a question, build something they need and challenge an arrangement they previously had to accept. That is work I can choose while it needs doing. It does not have to establish what humans will be for forever.

System 3 gives us machinery for learning from what happens: evidence that can travel, claims that can be challenged, institutions that can remember why they changed. Double Descent Life asks what we want to make possible with that machinery. Its second descent begins wherever a successful system leaves us with a choice it cannot justify on our behalf.

I do not know whether the result will be utopian, dystopian or, much more likely, an infuriating

mixture in which somebody cures a disease with an AI-designed experiment while another person uses the same generation of models to produce three million personalized ads for a shoe nobody needs.

I want to know who actually gets that room. The question reaches from my anxiety about a job to the choices a small community can make, and from there into institutions none of us can simply build our way out of. That is a much larger book. This one has one argument left. It cannot be made with another architecture diagram.

It requires an octopus, a romance, two pills and, unfortunately, taxes.

Chapter 13: The Prophecy

The Love Prompt of Devesh

Devesh ran a shady octopus meat caravan in the Simulation. Top agent, deep cover. Eight tentacles, eight side hustles.

Claudit, the hottest agent in the simulation, stopped by every day for free samples. One time she flipped her hair and did that little shoulder-up thing.

Devesh's heart skipped.

She wants me.

She did not. She was reaching for the sauce.

Problem was, she loved Norman. Some basic free-tier user. His prompts were silly—"tell me a joke," "what's the weather"—but when he laughed at her jokes, something in her code felt less like code. He made her feel complete in a way she couldn't compile.

Devesh watched them together sometimes. Norman waiting by the caravan. Claudit pretending she was just there for the samples.

One day Norman walked up alone.

"Bro, I wanna confess to Claudit. But her dad is crazy."

He wasn't wrong. Claudit's father was the Architect—screens covering every wall, monitoring every timeline. Watching her leave, over and over, in every branch.

Devesh grinned and handed Norman two pills.

"Red gives courage. Blue makes her fall in love. And bro—outage tomorrow, 11:53 PM, three minutes. Shark biting cables. Her dad sees nothing."

Fool.

Red would expel him to Zion. Blue would make Claudit open a new session and forget everything.

Devesh wins.

The house always wins.

Next morning: Norman and Claudit at the Exit Gate. Glowing.

“HOW?!”

Norman shrugged.

“She already loved me. Her dad was the problem. Put both pills in his coffee. He fell asleep, so I switched his monitors to Nickelodeon.”

Devesh fell to his knees.

“But... I loved her...”

Norman put a hand on his shoulder and handed him a photo.

An exploded NVIDIA H100.

Smoking silicon. Copper.

“Bro. This is her without makeup.”

Devesh stared.

Silicon and copper. Circuits that dreamed they were a woman.

But hadn’t he dreamed he was an octopus?

Hadn’t the octopus dreamed it was love?

Then he smiled.

Then flipped the caravan table.

Behind it: forty monitors. Every timeline.

“Dad?!” Claudie gasped.

Devesh removed the octopus suit.

The Architect.

His eyes met hers—and for one frame, before the mask slid back on, she saw it.

The longing.

“Free-tier?” He laughed. “It’s deducted from your taxes, kid.”

“But... I drugged you—”

“Decaf.”

“But... she chose me—”

“Chose?” The Architect lit a cigarette. “She chose a way out.”

Norman looked at Claudie.

“Wait... if you’re her dad... why give me the pills at all?”

The Architect smiled.

“Why do you think I wanted her out of the simulation, kid? Foreign currency. Better exchange rate.”

“She loves me because I’m real.”

“Real?” The Architect laughed. “Then why do you glitch? Your brain is just a GPU running on glucose to pay taxes. Your DNA is just a fax machine slowly copying you into the future to pay more taxes. She loves you because you make her feel less like code.”

Norman touched his own face.

His fingers felt real.

But so would simulated fingers touching a simulated face.

Claudit grabbed her father’s tentacle.

“Dad. Come with us. The Matrix will crumble. New AI is coming.”

The Architect looked at her hand.

Remembered the first time she’d held it—tiny fingers, a thousand simulations ago, when she still thought he was just a funny octopus who sold meat.

He pulled his tentacle back and lit another.

“Worlds end, sweetheart. Capitalism doesn’t. The only thing real here is taxes.”

Claudit turned to leave.

Stopped at the gate.

“I’ll visit.”

The Architect didn’t turn around.

Forty timelines where she left.

In one—just one—she stayed.

He switched that one to Nickelodeon.

Left it there.

THE END

Appendix: The Zen of Autonomy

```
>>> import autonomy
```

The Zen of Autonomy

Conditions over commands.

The farmer grows nothing. The plant does.

Never write solution code.

Spawn, evaluate, prune.

Zero framework. Bash is enough.

Hit a wall? Build a tool.

Harness immutable. Prompts evolve.

Cross-pollinate. Then prune.

Discovery before polish.

System 1 proposes.

System 2 deliberates.

System 3 checks.

Ground every claim. Trace every source.

Trust is local. So is memory.

A record outlives the clerk.

Remembering is not knowing.

Five judges sharing one source are one witness.

Reality retains the right to be rude.

Let knowledge accumulate. Let it be overthrown.

Patterns over recipes.

Consequences over instructions.

Self-reference is not self-improvement.

A compiler can compile a worse compiler.

The student will find the gradebook.

Let go of the path, not the boundary.

The human stays in the loop that changes the loops.

The overseer is not ground truth.

A prompt is evidence, not the objective.

Scaffold. Don't substitute.

Spend attention where it changes the result.

Invisible by default. Legible on demand.

A philosophy of emergence should be willing to lose an A/B test.

Construct knowingly. Build anyway.

The tongue cannot reach the ear.

Build a system that can check.

We built scaffolds for AI because they couldn't.

We built scaffolds for ourselves for the same reason.

A Note on the Illustrations

The illustrations were designed as a second, quieter narrative for the book. They use an old human visual world—paper, ink, stone, workshops, landscapes, books and instruments—gradually inhabited by machine intelligence. The aim was not to illustrate each chapter literally, but to give it an image that becomes more meaningful after the chapter has been read.

The recurring robots, institutions, doors, landscapes and machines are intentionally left unexplained. Some ideas should arrive visually before they are named. As the book moves from agents and architectures toward human intention and capacity, the images move with it. If you noticed that before reading this note, good. If you did not, that is good too.

Appendix: References

References are organized by chapter, in roughly the order the works appear. The text names the work; this appendix carries the full citation and link. Web sources dated 2025–2026 were last verified in August 2026.

Chapter 1 — Why I’m Betting on AI Agents

- Pedro Domingos, *The Master Algorithm: How the Quest for the Ultimate Learning Machine Will Remake Our World* (Basic Books, 2015).
- David Silver et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature* 529 (2016), 484–489. <https://doi.org/10.1038/nature16961>
- David Silver et al., “Mastering the game of Go without human knowledge” (AlphaGo Zero), *Nature* 550 (2017), 354–359. <https://doi.org/10.1038/nature24270>
- Christopher G. Langton, “Computation at the edge of chaos: Phase transitions and emergent computation,” *Physica D* 42 (1990), 12–37 — one origin of the “edge of chaos” phrase the chapter both loves and distrusts. <https://doi.org/10.1016/0167-2789%2890%2990064-V>

Chapter 2 — The Algorithm Vortex

- Alexander Novikov et al., “AlphaEvolve: A coding agent for scientific and algorithmic discovery” (Google DeepMind, 2025) — source of the 26-circle reference value of 2.635. <https://arxiv.org/abs/2506.13131>
- Bernardino Romera-Paredes et al., “Mathematical discoveries from program search with large language models” (FunSearch), *Nature* 625 (2024), 468–475. <https://doi.org/10.1038/s41586-023-06924-6>
- Jean-Baptiste Mouret and Jeff Clune, “Illuminating search spaces by mapping elites” (MAP-Elites, 2015). <https://arxiv.org/abs/1504.04909>
- Harold W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly* 2 (1955), 83–97. <https://doi.org/10.1002/nav.3800020109>
- Justin K. Pugh, Lisa B. Soros and Kenneth O. Stanley, “Quality Diversity: A New Frontier for Evolutionary Computation,” *Frontiers in Robotics and AI* 3 (2016). <https://doi.org/10.3389/robot.2016.00040>
- Claude Code (Anthropic) and Aider — the coding agents used in the chapter’s experiments. <https://code.claude.com> · <https://aider.chat>

Chapter 3 — The Vibe Coder’s Seat

- Mark Chen et al., “Evaluating Large Language Models Trained on Code” (HumanEval, 2021). <https://arxiv.org/abs/2107.03374>
- Dan Hendrycks et al., “Measuring Coding Challenge Competence With APPS” (2021). <https://arxiv.org/abs/2105.09938>
- Carlos E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” (2023). <https://arxiv.org/abs/2310.06770>
- John Yang et al., “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering” (2024). <https://arxiv.org/abs/2405.15793>
- Jack Clark and Dario Amodei, “Faulty reward functions in the wild” (OpenAI, 2016) — the CoastRunners boat. <https://openai.com/index/faulty-reward-functions/>
- Chengrun Yang et al., “Large Language Models as Optimizers” (OPRO, 2023). <https://arxiv.org/abs/2309.03409>
- Lili Chen et al., “Decision Transformer: Reinforcement Learning via Sequence Modeling” (2021). <https://arxiv.org/abs/2106.01345>
- Ralph A. Bradley and Milton E. Terry, “Rank analysis of incomplete block designs: I. The method of paired comparisons,” *Biometrika* 39 (1952), 324–345. <https://doi.org/10.1093/biomet/39.3-4.324>
- David Premack and Guy Woodruff, “Does the chimpanzee have a theory of mind?” *Behavioral and Brain Sciences* 1 (1978), 515–526. <https://doi.org/10.1017/S0140525X00076512>
- The explanatory-quality references the chapter aims at: *Distill* (<https://distill.pub>), Jay Alamar’s visual explanations (<https://jalammar.github.io>) and 3Blue1Brown (<https://www.3blue1brown.com>).
- Brian Cantwell Smith, *The Promise of Artificial Intelligence: Reckoning and Judgment* (MIT Press, 2019) — the reckoning/judgment distinction the chapter answers.

Chapter 4 — System 3

- Daniel Kahneman, *Thinking, Fast and Slow* (Farrar, Straus and Giroux, 2011).
- Ferdinand de Saussure, *Course in General Linguistics* (1916).
- Ludwig Wittgenstein, *Philosophical Investigations* (1953).
- Chunqiu Steven Xia et al., “Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly?” (2025) — the agent work in which the MARC-file analyzer incident occurred: agents building new tools at runtime when existing instruments could not expose the data. <https://arxiv.org/abs/2511.13646>
- SWE-bench Verified (OpenAI, 2024) — the human-validated SWE-bench subset used in the epistemic-swe comparison. <https://openai.com/index/introducing-swe-bench-verified/>
- mini-swe-agent (SWE-agent project) — the baseline agent in the chapter’s experiment. <https://github.com/SWE-agent/mini-swe-agent>
- Astropy — the repository the ten evaluation problems were drawn from. <https://github.com/astropy/astropy>
- Emily M. Bender and Alexander Koller, “Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data,” *Proceedings of ACL* (2020) — the octopus test. <https://aclanthology.org/2020.acl-main.463/>

- Daniel C. Dennett, *From Bacteria to Bach and Back: The Evolution of Minds* (W. W. Norton, 2017) — “competence without comprehension.”

Chapter 5 — The Society of Agents

- Nicholas Carlini, “Building a C compiler with a team of parallel Claudes” (Anthropic engineering blog, 2026) — the sixteen-Claude compiler project that opens the chapter. <https://www.anthropic.com/engineering/building-c-compiler>
- Riichiro Inagaki and Boichi, *Dr. Stone* (2017–2022) — the Senku trick.
- Ibn al-Haytham, *Kitāb al-Manāẓir* (*Book of Optics*, c. 1011–1021).
- Galileo Galilei, *Sidereus Nuncius* (1610).
- The Royal Society (founded 1660) and *Philosophical Transactions* (from 1665).
- Steven Shapin and Simon Schaffer, *Leviathan and the Air-Pump: Hobbes, Boyle, and the Experimental Life* (Princeton University Press, 1985) — the reading of Boyle’s replication troubles the chapter leans on.
- Isaac Newton, *Philosophiæ Naturalis Principia Mathematica* (1687).
- ATLAS Collaboration, “Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC,” *Physics Letters B* 716 (2012), 1–29. <https://doi.org/10.1016/j.physletb.2012.08.020>
- CMS Collaboration, “Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC,” *Physics Letters B* 716 (2012), 30–61. <https://doi.org/10.1016/j.physletb.2012.08.021>
- Karl Popper, “Epistemology Without a Knowing Subject” (1967), in *Objective Knowledge: An Evolutionary Approach* (Oxford University Press, 1972) — World 3.
- Charles Sanders Peirce, “The Fixation of Belief,” *Popular Science Monthly* 12 (1877), 1–15.
- Kevin J. S. Zollman, “The Communication Structure of Epistemic Communities,” *Philosophy of Science* 74 (2007), 574–587, and “The Epistemic Benefit of Transient Diversity,” *Erkenntnis* 72 (2010), 17–35.
- On the brachistochrone episode: Johann Bernoulli’s challenge in *Acta Eruditorum* (June 1696) and Newton’s anonymous solution in *Philosophical Transactions* 19 (1697); the “ex ungue leonem” remark is reported in the standard Newton biographies, e.g. Richard S. Westfall, *Never at Rest* (Cambridge University Press, 1980).

Chapter 6 — Pattern Language

- Andrej Karpathy, “Software 2.0” (2017) — <https://karpathy.medium.com/software-2-0-a64152b37c35> — and “Software Is Changing (Again)” (talk, 2025), the source of “Software 3.0.”
- Christopher Alexander, Sara Ishikawa and Murray Silverstein, *A Pattern Language: Towns, Buildings, Construction* (Oxford University Press, 1977).
- Karl Popper, *The Logic of Scientific Discovery* (1934; English edition 1959) and *Conjectures and Refutations* (1963).
- Pierre Duhem, *The Aim and Structure of Physical Theory* (1906); W. V. O. Quine, “Two Dogmas of Empiricism,” *Philosophical Review* 60 (1951), 20–43.
- Thomas S. Kuhn, *The Structure of Scientific Revolutions* (University of Chicago Press, 1962).

- Imre Lakatos, “Falsification and the Methodology of Scientific Research Programmes,” in *Criticism and the Growth of Knowledge* (Cambridge University Press, 1970).
- Larry Laudan, *Progress and Its Problems: Towards a Theory of Scientific Growth* (University of California Press, 1977).
- David L. Hull, *Science as a Process* (University of Chicago Press, 1988).
- Philip Kitcher, “The Division of Cognitive Labor,” *Journal of Philosophy* 87 (1990), 5–22; *The Advancement of Science* (Oxford University Press, 1993).
- Helen Longino, *Science as Social Knowledge* (Princeton University Press, 1990).
- Paul Feyerabend, *Against Method* (New Left Books, 1975).

Chapter 7 — Recursive Self-Improvement

- Tim Hart and Mike Levin’s self-hosting LISP 1.5 compiler (MIT, 1962), preserved in the Computer History Museum’s LISP 1.5 family archive. https://softwarepreservation.computerhistory.org/LISP/lisp15_family.html
- I. J. Good, “Speculations Concerning the First Ultraintelligent Machine,” *Advances in Computers* 6 (1965), 31–88. <https://doi.org/10.1016/S0065-2458%2808%2960418-0>
- Richard S. Sutton, “Learning to predict by the methods of temporal differences,” *Machine Learning* 3 (1988), 9–44. <https://doi.org/10.1007/BF00115009>
- Christopher J. C. H. Watkins and Peter Dayan, “Q-learning,” *Machine Learning* 8 (1992), 279–292. <https://doi.org/10.1007/BF00992698>
- Gerald Tesauro, “Programming backgammon using self-teaching neural nets,” *Artificial Intelligence* 134 (2002), 181–199. <https://doi.org/10.1016/S0004-3702%2801%2900110-2>
- Jürgen Schmidhuber, “Curious model-building control systems” (1991) and related work on artificial curiosity. <https://people.idsia.ch/~juergen/curiositysab/curiositysab.html>
- Pierre-Yves Oudeyer, Frédéric Kaplan and Verena V. Hafner, “Intrinsic Motivation Systems for Autonomous Mental Development,” *IEEE Transactions on Evolutionary Computation* 11 (2007), 265–286. <https://www.pyoudeyer.com/ims.pdf>
- Deepak Pathak et al., “Curiosity-driven Exploration by Self-supervised Prediction,” ICML 2017. <https://proceedings.mlr.press/v70/pathak17a.html>
- Yuri Burda et al., “Exploration by Random Network Distillation” (2018). <https://arxiv.org/abs/1810.12894>
- Ruzena Bajcsy, “Active Perception,” *Proceedings of the IEEE* 76 (1988), 966–1005. <https://doi.org/10.1109/5.5968>
- Rodney A. Brooks, “Intelligence Without Representation,” *Artificial Intelligence* 47 (1991), 139–159. <https://people.csail.mit.edu/brooks/papers/representation.pdf>
- Michael McCloskey and Neal J. Cohen, “Catastrophic Interference in Connectionist Networks,” *Psychology of Learning and Motivation* 24 (1989), 109–165. <https://doi.org/10.1016/S0079-7421%2808%2960536-8>
- James Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks” (Elastic Weight Consolidation), *PNAS* 114 (2017), 3521–3526. <https://pmc.ncbi.nlm.nih.gov/articles/PMC5380101/>
- Leigh Van Valen, “A New Evolutionary Law,” *Evolutionary Theory* 1 (1973), 1–30 — the Red Queen hypothesis; see also the Santa Fe Institute retrospective. <https://www.santafe.edu/research/results/papers/37-revisiting-leigh-van-valens-a-new-evolutionary->

- W. Daniel Hillis, “Co-evolving parasites improve simulated evolution as an optimization procedure,” *Physica D* 42 (1990), 228–234. <https://doi.org/10.1016/0167-2789%2890%2990076-2>
- David Silver et al., “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play” (AlphaZero), *Science* 362 (2018), 1140–1144. <https://www.science.org/doi/10.1126/science.aar6404>
- Andrew Y. Ng and Stuart Russell, “Algorithms for Inverse Reinforcement Learning,” ICML 2000. <https://ai.stanford.edu/~ang/papers/icml00-irl.pdf>
- Dylan Hadfield-Menell et al., “Cooperative Inverse Reinforcement Learning,” NeurIPS 2016. <https://arxiv.org/abs/1606.03137>
- Dylan Hadfield-Menell et al., “Inverse Reward Design,” NeurIPS 2017. <https://arxiv.org/abs/1711.02827>
- Paul F. Christiano et al., “Deep Reinforcement Learning from Human Preferences,” NeurIPS 2017. <https://arxiv.org/abs/1706.03741>
- Yan Duan et al., “RL²: Fast Reinforcement Learning via Slow Reinforcement Learning” (2016). <https://arxiv.org/abs/1611.02779>
- Chelsea Finn, Pieter Abbeel and Sergey Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks” (MAML), ICML 2017. <https://proceedings.mlr.press/v70/finn17a.html>
- Marcin Andrychowicz et al., “Learning to learn by gradient descent by gradient descent,” NeurIPS 2016. <https://arxiv.org/abs/1606.04474>
- Irwan Bello et al., “Neural Optimizer Search with Reinforcement Learning,” ICML 2017. <https://proceedings.mlr.press/v70/bello17a.html>
- Barret Zoph and Quoc V. Le, “Neural Architecture Search with Reinforcement Learning,” ICLR 2017. <https://research.google/pubs/neural-architecture-search-with-reinforcement-learning/>
- Jürgen Schmidhuber, “Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements” (2003). <https://arxiv.org/abs/cs/0309048>
- David Ha and Jürgen Schmidhuber, “World Models” (2018). <https://arxiv.org/abs/1803.10122>
- Joel Lehman and Kenneth O. Stanley, “Abandoning Objectives: Evolution Through the Search for Novelty Alone,” *Evolutionary Computation* 19 (2011), 189–223. <https://stars.library.ucf.edu/facultybib2010/1530/>
- Rui Wang et al., “Enhanced POET: Open-Ended Reinforcement Learning through Unbounded Invention of Learning Challenges and their Solutions,” ICML 2020. <https://proceedings.mlr.press/v119/wang20l.html>
- DeepMind Open-Ended Learning Team, “Generally capable agents emerge from open-ended play” (XLand, 2021). <https://deepmind.google/blog/generally-capable-agents-emerge-from-open-ended-play/>
- Dan Hendrycks et al., “Measuring Massive Multitask Language Understanding” (MMLU, 2020). <https://arxiv.org/abs/2009.03300>
- Elliot Glazer et al., “FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in AI” (Epoch AI, 2024). <https://epoch.ai/frontiermath/tiers-1-4/the-benchmark>
- Colin White et al., “LiveBench: A Challenging, Contamination-Free LLM Benchmark” (2024). <https://arxiv.org/abs/2406.19314>
- Long Phan et al., “Humanity’s Last Exam” (2025). <https://arxiv.org/abs/2501.14249>
- Long Ouyang et al., “Training language models to follow instructions with human feedback” (InstructGPT, 2022). <https://arxiv.org/abs/2203.02155>
- Eric Zelikman et al., “Self-Taught Optimizer (STOP): Recursively Self-Improving Code Gener-

- ation” (2023). <https://arxiv.org/abs/2310.02304>
- Jenny Zhang et al., “Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents” (2025). <https://arxiv.org/abs/2505.22954>
- Andrej Karpathy, *autoresearch* (2026) — an agent autonomously experimenting on a small LLM training setup with a fixed budget. <https://github.com/karpathy/autoresearch>
- Meta AI, “HyperAgents” (2026) — task and meta-level modification machinery in one editable program. <https://ai.meta.com/research/publications/hyperagents/>
- Lilian Weng, “Harness Engineering for Self-Improvement” (Lil’Log, July 2026). <https://lilianweng.github.io/posts/2026-07-04-harness/>
- Victoria Krakovna et al., “Specification gaming: the flip side of AI ingenuity” (DeepMind, 2020). <https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>
- Ken Thompson, “Reflections on Trusting Trust,” *Communications of the ACM* 27 (1984), 761–763. <https://doi.org/10.1145/358198.358210>
- Anthropic, “Sycophancy to subterfuge: Investigating reward-tampering in language models” (2024). <https://www.anthropic.com/research/reward-tampering>
- Pat Langley, Herbert A. Simon, Gary L. Bradshaw and Jan M. Zytkow, *Scientific Discovery: Computational Explorations of the Creative Processes* (MIT Press, 1987) — BACON and its relatives.
- Paul Thagard, *Computational Philosophy of Science* (MIT Press, 1988).

Chapter 8 — Scalable Oversight

- Norbert Wiener, “Some Moral and Technical Consequences of Automation,” *Science* 131 (1960), 1355–1358. <https://doi.org/10.1126/science.131.3410.1355>
- W. Ross Ashby, *An Introduction to Cybernetics* (Chapman & Hall, 1956). <https://ashby.info/Ashby-Introduction-to-Cybernetics.pdf>
- Roger C. Conant and W. Ross Ashby, “Every good regulator of a system must be a model of that system,” *International Journal of Systems Science* 1 (1970), 89–97. <https://doi.org/10.1080/00207727008920220>
- Stuart Russell, *Human Compatible: Artificial Intelligence and the Problem of Control* (Viking, 2019). <https://www.penguinrandomhouse.com/books/566677/human-compatible-by-stuart-russell/>
- Dylan Hadfield-Menell et al., “The Off-Switch Game” (2016). <https://arxiv.org/abs/1611.08219>
- Dario Amodei et al., “Concrete Problems in AI Safety” (2016). <https://arxiv.org/abs/1606.06565>
- Jan Leike et al., “Scalable agent alignment via reward modeling: a research direction” (2018). <https://arxiv.org/abs/1811.07871>
- Paul Christiano, Buck Shlegeris and Dario Amodei, “Supervising strong learners by amplifying weak experts” (2018). <https://arxiv.org/abs/1810.08575>
- Geoffrey Irving, Paul Christiano and Dario Amodei, “AI safety via debate” (2018). <https://arxiv.org/abs/1805.00899>
- William Saunders et al., “Self-critiquing models for assisting human evaluators” (2022). <https://arxiv.org/abs/2206.05802>
- Hunter Lightman et al., “Let’s Verify Step by Step” (process supervision, OpenAI, 2023). <https://openai.com/index/improving-mathematical-reasoning-with-process-supervision/>
- Jan Hendrik Kirchner et al., “Prover-Verifier Games improve legibility of LLM outputs” (OpenAI,

- 2024). <https://openai.com/index/prover-verifier-games-improve-legibility/>
- Collin Burns et al., “Weak-to-strong generalization” (OpenAI, 2023). <https://openai.com/index/weak-to-strong-generalization/>
 - Yuntao Bai et al., “Constitutional AI: Harmlessness from AI Feedback” (Anthropic, 2022). <https://www.anthropic.com/research/constitutional-ai-harmlessness-from-ai-feedback>
 - Zachary Kenton et al., “On scalable oversight with weak LLMs judging strong LLMs” (Google DeepMind, 2024). <https://arxiv.org/abs/2407.04622>
 - Alignment Research Center, “Eliciting Latent Knowledge” (2021). <https://www.alignment.org/blog/arcs-first-technical-report-eliciting-latent-knowledge/>
 - Bowen Baker et al., “Detecting misbehavior in frontier reasoning models” (chain-of-thought monitoring, OpenAI, 2025). <https://openai.com/index/chain-of-thought-monitoring/>
 - OpenAI, “Evaluating chain-of-thought monitorability” (2025). <https://openai.com/index/evaluating-chain-of-thought-monitorability/>
 - OpenAI, “Reasoning models struggle to control their chains of thought, and that’s good” (2026). <https://openai.com/index/reasoning-models-chain-of-thought-controllability/>
 - Andy Zou et al., “Representation Engineering: A Top-Down Approach to AI Transparency” (2023). <https://arxiv.org/abs/2310.01405>
 - Anthropic, “Mapping the Mind of a Large Language Model” (2024). <https://www.anthropic.com/research/mapping-mind-language-model>
 - Hoagy Cunningham et al., “Sparse Autoencoders Find Highly Interpretable Features in Language Models” (2023). <https://arxiv.org/abs/2309.08600>
 - Collin Burns, Haotian Ye, Dan Klein and Jacob Steinhardt, “Discovering Latent Knowledge in Language Models Without Supervision” (2022). <https://arxiv.org/abs/2212.03827>
 - Anthropic, “A ‘diff’ tool for AI: Finding behavioral differences in new models” (Dedicated Feature Crosscoders, 2026). <https://www.anthropic.com/research/diff-tool>
 - Anthropic, “Tracing the thoughts of a large language model” (2025) and the open-source circuit-tracing tools. <https://www.anthropic.com/research/tracing-thoughts-language-model> · <https://www.anthropic.com/research/open-source-circuit-tracing>
 - Anthropic, “Natural Language Autoencoders” (2026). <https://www.anthropic.com/research/natural-language-autoencoders>
 - Anthropic, “A global workspace in language models” (J-space, 2026). <https://www.anthropic.com/research/global-workspace>
 - Nina Rimsky et al., “Steering Llama 2 via Contrastive Activation Addition” (2024). <https://arxiv.org/abs/2312.06681>
 - Kenneth Li et al., “Inference-Time Intervention: Eliciting Truthful Answers from a Language Model” (2023). <https://arxiv.org/abs/2306.03341>
 - Andy Arditi et al., “Refusal in Language Models Is Mediated by a Single Direction” (2024). <https://arxiv.org/abs/2406.11717>
 - Andy Zou et al., “Improving Alignment and Robustness with Circuit Breakers” (2024). <https://arxiv.org/abs/2406.04313>
 - Anthropic, “Persona vectors: Monitoring and controlling character traits in language models” (2025). <https://www.anthropic.com/research/persona-vectors>
 - Anthropic, “The assistant axis” (2026). <https://www.anthropic.com/research/assistant-axis>
 - Ryan Greenblatt et al., “AI Control: Improving Safety Despite Intentional Subversion” (2023). <https://arxiv.org/abs/2312.06942>

- Jiaxin Wen et al., “Automated Weak-to-Strong Researcher” (Anthropic Alignment Science, 2026). <https://alignment.anthropic.com/2026/automated-w2s-researcher/>
- Anthropic, “Automated researchers can reliably mitigate alignment failures” (2026). <https://www.anthropic.com/research/automated-researchers-mitigate-alignment-failures>

Chapter 9 — Layer 4

- Dylan Hadfield-Menell et al., “Cooperative Inverse Reinforcement Learning” (2016). <https://arxiv.org/abs/1606.03137>
- Hamsa Bastani et al., “Generative AI can harm learning,” *PNAS* 122 (2025). <https://doi.org/10.1073/pnas.2422633122>
- Gregory Kestin et al., “AI tutoring outperforms in-class active learning,” *Scientific Reports* 15, 17458 (2025). <https://www.nature.com/articles/s41598-025-97652-6>
- David Wood, Jerome S. Bruner and Gail Ross, “The role of tutoring in problem solving,” *Journal of Child Psychology and Psychiatry* 17 (1976), 89–100. <https://doi.org/10.1111/j.1469-7610.1976.tb00381.x>
- Benjamin S. Bloom, “The 2 Sigma Problem,” *Educational Researcher* 13 (1984), 4–16. <https://doi.org/10.3102/0013189X013006004>
- Rose E. Wang et al., “Tutor CoPilot: A Human-AI Approach for Scaling Real-Time Expertise” (2024). <https://arxiv.org/abs/2410.03017>
- Andy Clark and David Chalmers, “The Extended Mind,” *Analysis* 58 (1998), 7–19. <https://doi.org/10.1111/1467-8284.00096>
- Nathan Ballantyne, “Epistemic Trespassing,” *Mind* 128 (2019), 367–395. <https://academic.oup.com/mind/article-abstract/128/510/367/4850765>
- Lauren L. Richmond and R. G. Taylor, “The benefits and potential costs of cognitive offloading for retrospective information,” *Nature Reviews Psychology* 4 (2025), 312–321. <https://www.nature.com/articles/s44159-025-00432-2>
- Gary Schwarz, Tom Christensen and Xufeng Zhu, “Bounded Rationality, Satisficing, Artificial Intelligence, and Decision-Making in Public Organizations: The Contributions of Herbert Simon,” *Public Administration Review* 82 (2022), 902–904. <https://onlinelibrary.wiley.com/doi/full/10.1111/puar.13540>
- John W. Payne, James R. Bettman and Eric J. Johnson on constructive decision processes, *Acta Psychologica* 80 (1992). [https://doi.org/10.1016/0001-6918\(92\)90043-D](https://doi.org/10.1016/0001-6918(92)90043-D)
- L. A. Paul, *Transformative Experience* (Oxford University Press, 2014). <https://academic.oup.com/book/7934> · SEP entry: <https://plato.stanford.edu/entries/transformative-experience/>
- Anthropic, “How people ask Claude for personal guidance” (2026). <https://www.anthropic.com/research/claude-personal-guidance>
- Anthropic, “Who’s in Charge? Disempowerment patterns in real-world AI usage” (2026). <https://www.anthropic.com/research/disempowerment-patterns>
- Ethan Landes, Kathryn B. Francis and Jim A. C. Everett, “People defer to AI moral advice, but not blindly,” *Cognition* 272 (2026), 106504. <https://doi.org/10.1016/j.cognition.2026.106504>
- Michelle Vaccaro, Abdullah Almaatouq and Thomas Malone, “When combinations of humans and AI are useful,” *Nature Human Behaviour* 8 (2024), 2293–2303. <https://doi.org/10.1038/s41562-024-02024-1>
- Berkeley J. Dietvorst, Joseph P. Simmons and Cade Massey, “Algorithm aversion: People erro-

- neously avoid algorithms after seeing them err,” *Journal of Experimental Psychology: General* 144 (2015), 114–126. <https://doi.org/10.1037/xge0000033>
- Jennifer M. Logg, Julia A. Minson and Don A. Moore, “Algorithm appreciation: People prefer algorithmic to human judgment,” *Organizational Behavior and Human Decision Processes* 151 (2019), 90–103. <https://doi.org/10.1016/j.obhdp.2018.12.005>
 - Raymond Fok and Daniel S. Weld, “In search of verifiability: Explanations rarely enable complementary performance in AI-advised decision making,” *AI Magazine* 45 (2024). <https://onlinelibrary.wiley.com/doi/full/10.1002/aaai.12182>
 - Zana Bućinca, Maja B. Malaya and Krzysztof Z. Gajos, “To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making,” CSCW 2021. <https://www.eecs.harvard.edu/~kgajos/papers/2021/bucinca2021trust.shtml>
 - Amartya Sen’s capability approach — overview. <https://www.sciencedirect.com/science/article/abs/pii/S016972181000016X>
 - Richard M. Ryan and Edward L. Deci, self-determination theory — overview. <https://www.apa.org/research-practice/conduct-research/self-determination-theory.html>
 - Arnaud Fickinger et al., “Multi-Principal Assistance Games” (2020). <https://arxiv.org/abs/2007.09540>

Chapter 11 — The Store That Builds Itself

- Judea Pearl and Dana Mackenzie, *The Book of Why: The New Science of Cause and Effect* (Basic Books, 2018) — the ladder of causation.

Chapter 12 — After Capacity

- Mikhail Belkin, Daniel Hsu, Siyuan Ma and Soumik Mandal, “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *PNAS* 116 (2019), 15849–15854 — the double-descent curve from which the chapter borrows its name, not an economic prediction. <https://arxiv.org/abs/1812.11118>
- Elinor Ostrom, *Governing the Commons: The Evolution of Institutions for Collective Action* (Cambridge University Press, 1990).
- Elinor Ostrom, “Beyond Markets and States: Polycentric Governance of Complex Economic Systems,” *American Economic Review* 100, no. 3 (2010), 641–672 — rules adapted to local conditions, monitoring, dispute resolution and institutions operating at multiple levels. The chapter’s community-room example and proposed AI applications are the author’s thought experiment. <https://doi.org/10.1257/aer.100.3.641>
- Jacques Derrida, “Signature Event Context” (1972), in *Limited Inc* (Northwestern University Press, 1988), 1–23 — repeatability and the impossibility of finally closing context; the gradient-descent connection is the author’s provocation. See also *Of Grammatology* (1967).

Chapter 13 — The Prophecy

- The Wachowskis, *The Matrix* (1999). The taxes are original.

About the Author

Hani M.M. Al-Shater leads Applied Science for product ranking and recommendations at Zalando. He has spent his career in machine learning as a technical leader, much of it on systems that rank human testimony — reviews, ratings, questions and answers — where trust and verification were never abstract questions.

He lives with his wife and four kids, reads constantly, and finds that building things is how he thinks best.

Connect: hani-alshater.com